

Projet de fin d'études

Victor BONNIN – Joris CASADAVANT

L'intégralité de notre projet est disponible à l'adresse suivante : github.com/VictorBonnin/projet_final.

Table des matières

Présentation de notre équipe (page 3)

Introduction (page 4)

Présentation de notre jeu de données (pages 5 et 6)

Bloc 2 – Concevoir et déployer une solution de traitement des données (pages 7 à 18)

1. Architecture distribuée : Kafka, Spark, MongoDB
2. Fonctionnement des composants : Producer, Kafka, Spark, MongoDB, Streamlit, PostgreSQL
3. Intégration continue : GitHub Actions, fichiers de test
4. Logs et monitoring
5. Résumé des compétences validées

Bloc 1 – Concevoir et développer une architecture de stockage de données (pages 19 à 49)

1. Mise en place de PostgreSQL (modèle relationnel, gold_layer)
2. SQL : création de tables et vues métiers
3. Mise en place de MongoDB (base NoSQL)
4. Utilisation de Redis (clé-valeurs)
5. Utilisation de Parquet (base orientée colonne)
6. Utilisation de Neo4j (base orientée graphe)
7. Datalake (architecture globale)
8. Une API Web RESTful fonctionnelle et présentation de l'API

Bloc 3 – Implémenter et optimiser des solutions de stockage et de traitement de données sur le cloud (pages 50 à 62)

1. Projet Pollution / Population / Météo
2. Architecture Bronze / Silver / Gold avec Microsoft Fabric
3. Nettoyage, normalisation et jointures des données
4. Intégration dans Lakehouse Fabric
5. Modèle conceptuel et sémantique
6. Mise en place de SCD1 et SCD2
7. Visualisations dans Power BI
8. Sécurité et conformité
9. Résumé des compétences validées

Bloc 4 – Implémenter des méthodes d'intelligence artificielle (pages 51 à 89)

- 1. Préparation des données**
 - A. Extraction depuis PostgreSQL**
 - B. Nettoyage, segmentation des données (par gamme de prix)**
 - C. Traitement des valeurs aberrantes**
 - D. Analyse exploratoire**
- 2. Modélisation prédictive**
 - A. Régression linéaire, Ridge, Random Forest, Gradient Boosting, XGBoost, LightGBM**
 - B. Feature engineering**
 - C. Entraînement et comparaison des modèles**
 - D. Résultats, scores et interprétabilité**
 - E. Déploiement des modèles**

Bloc 5 – Concevoir une stratégie de management et de gouvernance de données pour transformer les données en informations créatrices de valeur (pages 90 à 106)

- 1. Politique de gouvernance des données**
- 2. Catalogue de données et métadonnées**
- 3. Qualité des données et indicateurs associés**
- 4. Sécurité, conformité et RGPD**
- 5. Traçabilité et cycle de vie des données**
- 6. Documentation technique et fonctionnelle**
- 7. Auditabilité et gouvernance opérationnelle**
- 8. Résumé des compétences validées**

Présentation de notre équipe :

Je m'appelle Victor BONNIN, je suis actuellement étudiant à l'Efrei en alternance chez SFR. Je poursuis mon cursus dans le domaine de l'informatique et du numérique, avec un intérêt particulier pour l'analyse de données. Je suis en alternance chez SFR, en tant que chef de projet Data. Passionné d'informatique, j'adore travailler dans un mode en constante évolution car cela me pousse à toujours me donner à 100%.



Je m'appelle Joris CASADAVANT je suis étudiant au sein de l'EFREI et en alternance chez Micropole depuis trois ans. C'est une Entreprise de Services du Numérique (ESN) spécialisée dans la data, le cloud et la transformation digitale. Présente en Europe, elle accompagne de grands clients dans la mise en œuvre de projets innovants. Au sein du pôle Data Experience, j'ai eu l'opportunité de travailler sur des missions concrètes pour des clients dans le milieu du luxe. Cette expérience m'a permis de développer mes compétences techniques, de collaborer avec des équipes expertes, et d'évoluer dans un environnement dynamique et stimulant.

Introduction :

Dans le cadre de notre projet de fin d'études, nous avons choisi de centrer notre travail autour d'un jeu de données commun, structuré à partir des fichiers de demandes de valeurs foncières mis à disposition par le gouvernement français.

Ce choix nous a permis de construire une cohérence technique et analytique entre les différents blocs du projet, tout en nous confrontant à un volume de données massives représentatif des enjeux actuels de la data. L'objectif principal de ce projet était de mettre en œuvre une architecture complète de traitement de données, depuis l'ingestion en temps réel jusqu'à l'analyse décisionnelle et la modélisation prédictive. Pour cela, nous avons mobilisé des technologies modernes telles qu'Apache Kafka, Spark Structured Streaming, MongoDB, PostgreSQL, Redis ou encore Neo4j, en suivant une logique de pipeline orientée vers la scalabilité, la résilience et la modularité.

Notre rapport présente donc l'ensemble des étapes techniques et méthodologiques que nous avons mises en œuvre : depuis l'analyse des données brutes jusqu'à la création d'un data lake structuré, en passant par la visualisation, le stockage relationnel et non-relationnel, le déploiement d'une API sécurisée et l'application de modèles d'intelligence artificielle pour la prédiction de valeurs foncières. Notre démarche s'inscrit ainsi dans une logique professionnelle de bout en bout, à la croisée des compétences en data engineering, cloud computing, machine learning, BI et gouvernance des données.

Nous allons donc dans ce rapport, commencer par vous présenter le bloc 2, à savoir la création d'un streaming de données, pour pouvoir ensuite voir le bloc 1, qui contiendra notre architecture (base de données). Ensuite, nous vous présenteront le bloc 3, qui se concentrera sur la partie cloud et BI. Nous verrons après cela le bloc 4, avec l'entraînement d'un modèle prédictif sur nos données stockées précédemment. Pour finir, nous verrons le bloc 5, à savoir la partie sur la gouvernance des données.

Présentation de notre jeu de données :

Notre projet va tourner principalement autour du même jeu de données (blocs 1,2 et 4). En effet, nous avons décidé qu'il était plus intéressant, pour la lecture et la compréhension de tous nos projets, d'avoir une structure commune.

Nous allons donc commencer par vous présenter les données brutes. Nous avons récupéré ces dernières sur un site du gouvernement, à l'adresse suivante :

<https://www.data.gouv.fr/fr/datasets/demandes-de-valeurs-foncieres/>

En bas de cette page, nous avons différents fichiers contenant des données immobilières pour les années 2020 à 2024. Ce sont ces fichiers que nous allons utiliser.

Structure des données :

Chaque ligne du fichier représente une mutation immobilière, c'est-à-dire une opération d'achat/vente, et contient de nombreuses informations réparties en plusieurs catégories :

- 1. Informations sur les transactions :
 - Date mutation : date de la vente
 - Nature mutation : type d'opération (ex. : Vente)
 - Valeur foncière : montant de la transaction en euros
- 2. Localisation des biens :
 - Commune, Code postal, Code département
 - Voie, Type de voie, No voie
- 3. Caractéristiques des biens :
 - Type local : Maison, Appartement, Dépendance, etc.
 - Surface réelle bâtie : surface habitable bâtie (en m²)
 - Nombre pièces principales, surface terrain
- 4. Découpage cadastral :
 - Section, No plan, No Volume, etc.
 - Nombre de lots, Surface carrée des lots

Les données sont donc très complètes. Pour nous donner un ordre d'idée, nous avons fait tourner un petit script nous permettant de compter le nombre de lignes de chaque fichier brut :

```
1      ✓ Fin de traitement du fichier : ValeursFoncieres-2020.txt
2      📊 Total lignes envoyées : 3522416
3
4      ✓ Fin de traitement du fichier : ValeursFoncieres-2021.txt
5      📊 Total lignes envoyées : 4673720
6
7      ✓ Fin de traitement du fichier : ValeursFoncieres-2022.txt
8      📊 Total lignes envoyées : 4671911
9
10     ✓ Fin de traitement du fichier : ValeursFoncieres-2023.txt
11     📊 Total lignes envoyées : 3806978
12
13     ✓ Fin de traitement du fichier : ValeursFoncieres-2024.txt
14     📊 Total lignes envoyées : 3458643
15
16     -----
17     3522416 + 4673720 + 4671911 + 3806978 + 3458643 =
18
19     📊 20 133 668 lignes
```

Nombre de lignes de chaque fichier

Maintenant que nous vous avons présenté les données, nous allons pouvoir attaquer les différents projets.

BLOC 2 – Concevoir et déployer une solution de traitement des données

Objectifs du bloc :

B2	Concevoir en s'appuyant sur une veille technologique et mettre en œuvre une architecture distribuée répondant au besoin du client pour traiter les données massives en entreprise en utilisant les technologies de traitement
	Implémenter un système distribué en utilisant des technologies de streaming identifiées à partir d'une veille pour traiter des données sur une période précise ou en temps quasi réel
	Transformer les données provenant de différentes sources en prenant en compte la variété de données pour faire de l'analytique à échelle (intégration, formatage, manipulation, stockage données multidimensionnelles)
	Optimiser la performance des pipelines en utilisant les techniques d'intégration et de mise en scène adéquates pour le traitement des données massives
	Automatiser la création, les tests, l'intégration et le déploiement des pipelines de données en s'appuyant sur une veille technologique qui permet d'identifier et de mobiliser les solutions pour maximiser l'efficacité et réduire le 'time to market' tout en utilisant les technologies de containerisation et d'ordonnancement.

Une architecture de traitement distribué de données :

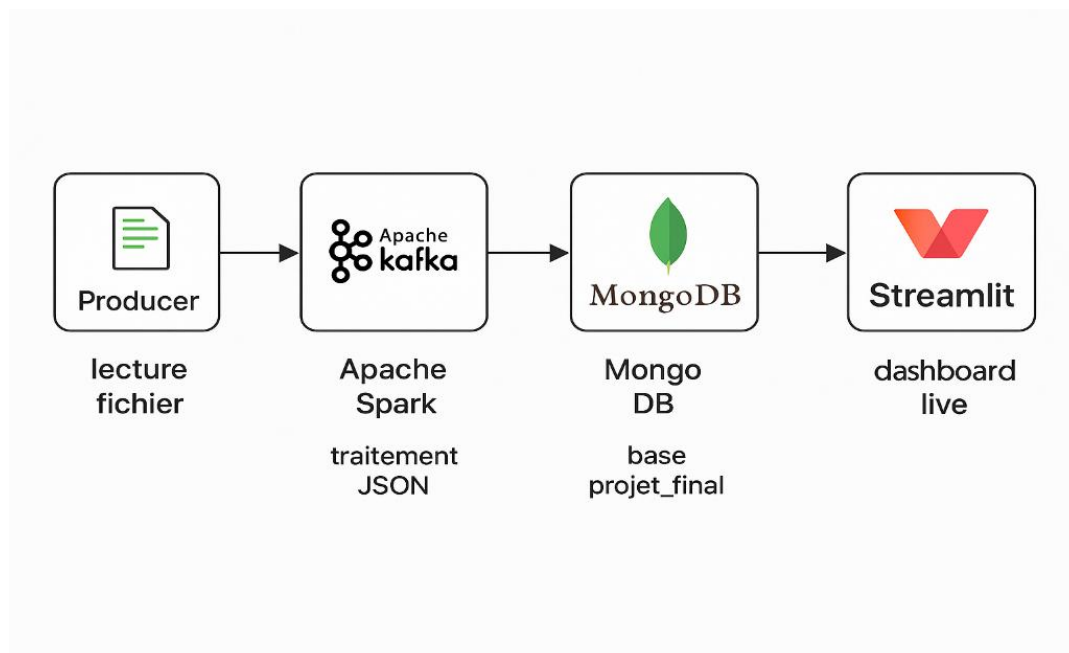
Afin de construire une architecture distribuée de traitement de données massives, nous avons retenu l'association Apache Kafka, Apache Spark Structured Streaming et MongoDB.

Kafka a été choisi pour sa capacité à gérer efficacement de larges flux de données en continu grâce à son architecture distribuée native, garantissant la résilience et la scalabilité horizontale des traitements.

Spark Structured Streaming a été sélectionné pour son aptitude à traiter en micro-batch ou en temps quasi réel de grands volumes de données en offrant une faible latence et une optimisation automatique des ressources, tout en garantissant la tolérance aux pannes par ses mécanismes de checkpoint et de reprise.

Enfin, MongoDB a été intégré comme solution de persistance grâce à son modèle NoSQL orienté documents, particulièrement adapté pour stocker des données semi-structurées provenant du streaming, et offrant une grande souplesse pour des opérations d'analytique rapide sans schéma rigide.

Cette combinaison de technologies permet ainsi d'assurer l'ingestion, le traitement en temps réel et le stockage scalable des données tout en préparant une future exploitation efficace et modulaire.



Nous allons maintenant voir plus en détail le fonctionnement de chaque composants de l'architecture.

1. Producer (Producteur de données)

Le producer lit les données brutes depuis les fichiers textes, les structures au format JSON, et les envoie dans le topic Kafka dédié.

C'est la porte d'entrée du système qui simule un flux de données continu en micro-batches.

2. Kafka (Système de messagerie distribuée)

Kafka joue le rôle d'intermédiaire de transport : il reçoit les messages du producteur et les stocke temporairement dans un topic (donnees_topic).

Cela permet de découpler l'ingestion de données de leur traitement, garantissant une haute disponibilité et une résilience à la panne.

Dans notre cas, nous avons mis tous nos fichiers bruts dans un dossier. Kafka va ainsi les envoyer vers le producer un par un, par batch de 20 000 lignes.

```
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 11911 messages restants envoyés.

✓ Fin de traitement du fichier : ValeursFoncières-2022.txt
📁 Total lignes envoyées : 4671911

=====
📁 DÉBUT DU TRAITEMENT DU FICHIER : ValeursFoncières-2023.txt
=====

✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
✓ 20000 messages envoyés.
```

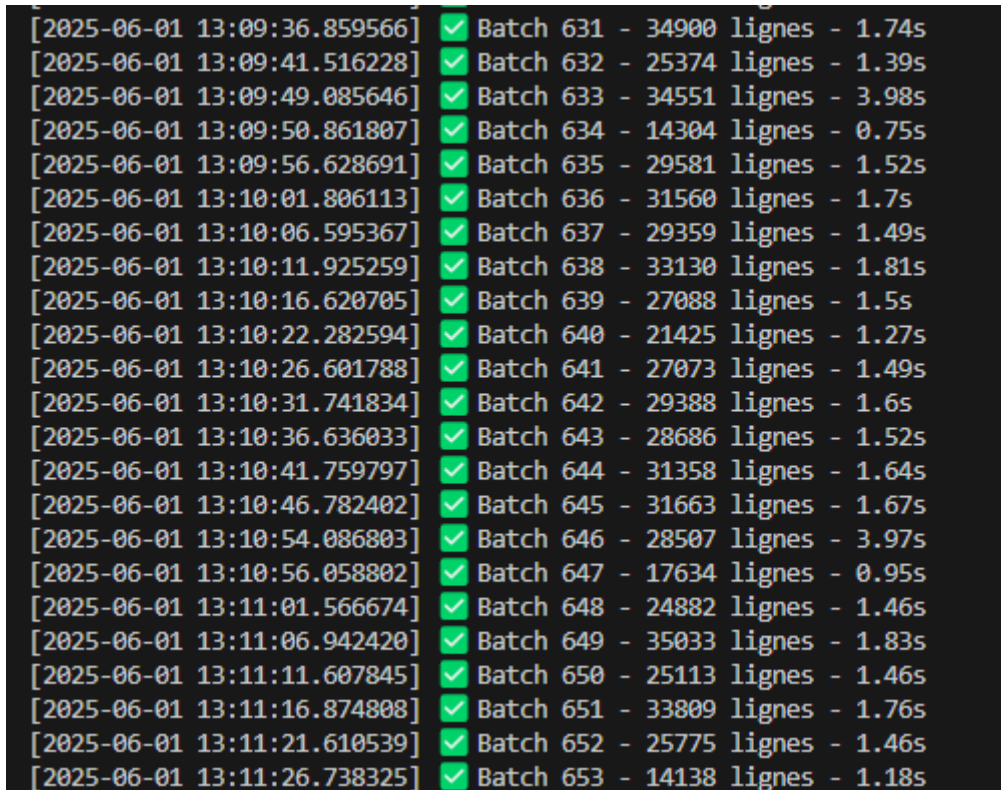
3. Spark Structured Streaming (Moteur de traitement temps réel)

Spark consomme les messages du topic Kafka en flux continu.

Il parse les données JSON, corrige dynamiquement les types de données (ex: valeur foncière convertie en float), remplace les valeurs nulles, et prépare les données pour un stockage

structuré. Il nous permet également de corriger les premières erreurs simples, comme par exemple remplacer les valeurs nulles par des 0, changer les formats de dates ou autre.

Grâce au micro-batching, Spark garantit un traitement rapide et scalable.



[2025-06-01 13:09:36.859566]	✓	Batch 631 - 34900 lignes - 1.74s
[2025-06-01 13:09:41.516228]	✓	Batch 632 - 25374 lignes - 1.39s
[2025-06-01 13:09:49.085646]	✓	Batch 633 - 34551 lignes - 3.98s
[2025-06-01 13:09:50.861807]	✓	Batch 634 - 14304 lignes - 0.75s
[2025-06-01 13:09:56.628691]	✓	Batch 635 - 29581 lignes - 1.52s
[2025-06-01 13:10:01.806113]	✓	Batch 636 - 31560 lignes - 1.7s
[2025-06-01 13:10:06.595367]	✓	Batch 637 - 29359 lignes - 1.49s
[2025-06-01 13:10:11.925259]	✓	Batch 638 - 33130 lignes - 1.81s
[2025-06-01 13:10:16.620705]	✓	Batch 639 - 27088 lignes - 1.5s
[2025-06-01 13:10:22.282594]	✓	Batch 640 - 21425 lignes - 1.27s
[2025-06-01 13:10:26.601788]	✓	Batch 641 - 27073 lignes - 1.49s
[2025-06-01 13:10:31.741834]	✓	Batch 642 - 29388 lignes - 1.6s
[2025-06-01 13:10:36.636033]	✓	Batch 643 - 28686 lignes - 1.52s
[2025-06-01 13:10:41.759797]	✓	Batch 644 - 31358 lignes - 1.64s
[2025-06-01 13:10:46.782402]	✓	Batch 645 - 31663 lignes - 1.67s
[2025-06-01 13:10:54.086803]	✓	Batch 646 - 28507 lignes - 3.97s
[2025-06-01 13:10:56.058802]	✓	Batch 647 - 17634 lignes - 0.95s
[2025-06-01 13:11:01.566674]	✓	Batch 648 - 24882 lignes - 1.46s
[2025-06-01 13:11:06.942420]	✓	Batch 649 - 35033 lignes - 1.83s
[2025-06-01 13:11:11.607845]	✓	Batch 650 - 25113 lignes - 1.46s
[2025-06-01 13:11:16.874808]	✓	Batch 651 - 33809 lignes - 1.76s
[2025-06-01 13:11:21.610539]	✓	Batch 652 - 25775 lignes - 1.46s
[2025-06-01 13:11:26.738325]	✓	Batch 653 - 14138 lignes - 1.18s

4. MongoDB (Base de données NoSQL)

Les données transformées sont stockées dans MongoDB, dans une base appelée projet_final et une collection streaming_resultats.

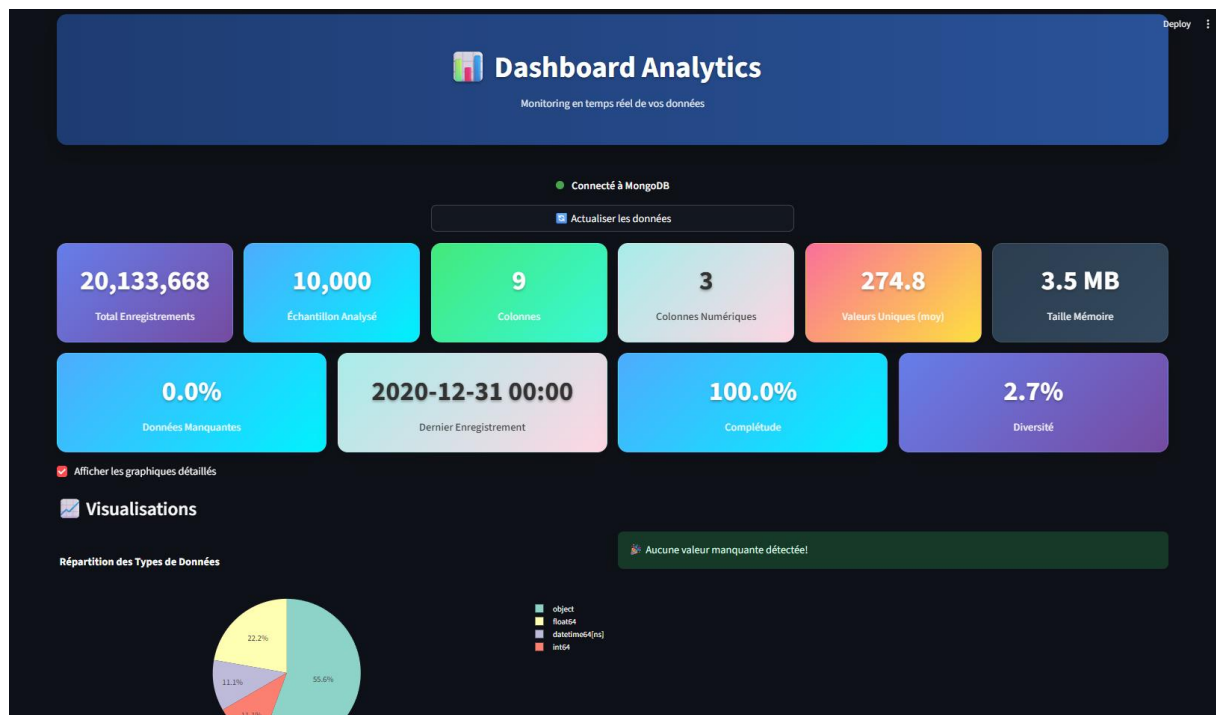
MongoDB est adapté aux flux semi-structurés car il permet d'enregistrer sans schéma rigide, tout en assurant une bonne rapidité d'accès aux données pour l'analyse. C'est la solution parfaite pour notre projet, car nous pouvons ainsi regrouper toutes nos données facilement.

```
test> use projet_final
switched to db projet_final
projet_final> db.streaming_resultats.find().pretty()
[
  {
    _id: ObjectId('683c44c488464d50c98de645'),
    fichier_source: 'ValeursFoncières-2020',
    date_mutation: ISODate('2020-01-07T00:00:00.000Z'),
    nature_mutation: 'Vente',
    valeur_fonciere: 8000,
    code_postal: '1250',
    commune: 'CEYZERAT',
    type_local: '',
    surface_reelle_bati: 0,
    nombre_pieces_principales: 0
  },
  {
    _id: ObjectId('683c44c488464d50c98de646'),
    fichier_source: 'ValeursFoncières-2020',
    date_mutation: ISODate('2020-01-02T00:00:00.000Z'),
    nature_mutation: 'Vente',
    valeur_fonciere: 2175,
    code_postal: '1290',
    commune: 'LAIZ',
    type_local: '',
    surface_reelle_bati: 0,
    nombre_pieces_principales: 0
  },
]
```

5. Streamlit (Interface utilisateur de visualisation)

Streamlit propose une interface graphique simple et interactive pour consulter en temps réel les données présentes dans MongoDB.

L'application permet d'afficher un tableau complet des données ingérées, de rafraîchir manuellement la vue, et de consulter des statistiques globales sur les volumes de données traitées.



Streamlit de notre base MongoDB

Pour conclure :

L'architecture mise en œuvre est nativement évolutive et scalable.

Chaque composant est indépendant et conçu pour fonctionner en environnement distribué :

- Kafka permet de gérer un nombre illimité de producteurs et de consommateurs en ajoutant simplement des partitions et des brokers supplémentaires.
- Spark Structured Streaming peut s'étendre horizontalement en ajoutant des workers pour paralléliser le traitement des données sans modifier la logique applicative.
- MongoDB, en mode cluster, permet d'absorber des volumes croissants de données tout en maintenant des temps d'accès rapides grâce à la réplication et au sharding.
- Streamlit peut être facilement mis à l'échelle via un déploiement en conteneurs pour supporter plus d'utilisateurs si nécessaire.

6. PostgreSQL (Stockage de données structuré)

Nous avons utilisé PostgreSQL comme base de données relationnelle pour notre projet. En effet, cette BDD nous permet de stocker des données transformées dans une structure de données optimisée pour l'analyse multidimensionnelle. Contrairement à MongoDB que nous avons utilisé précédemment pour le streaming, PostgreSQL est ici utilisé pour sa capacité à stocker les données de manière structurée. Cet environnement va nous permettre de pouvoir générer des scripts SQL pour, dans la finalité, utiliser des outils de visualisation ou ouvrir le projet vers d'autres analyses plus complexes (Machine Learning, modèles de prédictions, ...).

Ainsi, nos données issues du streaming de notre bronze_layer vont pouvoir directement arriver dans des tables structurées, desquelles nous allons ensuite pouvoir générer différents cubes.

L'utilisation de PostgreSQL s'explique aussi par sa facilité de mise en place, sa richesse fonctionnelle assurant la prise en charge de fonctions d'agrégations, création de vues, utilisation de requêtes complexes, modélisation multidimensionnelle. Aussi, PostgreSQL est très largement répandu dans le monde de la data, permettant ainsi la compatibilité avec de nombreux outils d'ETL et de BI, dont nous aurons très probablement besoin dans les futures parties du projet (Tableau, PowerBI, ...).

Pour finir, la richesse de nos datasets nous inquiétait beaucoup. PostgreSQL nous est alors parvenu comme une solution très fonctionnelle, car cet outil est capable de gérer les millions de lignes.

```

✓ Données 2023-02 insérées.
📦 Traitement 2023-03...
✓ Données 2023-03 insérées.
📦 Traitement 2023-02...
✓ Données 2023-02 insérées.
📦 Traitement 2023-03...
✓ Données 2023-03 insérées.
✓ Données 2023-02 insérées.
📦 Traitement 2023-03...
✓ Données 2023-03 insérées.
✓ Données 2023-03 insérées.
📦 Traitement 2023-04...
✓ Données 2023-04 insérées.
📦 Traitement 2023-05...
✓ Données 2023-05 insérées.
📦 Traitement 2023-06...
✓ Données 2023-06 insérées.
📦 Traitement 2023-07...
✓ Données 2023-07 insérées.
📦 Traitement 2023-08...
✓ Données 2023-08 insérées.
📦 Traitement 2023-09...
✓ Données 2023-09 insérées.
📦 Traitement 2023-10...
✓ Données 2023-10 insérées.
📦 Traitement 2023-11...
✓ Données 2023-11 insérées.
📦 Traitement 2023-12...
✓ Données 2023-12 insérées.
✓ Transfert complet terminé.
sparkuser@d92a3d130402:/app/Silver_layer$

```

Ingestion des données dans notre base PostgreSQL

The screenshot shows the DBeaver interface with a PostgreSQL database named 'projet_fin'. The schema 'silver_results' is expanded, showing a table 'silver_results'. The table contains data for various real estate transactions, including details like code_postal, commune, date_mutation, fichier_source, nature_mutation, nombre_pieces_principales, surface_mette_bati, type_loca, valeur_foncier, and part_date.

id	code_postal	commune	date_mutation	fichier_source	nature_mutation	nombre_pieces_principales	surface_mette_bati	type_loca	valeur_foncier	part_date
1	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	0	0	0	430000	2023-06-01
2	41100002004071000000	CHALLANS	2020-01-06	ValeurFoncieres-2020	Vente	0	0	0	46000	2023-06-01
3	7900000000000000000000	TALENCE	2020-01-21	ValeurFoncieres-2020	Vente	3	94	Maison	400000	2023-06-01
4	3420000000000000000000	PIERRE BENITE	2020-01-13	ValeurFoncieres-2020	Vente en l'état futur d'achèvement	0	0	Dependence	242156	2023-06-01
5	3340010000000000000000	DEUXVILLE	2020-01-28	ValeurFoncieres-2020	Vente	4	0	Maison	148000	2023-06-01
6	9440000000000000000000	YVONCHEN	2020-01-23	ValeurFoncieres-2020	Vente	0	0	0	220000	2023-06-01
7	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente en l'état futur d'achèvement	0	0	0	214400	2023-06-01
8	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	4	73	Maison	130000	2023-06-01
9	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente en l'état futur d'achèvement	0	0	Dependence	28000	2023-06-01
10	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	3	63	Appartement	79000	2023-06-01
11	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	4	63	Maison	107000	2023-06-01
12	41100002004071000000	CHALLANS	2020-01-06	ValeurFoncieres-2020	Vente	0	0	Maison	130000	2023-06-01
13	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	5	144	Maison	396000	2023-06-01
14	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	1	18	Appartement	60000	2023-06-01
15	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	0	0	0	10000	2023-06-01
16	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente en l'état futur d'achèvement	0	0	Appartement	214000	2023-06-01
17	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	0	0	0	1	2023-06-01
18	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	3	67	Appartement	135000	2023-06-01
19	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	0	0	0	150	2023-06-01
20	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	0	0	0	6440	2023-06-01
21	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	3	70	Appartement	272000	2023-06-01
22	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	0	0	0	60000	2023-06-01
23	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	0	0	0	7000	2023-06-01
24	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	1	23	Appartement	48000	2023-06-01
25	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	0	0	Dependence	18000	2023-06-01
26	30304200404070395003010	SAINT-ETIENNE	2020-01-09	ValeurFoncieres-2020	Vente	0	0	Dependence	120000	2023-06-01

Illustration des données dans notre base (DBeaver)

Environnement de test :

GitHub Actions (Intégration continue et automatisation des tests) :

Afin d'assurer la robustesse et la maintenabilité du pipeline de traitement, nous avons intégré un processus d'intégration continue (CI) avec GitHub Actions. Cette plateforme permet de déclencher automatiquement des tests à chaque mise à jour du code, garantissant ainsi la non-régression du projet.

Plutôt que d'exécuter tout l'environnement distribué (Kafka, Spark, Mongo), le choix a été fait de tester de manière isolée certaines briques fonctionnelles à travers des scripts de test unitaires. Ces tests, écrits en Python avec pytest, vérifient notamment l'importabilité des modules, la structure du schéma des données JSON, ou la transformation correcte de types. Cela permet une vérification rapide et fiable à chaque modification du code source, sans devoir reconstruire l'architecture complète.

Ce choix s'inscrit dans une bonne pratique d'ingénierie logicielle : tester les modules en amont, valider la cohérence du projet, et préparer une intégration continue plus large si nécessaire (tests d'intégration ou déploiement sur un environnement de staging).

 Modification de l'API, ajout d'un fichier de logs CI - Tests pipeline PySpark #15: Commit f8fef0f pushed by VictorBonnin	main	 last month  52s
 modifications dus rapport CI - Tests pipeline PySpark #14: Commit 4939853 pushed by VictorBonnin	main	 last month  55s
 modifications sur les données globales + resets + Parquet... CI - Tests pipeline PySpark #13: Commit 0cea4aa pushed by VictorBonnin	main	 last month  1m 8s
 Modification du stream pour les dates automatiques + ag... CI - Tests pipeline PySpark #12: Commit a656704 pushed by VictorBonnin	main	 last month  1m 5s
 Modification du stream pour les dates automatiques CI - Tests pipeline PySpark #11: Commit 237b5e8 pushed by VictorBonnin	main	 last month  57s
 Correction de plein de bugs, tout refonctionne ici. CI - Tests pipeline PySpark #10: Commit 96bf051 pushed by VictorBonnin	main	 last month  53s
 Modifications API + tests CI - Tests pipeline PySpark #9: Commit 3f29055 pushed by VictorBonnin	main	 last month  59s

Workflow de notre github actions

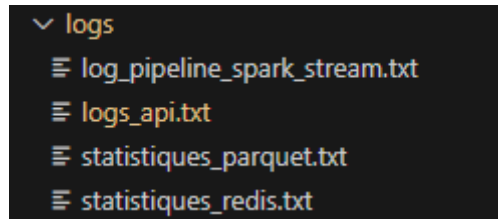
Fichiers txt souples et réduits :

Les fichiers .txt utilisés pour tester le pipeline intégral ont été réduits au maximum, sont légers et structurés comme les fichiers globaux. Ainsi, ils permettent de simuler un flux de donnée réaliste tout en facilitant le débogage et l'analyse des traitements.

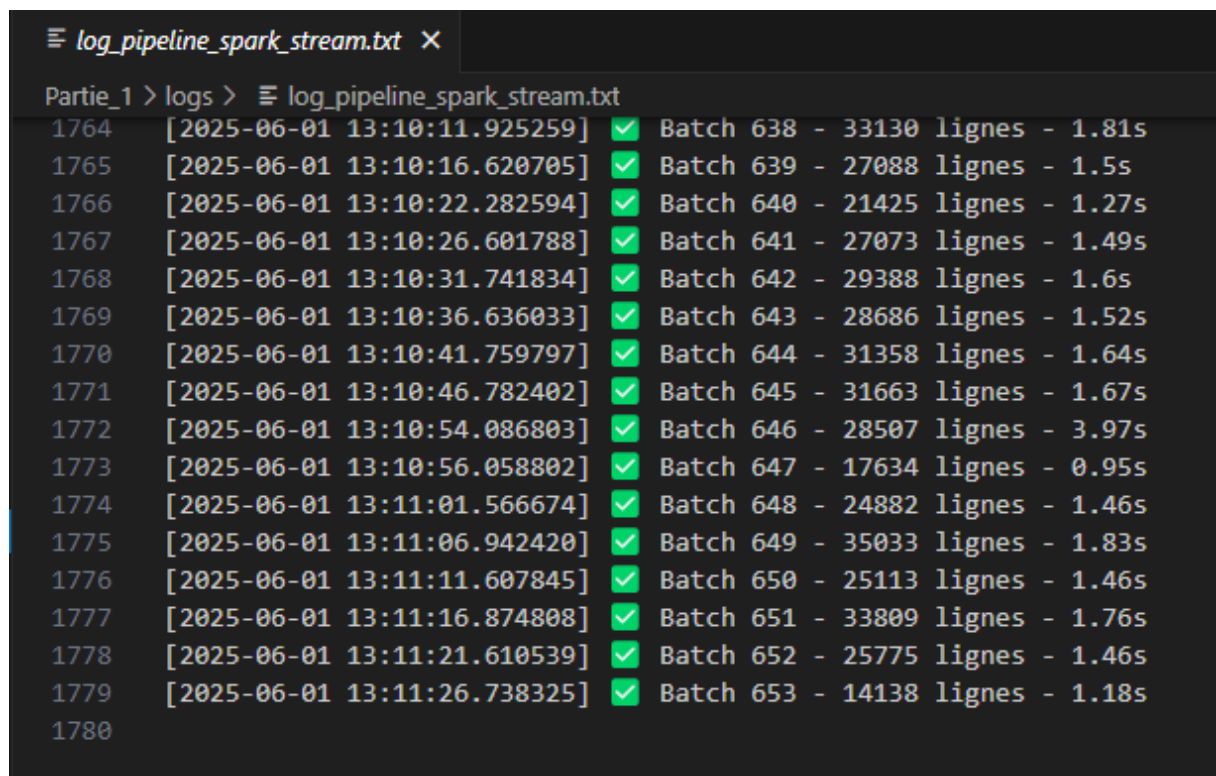
Nous pouvons grâce à cela faire des tests rapides et bien simuler le comportement de tout notre pipeline sans avoir à perdre du temps de génération des fichiers. Ce choix nous permet donc

d’être flexible dans notre manière de travailler, et de pouvoir jouer avec les différentes étapes du projet tout en gardant un œil sur des données bien précises.

Ainsi, nous pourrions également par la suite mettre en place des alertes sur nos rapports BI en lisant ces fichiers de logs.



Regroupement de nos fichiers de logs dans un dossier



Partie_1 > logs > log_pipeline_spark_stream.txt	
1764	[2025-06-01 13:10:11.925259] ✓ Batch 638 - 33130 lignes - 1.81s
1765	[2025-06-01 13:10:16.620705] ✓ Batch 639 - 27088 lignes - 1.5s
1766	[2025-06-01 13:10:22.282594] ✓ Batch 640 - 21425 lignes - 1.27s
1767	[2025-06-01 13:10:26.601788] ✓ Batch 641 - 27073 lignes - 1.49s
1768	[2025-06-01 13:10:31.741834] ✓ Batch 642 - 29388 lignes - 1.6s
1769	[2025-06-01 13:10:36.636033] ✓ Batch 643 - 28686 lignes - 1.52s
1770	[2025-06-01 13:10:41.759797] ✓ Batch 644 - 31358 lignes - 1.64s
1771	[2025-06-01 13:10:46.782402] ✓ Batch 645 - 31663 lignes - 1.67s
1772	[2025-06-01 13:10:54.086803] ✓ Batch 646 - 28507 lignes - 3.97s
1773	[2025-06-01 13:10:56.058802] ✓ Batch 647 - 17634 lignes - 0.95s
1774	[2025-06-01 13:11:01.566674] ✓ Batch 648 - 24882 lignes - 1.46s
1775	[2025-06-01 13:11:06.942420] ✓ Batch 649 - 35033 lignes - 1.83s
1776	[2025-06-01 13:11:11.607845] ✓ Batch 650 - 25113 lignes - 1.46s
1777	[2025-06-01 13:11:16.874808] ✓ Batch 651 - 33809 lignes - 1.76s
1778	[2025-06-01 13:11:21.610539] ✓ Batch 652 - 25775 lignes - 1.46s
1779	[2025-06-01 13:11:26.738325] ✓ Batch 653 - 14138 lignes - 1.18s
1780	

Logs de notre stream spark

Cette architecture respecte les principes modernes du Big Data : décentralisation, résilience, élasticité et facilité de maintenance.

Elle permet de s'adapter dynamiquement à l'augmentation des volumes, des vitesses d’ingestion et des besoins métiers, tout en minimisant les coûts d'évolution.

Résumé de validation de compétence du bloc 2 :

1. Une architecture de traitement distribué est proposée :

C1 – Veille technologique : Une étude comparative des frameworks big data a conduit au choix de Spark pour le traitement, Kafka pour l'ingestion, MongoDB pour le streaming et PostgreSQL pour le stockage.

C2 – Pertinence de l'architecture : L'architecture répond à un besoin hybride de traitement batch et temps réel, en respectant les contraintes du client.

C3 – Ressources suffisantes : La puissance de calcul et la mémoire sont assurées via une infrastructure Docker maîtrisée, et scalable dans le cadre d'une évolution du projet.

C4 – Environnements logiciels adaptés : Les environnements Spark, Kafka, MongoDB et PostgreSQL sont pleinement intégrés.

C5 – Traitement complet : Les données sont traitées à la fois en batch (silver) et en streaming (bronze).

C6 – Résilience : L'utilisation de Kafka et le découplage des composants assurent la tolérance aux pannes.

C7 – Scalabilité : L'ensemble du système est horizontalement scalable via Spark, Kafka et MongoDB.

2. Une solution de streaming distribué est mise en place :

C1 – Veille sur les outils de streaming : Kafka et Spark Structured Streaming ont été sélectionnés et justifiés.

C2 – Traitement en micro-batch : Spark Structured Streaming traite les données en micro-batches de 5 secondes.

C3 – Traitement en temps réel : L'architecture permet un traitement quasi temps réel avec monitoring intégré.

3. Les données sont extraites, transformées et mise à disposition :

C1 – Formatage des données : Nettoyage, typage, filtrage et enrichissement réalisés.

C2 – Intégration de données : Plusieurs fichiers injectés dans Kafka via des scripts adaptés.

C3 – Modélisation multidimensionnelle : CUBES analytiques créés dans PostgreSQL pour l'analyse (disponible dans la description du bloc 1).

4. Les pipelines sont développés :

C1 – Sources multiples : Automatisation de la lecture de plusieurs fichiers dans Kafka.

C2 – Accès rapide : Consultation directe des données dans Mongo et PostgreSQL.

C3 – Suivi des performances : Logs Spark disponibles pour visualiser les flux.

C4 – Ordonnancement : Orchestration et suivi réalisés via journaux et scripts Docker.

C5 – Monitoring : Les performances sont observables via les logs système et applicatifs.

C6 – Extensibilité : Le système gère dynamiquement plusieurs fichiers.

5. La création, les tests, l'intégration et le déploiement des pipelines de données sont automatisés

C1 – Veille technologique DevOps : Docker est utilisé efficacement. GitHub Actions est en cours d'intégration.

C2 – Containerisation : Chaque composant est encapsulé dans un conteneur dédié.

C3 – Orchestration continue : GitHub Actions permet un déploiement continu.

C4 – Tests automatisés : Tests réalisés avec des jeux de fichiers et pipelines supervisés.

BLOC 1 – Concevoir et développer une architecture de stockage de données

Objectifs du bloc :

B1	Concevoir et développer une base de données relationnelle en réponse aux besoins d'un client en vue de la mise à disposition de ses données structurées pour un tout en utilisant les technologies et les langages de requêtes adaptés aux développements envisagés
	Concevoir et développer une base de données non-relationnelle en vue de la mise à disposition des données semis-structurées et non- structurées pour un traitement analytique ou d'intelligence artificielle tout en utilisant les technologies et les langages de requêtes adaptés
	Concevoir et construire un lac de données « Datalake » en choisissant les architectures, les indicateurs de performance et les solutions de stockage appropriées afin d'intégrer les données provenant des systèmes d'information multivariés : Données non structurées, semi-structurées, structurées et des solutions de stockage de données massives ; redondantes.
	Créer une API en utilisant les technologies qui permettent de rendre accessible les données en augmentant l'efficacité et la praticité des applications et des services

Mettre en place une base de données relationnelle :

Maintenant que nous avons des données qui sont injectées dans une base relationnelle, il va falloir les ordonnancer. Pour se faire, nous allons faire tourner une succession de scripts SQL qui vont nous permettre de créer notre base de données finale, notre gold_layer.

1. Modélisation de la base gold_layer

Après avoir structuré nos données dans une table unique dans la *Silver Layer* (appelée *silver_resultats*), nous avons souhaité aller plus loin dans la modélisation pour répondre aux exigences d'un stockage relationnel propre et évolutif. Cela nous a permis de valider les compétences du bloc 1 en construisant un vrai modèle relationnel basé sur des tables normalisées et des vues métiers prêtes à l'analyse.

- Étape 1 – Analyse et identification des entités

Nous avons commencé par identifier les redondances dans notre table *silver_resultats*. Certaines colonnes comme *commune*, *type_local* ou *nature_mutation* contiennent des valeurs répétées qui auraient tout intérêt à être isolées dans des tables de référence. C'est ce que nous avons fait en créant trois nouvelles tables :

- communes (avec un identifiant unique pour chaque nom de commune),
- types_locaux (types de biens immobiliers),
- natures_mutation (nature de la transaction, comme vente ou échange).

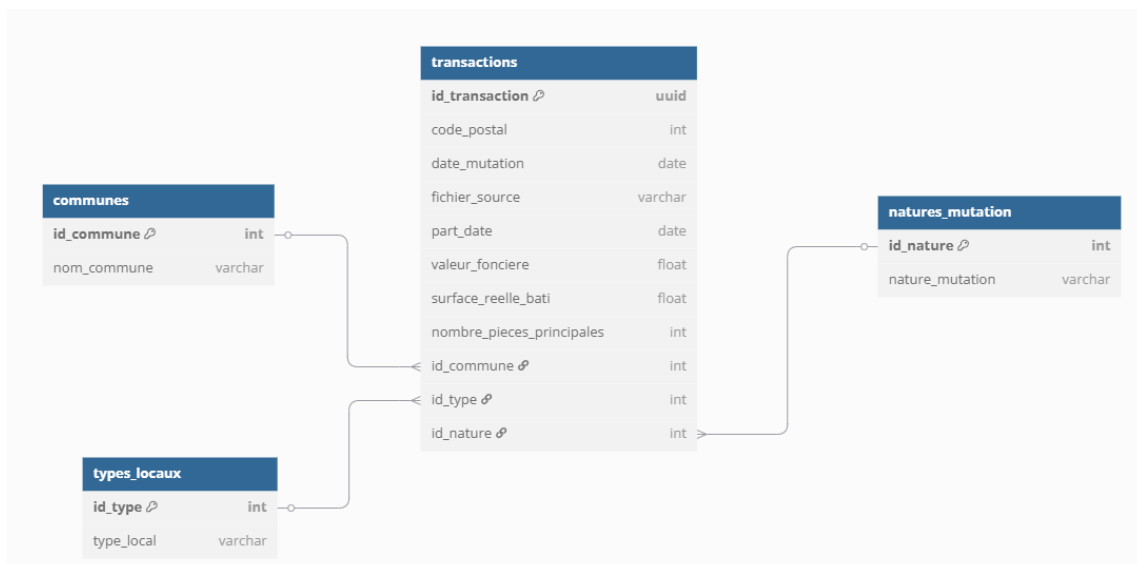


Diagramme effectué sur dbdiagram.io

- Étape 2 – Construction de la table transactions

Nous avons ensuite créé une table centrale transactions, qui contient les informations propres à chaque opération immobilière (valeur foncière, surface, date, etc.). Cette table référence les tables précédentes grâce à des clés étrangères (id_commune, id_type, id_nature), ce qui permet d'éviter la redondance et garantit l'intégrité des données.

Un point important ici a été la gestion de l'identifiant unique id_unique, généré en amont dans la Silver Layer. Comme nous voulions le réutiliser directement dans la table transactions (qui utilise le type UUID), nous avons dû adapter l'insertion en veillant à respecter les types de données attendus.

2. Création des tables dans Dbeaver

Voici les commandes à exécuter une par une :

Table des communes, avec un Id qui sera connecté à notre table principale (transactions)

-- Table des communes

```
CREATE TABLE IF NOT EXISTS communes (  
    id_commune SERIAL PRIMARY KEY,  
    nom_commune TEXT UNIQUE NOT NULL  
);
```

Table des locaux (infrastructures), avec un Id qui sera connecté à notre table principale (transactions)

-- Table des types de locaux

```
CREATE TABLE IF NOT EXISTS types_locaux (  
    id_type SERIAL PRIMARY KEY,  
    type_local TEXT UNIQUE NOT NULL  
);
```

Table des types de transactions, avec un Id qui sera connecté à notre table principale (transactions)

-- Table des natures de mutation

```
CREATE TABLE IF NOT EXISTS natures_mutation (  
    id_nature SERIAL PRIMARY KEY,  
    nature_mutation TEXT UNIQUE NOT NULL  
);
```

Table des transactions, qui sera notre table principale.

-- Table des transactions (Gold Layer principale)

```
CREATE TABLE IF NOT EXISTS transactions (  
    id_transaction UUID PRIMARY KEY,  
    id_unique VARCHAR(255),  
    code_postal VARCHAR(10),  
    date_mutation DATE,  
    fichier_source TEXT,  
    part_date DATE,  
    valeur_fonciere FLOAT,  
    surface_reelle_bati FLOAT,  
    nombre_pieces_principales INT,  
    id_commune INT REFERENCES communes(id_commune),  
    id_type INT REFERENCES types_locaux(id_type),  
    id_nature INT REFERENCES natures_mutation(id_nature)  
);
```

2. Injection des données

Maintenant que nos tables ont bien été créées, il faut y injecter les données. Nous pouvons, pour faire cela, utiliser les données de notre table 'silver_layer', car nous savons que les données ont déjà été nettoyées. Ainsi, grâce à ce travail, l'injection ne se résume plus qu'en quelques lignes.

-- Insérer les communes

```
INSERT INTO communes (nom_commune)  
  
SELECT DISTINCT commune  
  
FROM silver_resultats  
  
WHERE commune IS NOT NULL;
```

-- Insérer les types de locaux

```
INSERT INTO types_locaux (type_local)  
  
SELECT DISTINCT type_local  
  
FROM silver_resultats  
  
WHERE type_local IS NOT NULL;
```

-- Insérer les natures de mutation

INSERT INTO natures_mutation (nature_mutation)

SELECT DISTINCT nature_mutation

FROM silver_resultats

WHERE nature_mutation IS NOT NULL;

-- Insérer les données globales de la table principale

INSERT INTO transactions (

id_transaction,

id_unique,

code_postal,

date_mutation,

fichier_source,

part_date,

valeur_fonciere,

surface_reelle_bati,

nombre_pieces_principales,

id_commune,

id_type,

id_nature

)

SELECT

gen_random_uuid() **AS** id_transaction,

sr.id_unique,

sr.code_postal,

sr.date_mutation,

sr.fichier_source,

sr.part_date,

sr.valeur_fonciere,

sr.surface_reelle_bati,

sr.nombre_pieces_principales,

c.id_commune,

t.id_type,

```
n.id_nature  
  
FROM silver_resultats sr  
  
JOIN communes c ON sr.commune = c.nom_commune  
  
JOIN types_locaux t ON sr.type_local = t.type_local  
  
JOIN natures_mutation n ON sr.nature_mutation = n.nature_mutation;
```

Nous pouvons, maintenant que les données ont été injectées, vérifier qu'elles sont conformes dans nos nouvelles tables :

-- Aperçu des transactions les plus récentes

```
SELECT *  
  
FROM transactions  
  
ORDER BY date_mutation DESC  
  
LIMIT 10;
```

-- Vérifier les communes référencées dans les transactions

```
SELECT t.id_transaction, c.nom_commune, t.valeur_fonciere, t.surface_reelle_bati, nm.nature_mutation  
  
FROM transactions t  
  
JOIN communes c ON t.id_commune = c.id_commune  
  
join natures_mutation nm on t.id_nature = nm.id_nature  
  
order by t.valeur_fonciere DESC;
```

-- Vérifier la liste complète des types de locaux

```
SELECT * FROM types_locaux ORDER BY id_type;
```

Étape 3 – Création des vues métiers (Gold Layer)

Pour faciliter l'analyse, nous avons créé plusieurs vues SQL. Ces vues sont pensées comme des couches de lecture métier, prêtes à être utilisées dans un outil de BI ou par un utilisateur métier :

- Une vue enrichie `vue_gold_transactions_detaillees` qui rassemble toutes les informations d'une transaction en effectuant les jointures nécessaires.
- Une vue de statistiques par commune (`vue_gold_stats_par_commune`) pour suivre les volumes et la valeur moyenne des ventes.
- Une vue par type de bien immobilier (`vue_gold_stats_par_type_local`) pour observer les tendances de prix selon la nature du logement.

De cette manière, en cas de forte sollicitation de la base de données, celle-ci ne sera pas saturée, et les futurs rapports BI pourront s'exécuter sans monopoliser toutes les ressources du système. Cette architecture relationnelle est donc scalable et parfaitement adaptée à un modèle d'analyse comme le nôtre.

Voici maintenant les codes des vues SQL que nous avons créés pour renforcer notre gold_layer :

Vue principale :

```
CREATE OR REPLACE VIEW dl_gold_transactions_detaillees AS
```

```
SELECT
```

```
    t.id_transaction,  
    t.code_postal,  
    c.nom_commune,  
    t.date_mutation,  
    t.fichier_source,  
    t.part_date,  
    t.valeur_fonciere,  
    t.surface_reelle_bati,  
    t.nombre_pieces_principales,  
    tl.type_local,  
    nm.nature_mutation
```

```
FROM transactions t
```

```
JOIN communes c ON t.id_commune = c.id_commune
```

```
JOIN types_locaux tl ON t.id_type = tl.id_type
```

```
JOIN natures_mutation nm ON t.id_nature = nm.id_nature;
```

Vue des statistiques par commune :

```
CREATE OR REPLACE VIEW dl_gold_stats_par_commune AS
```

```
SELECT
```

```
    c.nom_commune,  
    COUNT(*) AS nb_transactions,  
    ROUND(AVG(t.valeur_fonciere)) AS valeur_moyenne,  
    ROUND(SUM(t.surface_reelle_bati)) AS surface_totale
```

```
FROM transactions t
```

```
JOIN communes c ON t.id_commune = c.id_commune
```



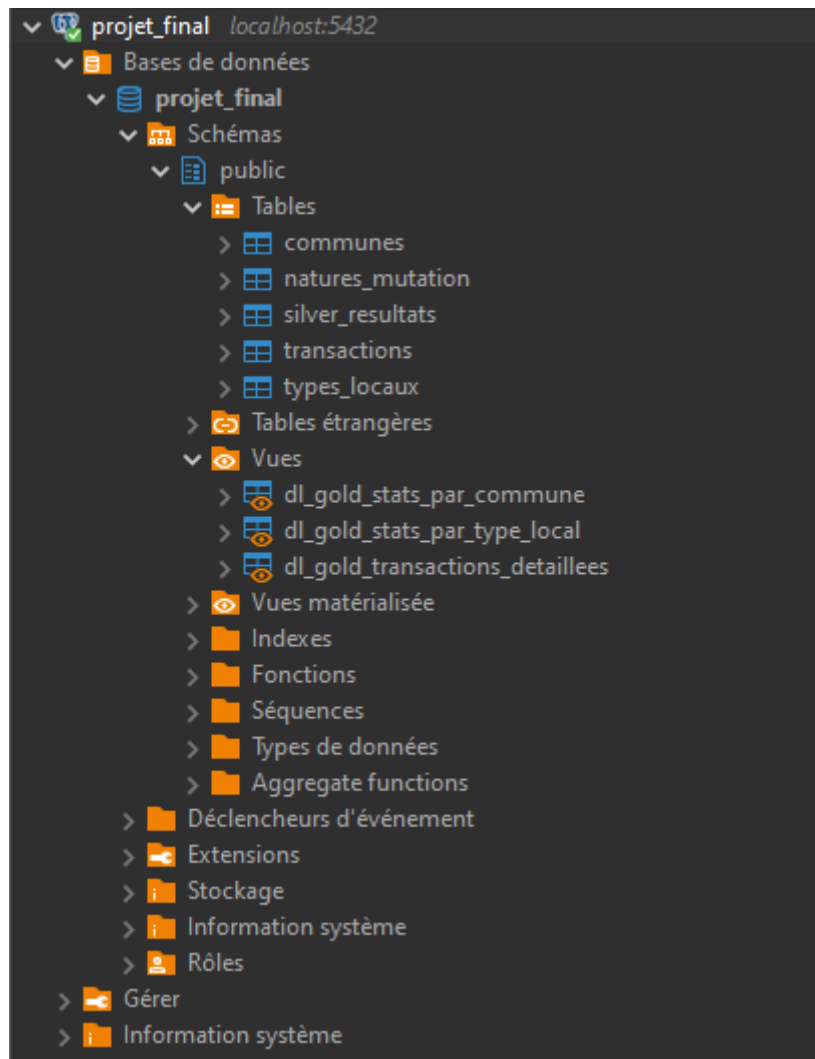
```
GROUP BY c.nom_commune  
ORDER BY nb_transactions DESC;
```

Vue des tendances par type de bien :

```
CREATE OR REPLACE VIEW vue_gold_stats_par_type_local AS  
SELECT  
    tl.type_local,  
    COUNT(*) AS nb_ventes,  
    ROUND(AVG(t.valeur_fonciere)) AS prix_moyen  
FROM transactions t  
JOIN types_locaux tl ON t.id_type = tl.id_type  
GROUP BY tl.type_local  
ORDER BY prix_moyen DESC;
```

Étape 4 – Vérification et tests

Chaque insertion et création de vue a été testée dans DBeaver, avec des scripts SQL simples de lecture (SELECT * FROM ...). Cela nous a permis de vérifier la cohérence des jointures, l'absence de doublons, et la validité des clés étrangères. À terme, ces vues permettront de construire des tableaux de bord plus efficaces et d'alimenter notre API.



Voici un exemple de données que nous pouvons retrouver dans notre dernier niveau de DBB :

id_transaction	code_postal	nom_commune	date_mutation	fichier_source	part_date	valeur_fonciere	surface_reelle_bati	nombre_pieces_principales	
27cee528-985a-410a-8191-4dd95019178e	1500	AMBRONAY	2022-01-10	sample_data_2022	2025-05-03	580 000	0	0	
f64ff80a-f719-45c7-bd38-02daa9630d2e	1140	SAINT-DIDIER-SUR-CHALA	2023-01-03	sample_data_2023	2025-05-03	325 000	0	0	
d05860cc-3288-4c0a-9c5a-aadf777fa697	1140	SAINT-DIDIER-SUR-CHALA	2023-01-03	sample_data_2023	2025-05-03	325 000	70	4	Mais
3e130759-7510-447f-a362-2e55886af034	1210	ORNEIX	2023-01-03	sample_data_2023	2025-05-03	770 000	136	7	Mais
6008b3ca-3b92-4e12-a6ee-f9b5fedae6ea	1450	SERRIERES-SUR-AIN	2023-01-03	sample_data_2023	2025-05-03	152 200	64	3	Mais
d6ad5ac3-6f3a-4b78-934d-dfab9d84ea7a	1160	PRIAY	2023-01-04	sample_data_2023	2025-05-03	430 000	0	0	
81bf4e3e-0183-471b-b229-512a224f3878	1250	SIMANDRE-SUR-SURAN	2023-01-04	sample_data_2023	2025-05-03	8 500	0	0	
88126d86-c5ae-4645-a4a6-6df79b225d55	1000	BOURG-EN-BRESSE	2023-01-04	sample_data_2023	2025-05-03	209 000	68	3	Appa
f05418a5-aa02-4910-a31c-3e6437e1944e	1000	BOURG-EN-BRESSE	2023-01-04	sample_data_2023	2025-05-03	209 000	0	0	Dépe
43fbfa4a-6be3-43fc-8358-ea195c4b1b8f	1130	ECHALLON	2023-01-04	sample_data_2023	2025-05-03	335 000	106	6	Mais
a1479748-4683-49ad-b616-f2cd545e8642	1630	ST-GENIS-POUILLY	2023-01-05	sample_data_2023	2025-05-03	1 070 000	233	8	Appa
b01d4c32-ca51-4d3c-bf1a-6b1e637b5309	1630	ST-GENIS-POUILLY	2023-01-05	sample_data_2023	2025-05-03	1 070 000	0	0	Dépe
c2e8a305-ca21-4cf6-a5e1-f77f6119eef4	1800	SAINT-JEAN-DE-NIOST	2023-01-05	sample_data_2023	2025-05-03	269 000	0	0	Dépe
634602e2-9593-4b59-93f6-0be49e5c6929	1800	SAINT-JEAN-DE-NIOST	2023-01-05	sample_data_2023	2025-05-03	269 000	73	3	Mais
513b9e43-1800-431e-b0bd-708469a75f6b	1340	ATTIGNAT	2023-01-06	sample_data_2023	2025-05-03	85 520	0	0	
86fbee6f-1466-4e9d-bb91-cdaf65ba600b	1480	VILLENEUVE	2023-01-06	sample_data_2023	2025-05-03	6 820	0	0	
4c7be8ac-36fc-48c7-8577-181597e7ced8	1500	AMBERIEU-EN-BUGEY	2023-01-06	sample_data_2023	2025-05-03	10 040	0	0	
21ae1568-d2d6-46e9-82ab-aac9e504587a	1500	AMBERIEU-EN-BUGEY	2023-01-06	sample_data_2023	2025-05-03	12 642	0	0	
d5c7940f-a343-47b5-9402-6880396e0357	1340	ATTIGNAT	2023-01-06	sample_data_2023	2025-05-03	85 520	0	0	Dépe
10a14c07-0d91-4032-a2c1-1d83a7cb7dc6	1340	ATTIGNAT	2023-01-06	sample_data_2023	2025-05-03	85 520	160	5	Mais
85beca2a-3071-40fb-9579-892caf9d38cf	1230	CHALEY	2024-01-02	sample_data_2024	2025-05-03	346,5	0	0	
f968c5c6-9803-4012-a1d0-78e813a722b4	1170	CHEVRY	2024-01-03	sample_data_2024	2025-05-03	10 000	0	0	
f417141e-2b87-453a-a12b-659630b4ac4c	1170	GEX	2024-01-03	sample_data_2024	2025-05-03	329 500	89	4	Appa
3a639f6f-343c-40c0-923c-eb0701a3bf09	1170	GEX	2024-01-03	sample_data_2024	2025-05-03	329 500	0	0	Dépe
1b92d083-4437-414b-9c27-74943e967dd9	1290	LAIZ	2024-01-08	sample_data_2024	2025-05-03	249 000	0	0	
8b137022-a79b-4015-9731-c04e8a0f9671	1110	PLATEAU D'HAUTEVILLE	2024-01-09	sample_data_2024	2025-05-03	20 000	0	0	

Nous pouvons bien voir ici que les données ont bien été préparées et normalisées. Chaque colonne respecte un format adapté à son usage :

- Les UUIDs assurent l'unicité de chaque transaction sans ambiguïté

- Les dates sont bien typées au format date
- Les champs numériques (valeur foncière, surface bâtie, nombre de pièces) sont bien convertis en types numériques exploitables

Ce niveau de préparation permet d'offrir un socle de données fiable, propre et directement exploitable pour la création de rapports ou l'entraînement de modèles prédictifs.

Mettre en place une base de données non-relationnelle :

C1 & 2

Pour la partie de la base de données non relationnelle, nous avons comme décrit précédemment utilisé une base MongoDB. Nous avons choisi cette dernière pour sa souplesse face aux données semi-structurées issue des traitements en streaming.

L'objectif de cette dernière était de pouvoir stocker rapidement et efficacement les données JSON envoyées par Kafka, et traitées en continu par Spark. MongoDB c'est ainsi montré comme une solution parfaitement adaptée à ce besoin. Etant une base orientée documents, elle nous permettait ainsi de stocker au format JSON les enregistrements envoyés sans la contrainte d'un schéma rigide, nous permettant ainsi d'avoir un flux efficace.

Cette approche nous a ainsi apporté plusieurs avantages :

- Ingestion facile et rapide des données
- Analyse des données possible et visualisation en temps direct
- Intégration avec des outils de visualisation tels que Steamlit

Ce choix d'utiliser MongoDB nous a ainsi permis de répondre aux exigences de traitements des données en quasi temps réel tout en assurant la scalabilité horizontale du projet.

Pour vérifier que nos données existent bien dans MongoDB, nous pouvons vérifier en allant dans son container.

Ainsi, une fois à l'intérieur, nous pouvons coder en NoSQL pour naviguer dans notre db, et y chercher les données :

Connexion à notre base mongo : `'docker exec -it partie_1-mongodb mongosh'`

```
PS D:\Projet\projet_fin_etudes\projet_final> docker exec -it partie_1-mongodb mongosh
Current Mongosh Log ID: 68178bdb54488a6c21d861df
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.0
Using MongoDB: 8.0.9
Using Mongosh: 2.5.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2025-05-04T09:42:13.084+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2025-05-04T09:42:13.455+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2025-05-04T09:42:13.455+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2025-05-04T09:42:13.455+00:00: We suggest setting the contents of sysfsFile to 0.
2025-05-04T09:42:13.456+00:00: vm.max_map_count is too low
2025-05-04T09:42:13.456+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----

test> |
```

Ensuite, il faut nous connecter à notre db : `'use projet_final'`

```
test> show dbs
admin            40.00 KiB
config          108.00 KiB
local           40.00 KiB
projet_final     80.00 KiB
test> use projet_final
switched to db projet_final
projet_final>
```

Et pour finir, maintenant que nous sommes connectés au bon environnement, il nous suffit d'afficher les données présentes dans la base : `'db.streaming_resultats.find().pretty()'`

```
projet_final> db.streaming_resultats.find().pretty()
[
  {
    _id: ObjectId('6817369d2f36f05efcbe1723'),
    fichier_source: 'sample_data_2020',
    date_mutation: '07/01/2020',
    nature_mutation: 'Vente',
    valeur_fonciere: 8000,
    code_postal: '1250',
    commune: 'CEYZERIAT',
    type_local: '',
    surface_reelle_bati: 0,
    nombre_pieces_principales: 0
  },
  {
    _id: ObjectId('6817369d2f36f05efcbe1724'),
    fichier_source: 'sample_data_2020',
    date_mutation: '02/01/2020',
    nature_mutation: 'Vente',
    valeur_fonciere: 2175,
    code_postal: '1290',
    commune: 'LAIZ',
    type_local: '',
    surface_reelle_bati: 0,
    nombre_pieces_principales: 0
  },
  {
    _id: ObjectId('6817369d2f36f05efcbe1725'),
    fichier_source: 'sample_data_2020',
    date_mutation: '02/01/2020',
    nature_mutation: 'Vente',
    valeur_fonciere: 2175,
    code_postal: '1290',
    commune: 'LAIZ',
    type_local: '',
    surface_reelle_bati: 0,
    nombre_pieces_principales: 0
  },
  {
    _id: ObjectId('6817369d2f36f05efcbe1726'),
    fichier_source: 'sample_data_2020',
    date_mutation: '02/01/2020',
    nature_mutation: 'Vente',
    valeur_fonciere: 2175,
    code_postal: '1290',
    commune: 'LAIZ',
    type_local: '',
    surface_reelle_bati: 0,
    nombre_pieces_principales: 0
  }
]
```

Nous voyons bien avec cette capture d'écran que les données sont parfaitement disponibles dans notre base MongoDB.

C3

L'utilisation de Redis dans notre projet nous permet d'illustrer le fonctionnement d'une base de données clé-valeurs. En effet, Redis est un moteur NoSQL en mémoire, qui stocke les données sous forme de paires : une clé unique associée à une valeur.

Nous avons donc souhaité utiliser Redis pour suivre l'état de la base MongoDB, en enregistrant les valeurs suivantes :

- Nombre total de documents (lignes) -> *stats_total_documents*
- La moyenne des valeurs foncières -> *stat_moyenne_valeur_fonciere*
- La date de dernière mise à jour -> *last_update_stats*
- Les 5 communes les plus fréquentes -> *top_communes*

Ces données sont donc, comme Redis le permet, stockées à chaque exécution du script python et consultables n'importe quand. Par exemple, nous pouvons exécuter sur un moteur Redis la commande '*r.get("last_update_stats")*' qui va nous permettre de récupérer la dernière date de mise à jour.

Nous avons aussi décidé d'un point de vue pratique, de garder une trace de toutes les exécutions du fichier, nous permettant ainsi d'avoir les données de manière persistante dans un fichier .txt, pour assurer un suivi manuel et durable.

Dans notre container principal, nous avons donc ainsi à exécuter le script redis pour que ce dernier nous retourne les informations que nous avons décrit ci-dessus :

```
I have no name!@76c0c1d0cf47:/app$ python redis_example.py

Statistiques enregistrées dans Redis :
- Total documents : 246
- Moyenne valeur foncière : 176653.75
- Top 5 communes :
  · PLATEAU D'HAUTEVILLE (32)
  · BOURG-EN-BRESSE (32)
  · FOISSIAT (22)
  · SAINT-DENIS-LES-BOURG (16)
  · LAIZ (12)
- Dernière mise à jour : 2025-05-04 17:41:46
```

Pour finir, au besoin, nous pourrions aussi consulter les statistiques sauvegardées en dur dans un fichier txt :

```

statistiques_redis.txt U X
Partie_1 > logs > statistiques_redis.txt
1  Statistiques enregistrées dans Redis :
2  - Total documents : 246
3  - Moyenne valeur foncière : 176653.75
4  - Top 5 communes :
5      - PLATEAU D'HAUTEVILLE (32)
6      - BOURG-EN-BRESSE (32)
7      - FOISSIAT (22)
8      - SAINT-DENIS-LES-BOURG (16)
9      - LAIZ (12)
10 - Dernière mise à jour : 2025-05-04 17:41:46
11 -----

```

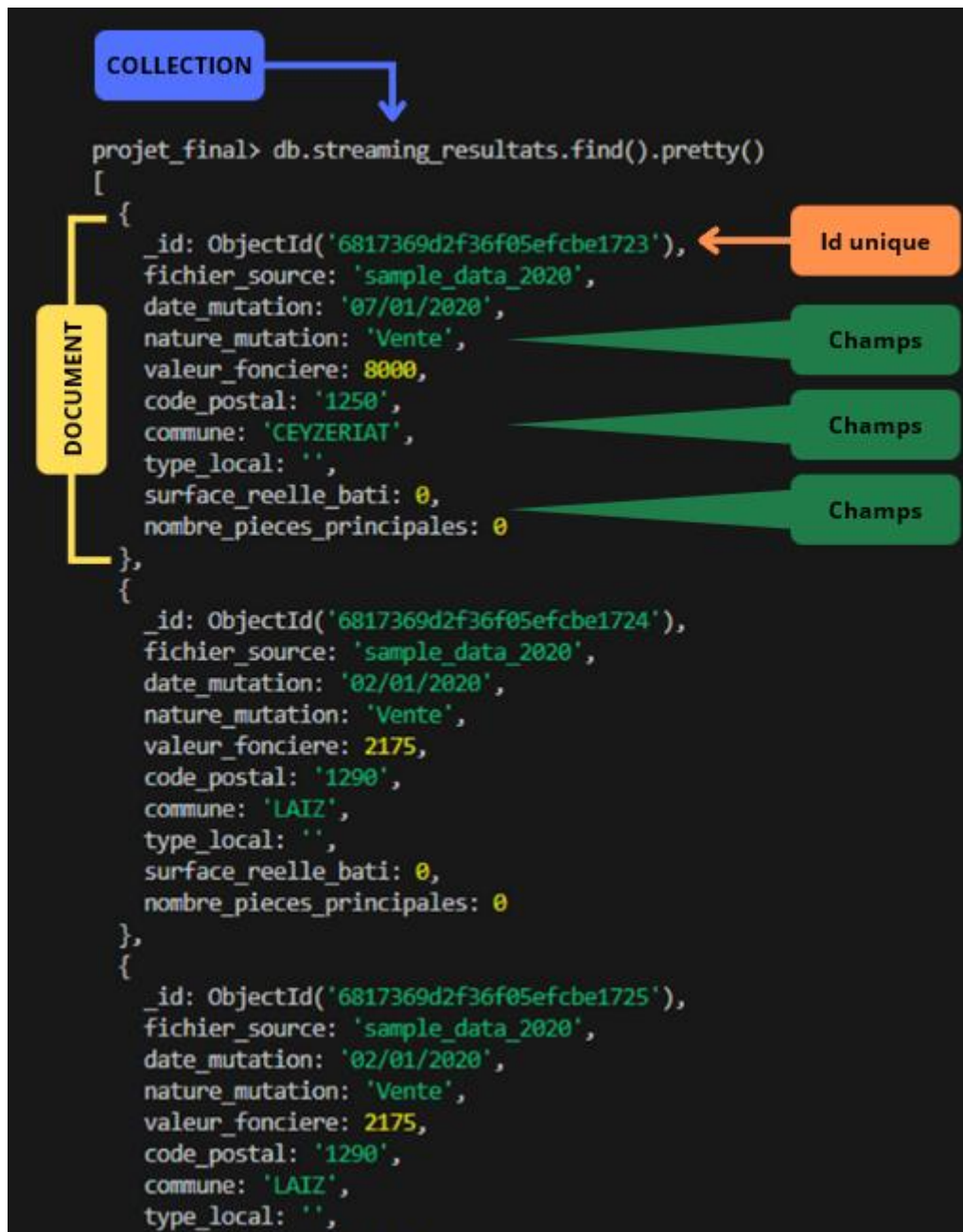
C4

Nous avons utilisé dans ce projet une base MongoDB orientée document. Elle stocke les données au format JSON, une structuration souple, sans schéma strict. Ce format s'adapte parfaitement avec le traitement des données issues de Spark.

Nous avons donc stocké nos données dans une base que nous avons appelée '**projet_final**', structurée selon le modèle natif de Mongo :

- Une base de données qui va regrouper l'ensemble des collections.
- Une ou plusieurs collections. Ici, nous avons créé notre collection '**streaming_resultats**', dans laquelle sera inscrit chaque flux au format JSON.
- Des documents : chaque transaction immobilière, donc chaque ligne issue des fichiers sources est un document indépendant.
- Des champs : Ces derniers sont des clés à l'intérieur des documents dans lesquels les données sont ensuite insérées.
- Un Id unique : Chaque document possède son id unique généré automatiquement.

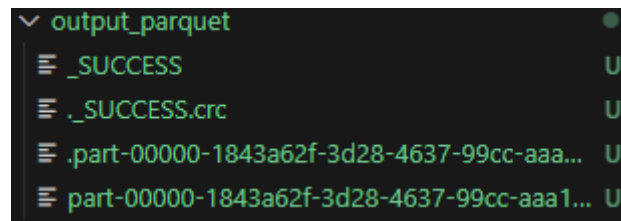
Ainsi, notre structuration respecte parfaitement un modèle orienté documents. Elle s'appuie exclusivement sur des objets JSON. Notre bronze_layer est ainsi parfaitement scalable, souple et adapté à l'ingestion de données en temps réel. Nous pouvons aussi, grâce à l'utilisation de cette structure, intégrer de nouveaux pipelines prenant leurs sources dans cette base.



C5

L'objectif de la compétence C5 est de démontrer notre capacité à utiliser une base de données orientée colonne. Ces dernières sont des bases conçues pour le traitement analytique à grande échelle. Dans notre projet, nous utilisons déjà un environnement Spark. La solution la plus adaptée à notre structure serait alors d'utiliser Parquet, qui est nativement supporté par Spark.

De plus, Parquet permet de stocker de grands volumes de données, et est hautement compressé, ce qui garantit un faible besoin de stockage. Enfin, parquet se base sur des fichiers colonnaires, c'est-à-dire des fichiers qui se lisent en colonne. Grâce à cela, lors de la lecture des données via une requête, Parquet ne va lire que les colonnes dont il a besoin, réduisant considérablement le temps de lecture et de chargement.



Dossier/fichiers générés automatiquement par Parquet pour enregistrer les données

Dans notre projet, nous avons décidé d'utiliser Parquet pour lire nos données depuis MongoDB, pour ensuite y faire un premier travail d'analyse :

- Affichage de lignes,
- Calculs de statistiques (moyennes, nombre total de documents, ...)
- Ecriture des résultats dans un nouveau fichier de logs, pour pouvoir faire une analyse comparative plus tard.

```
sparkuser@c13884bc539:/app/Parquet$ python read_parquet.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/05/05 21:43:02 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
25/05/05 21:43:02 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.

Exemple de données chargées depuis les fichiers Parquet :
```

_id	code_postal	commune	date_mutation	fichier_source	nature_mutation	nombre_pieces_principales	surface_reelle_bati	type_local	valeur_fonciere
{6819105b940aa312e99fbbf5}	1250	CEYZERIAT	07/01/2020	sample_data_2020	Vente	0	0.0		8000.0
{6819105b940aa312e99fbbf6}	1290	LAIZ	02/01/2020	sample_data_2020	Vente	0	0.0		2175.0
{6819105b940aa312e99fbbf7}	1290	LAIZ	02/01/2020	sample_data_2020	Vente	0	0.0		2175.0
{6819105b940aa312e99fbbf8}	1290	LAIZ	02/01/2020	sample_data_2020	Vente	0	0.0		2175.0
{6819105b940aa312e99fbbf9}	1290	LAIZ	02/01/2020	sample_data_2020	Vente	0	0.0		2175.0
{6819105b940aa312e99fbbfa}	1290	LAIZ	02/01/2020	sample_data_2020	Vente	0	0.0		2175.0
{6819105b940aa312e99fbbfb}	1960	PERONNAS	07/01/2020	sample_data_2020	Vente	0	0.0		75000.0
{6819105b940aa312e99fbbfc}	1370	VAL-REVERMONT	07/01/2020	sample_data_2020	Vente	0	0.0		123.0
{6819105b940aa312e99fbbfd}	1370	VAL-REVERMONT	07/01/2020	sample_data_2020	Vente	0	0.0		123.0
{6819105b940aa312e99fbbfe}	1270	COLIGNY	09/01/2020	sample_data_2020	Vente	0	0.0		72000.0

```
only showing top 10 rows

Statistiques sur les données :
- Nombre total de documents : 123
- Nombre de communes différentes : 38
- Moyenne de la valeur foncière : 176653.75 €

Top 5 des communes avec le plus de ventes :
```

commune	count
PLATEAU D'HAUTEVILLE	16
BOURG-EN-BRESSE	16
FOISSIAT	11
SAINT-DENIS-LES-BOURG	8
LAIZ	6

Nous avons pu ainsi découvrir l'intérêt d'une base orientée colonne, qui permet une analyse instantanée des données, tout en gardant un maximum d'efficacité grâce aux fichiers colonnaires.

Nous avons pu aussi montrer grâce à cette courte analyse l'importance de nettoyer les données que nous allons traiter ensuite.

C6

La partie C6 du bloc 1 concerne l'utilisation d'une base orientée graphe. Cette dernière a pour objectif de modéliser et stocker les données sous forme de nœuds et de relations. Contrairement à une base de données classique utilisant des jointures et tables, les bases de données graphes reposent sur les connexions entre les éléments. Cette approche est intéressante dans le cadre de structures complexes où tous les éléments s'entremêlent.

Pour l'utilisation d'une base orientée graphe dans notre projet, nous avons décidé d'utiliser Neo4j, une base relationnelle dynamique.

Nous avons donc développé un script qui nous permet de se connecter directement à notre base MongoDB pour y récupérer les données. Ainsi, nous allons pouvoir extraire les informations clés de chaque transactions immobilières (commune, valeur foncière, date de mutation, nature de la mutation, type de bien, code postal, etc.).

Voici donc le schéma auquel nous avons réfléchi pour pouvoir intégrer notre structure de données à une base en graphes :

(:NatureMutation)

└[:A_POUR_TYPE]→ (:TypeLocal)

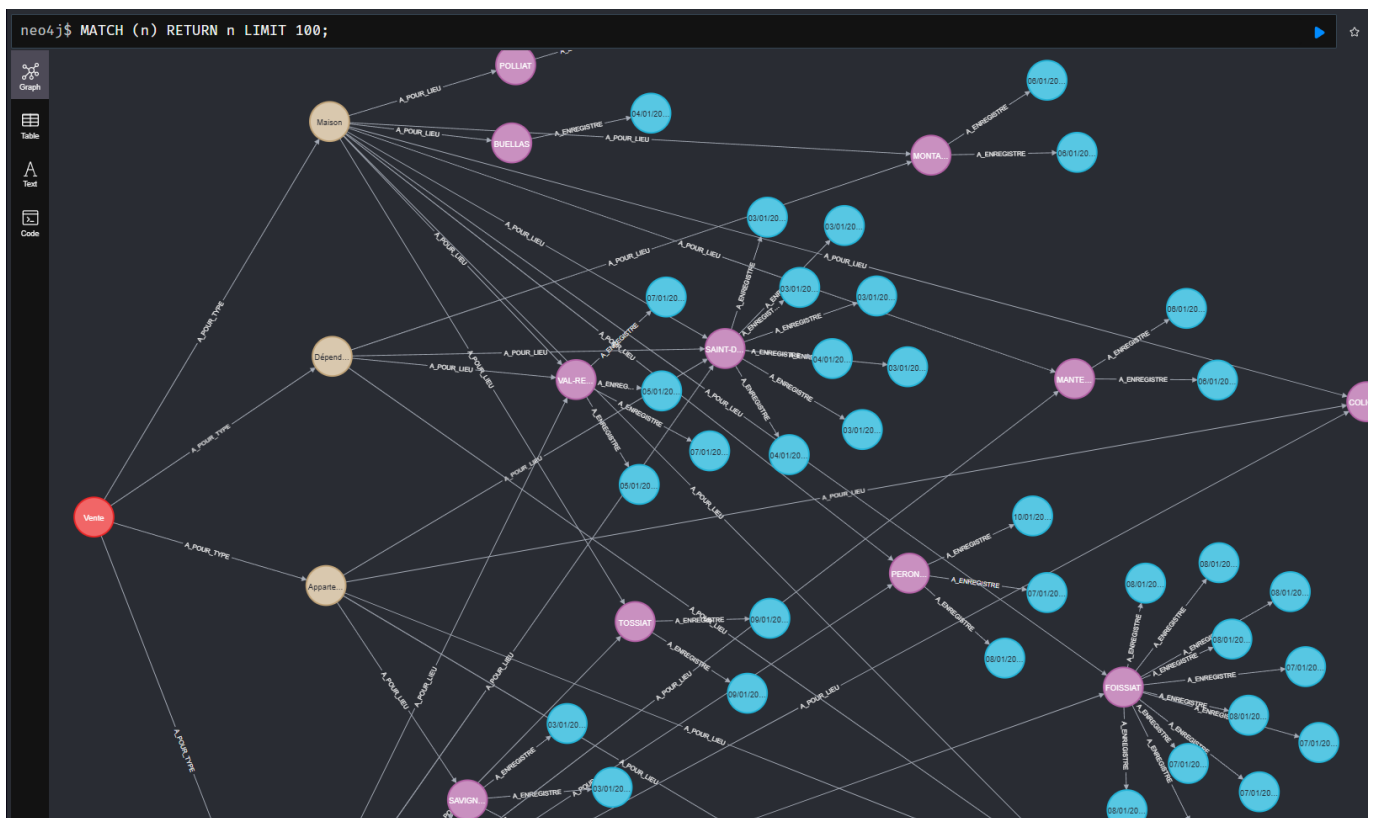
└[:A_POUR_LIEU]→ (:Commune)

└[:A_ENREGISTRE]→ (:Mutation {date, valeur, id})

Nous allons donc maintenant afficher les données dans Neo4j, pour voir si notre script correspond bien avec l'architecture qu'on voulait avoir affiché. Pour se faire, il nous suffit :

- d'exécuter le fichier *neo4j_transactions.py*
- aller sur le localhost://7474
- rentrer les identifiants de connexion :
 - o user : neo4j
 - o mot de passe : spark123

Nous arrivons sur l'interface, et nous pouvons alors exécuter un code basique pour afficher les données présentes : **MATCH (n) RETURN n LIMIT 100;**



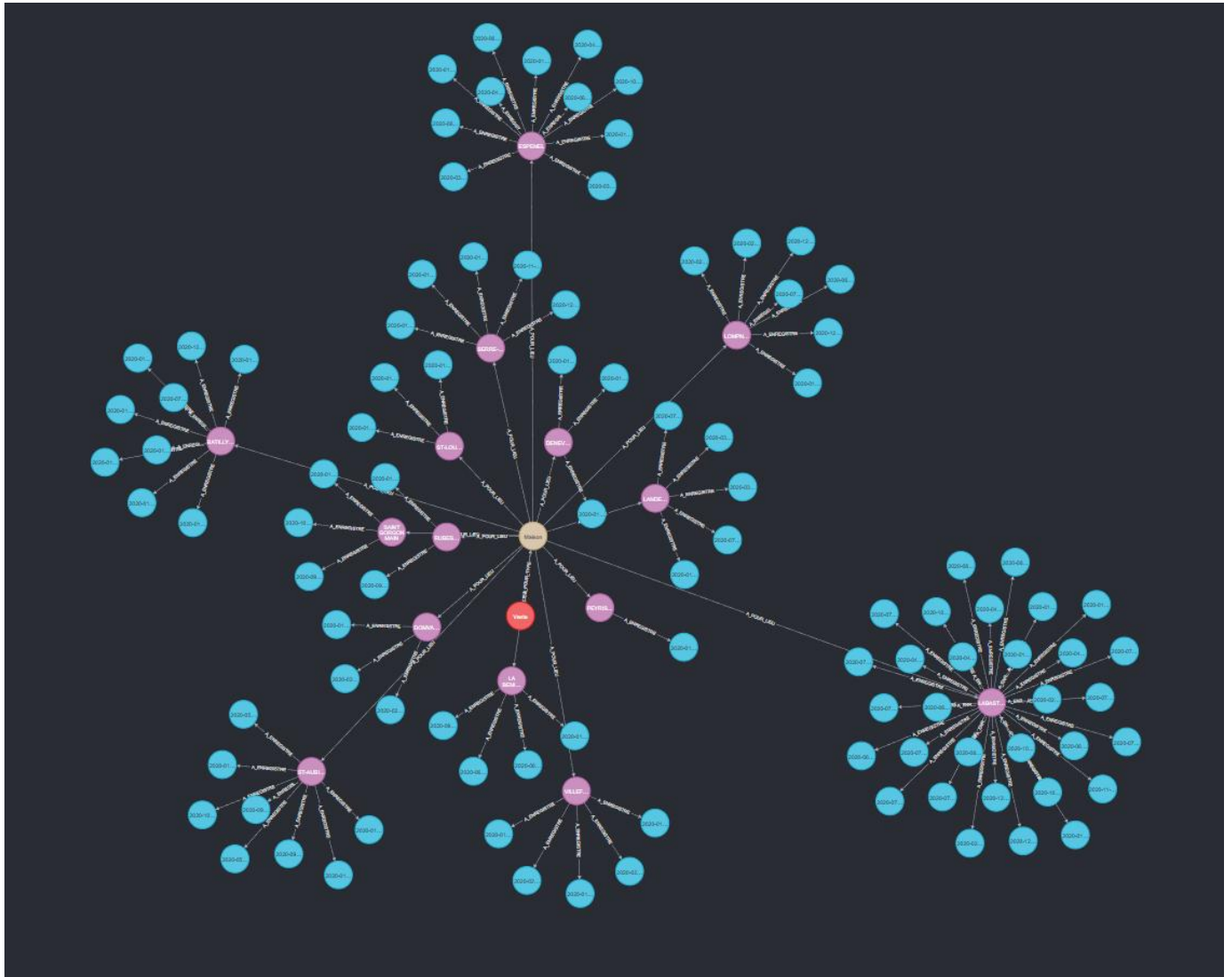
On peut voir qu'il y en a beaucoup, et que ça devient vite brouillon. Nous pouvons donc améliorer notre requête pour afficher, par exemple, les transactions qui concernent les maisons. Ainsi, nous pouvons avoir un affichage plus simple, plus clair avec cette requête :

```
MATCH (n:NatureMutation)-[:A_POUR_TYPE]->(t:TypeLocal {nom: "Maison"})
```

```
-[:A_POUR_LIEU]->(c:Commune)-[:A_ENREGISTRE]->(m:Mutation)
```

```
RETURN n, t, c, m
```

```
LIMIT 100
```



Maintenant, nous voyons clairement comment notre structure se décompose. Nous pouvons aussi voir les villes les plus présentes dans notre jeu de donnée.

Pour aller plus loin, on peut, encore une fois, afficher les communes par nombre de ventes. Cela nous donnerait donc la ligne de commande suivante :

```
MATCH (c:Commune)-[:A_ENREGISTRE]->(m:Mutation)
```

```
RETURN c.nom AS Commune, count(m) AS NombreMutations
```

ORDER BY NombreMutations DESC

LIMIT 100

```
neo4j$ MATCH (c:Commune)-[:A_ENREGISTRE]->(m:Mutation) RETURN c.nom AS Commune, count(m) AS NombreMutations ORDER BY NombreMutations DESC L...
```

	Commune	NombreMutations
1	"TOULOUSE"	28222
2	"NICE"	19416
3	"MONTPELLIER"	16286
4	"NANTES"	15616
5	"BORDEAUX"	12676
6	"LILLE"	10798
7		

Started streaming 100 records after 7 ms and completed after 1648 ms.

De cette manière, nous voyons que nous avons des résultats cohérents avec les données que nous avons dans notre base, ce qui montre que ces dernières ont bien été importées et que notre requête renvoie les bons résultats.

Maintenant, on peut essayer de chercher à faire une ville en particulier pour vérifier que les données ont toujours bien conformes :

MATCH (c:Commune {nom: "FOISSIAT"})-[:A_ENREGISTRE]->(m:Mutation)

RETURN m.date AS Date, m.valeur AS Valeur

ORDER BY m.date

LIMIT 100

```
neo4j$ MATCH (c:Commune {nom: "FOISSIAT"})-[:A_ENREGISTRE]->(m:Mutation) RETURN m.date AS Date, m.valeur AS Valeur ORDER BY m.date LIMIT 100
```

	Date	Valeur
1	"2020-01-13"	5275.0
2	"2020-01-13"	5275.0
3	"2020-01-17"	4000.0
4	"2020-01-17"	4000.0
5	"2020-01-22"	43295.62109375
6	"2020-01-22"	43295.62109375
7		

Started streaming 100 records in less than 1 ms and completed after 1 ms.

On peut aussi modifier un peu la requête les détails dans le return pour garder l’affichage en graphs :

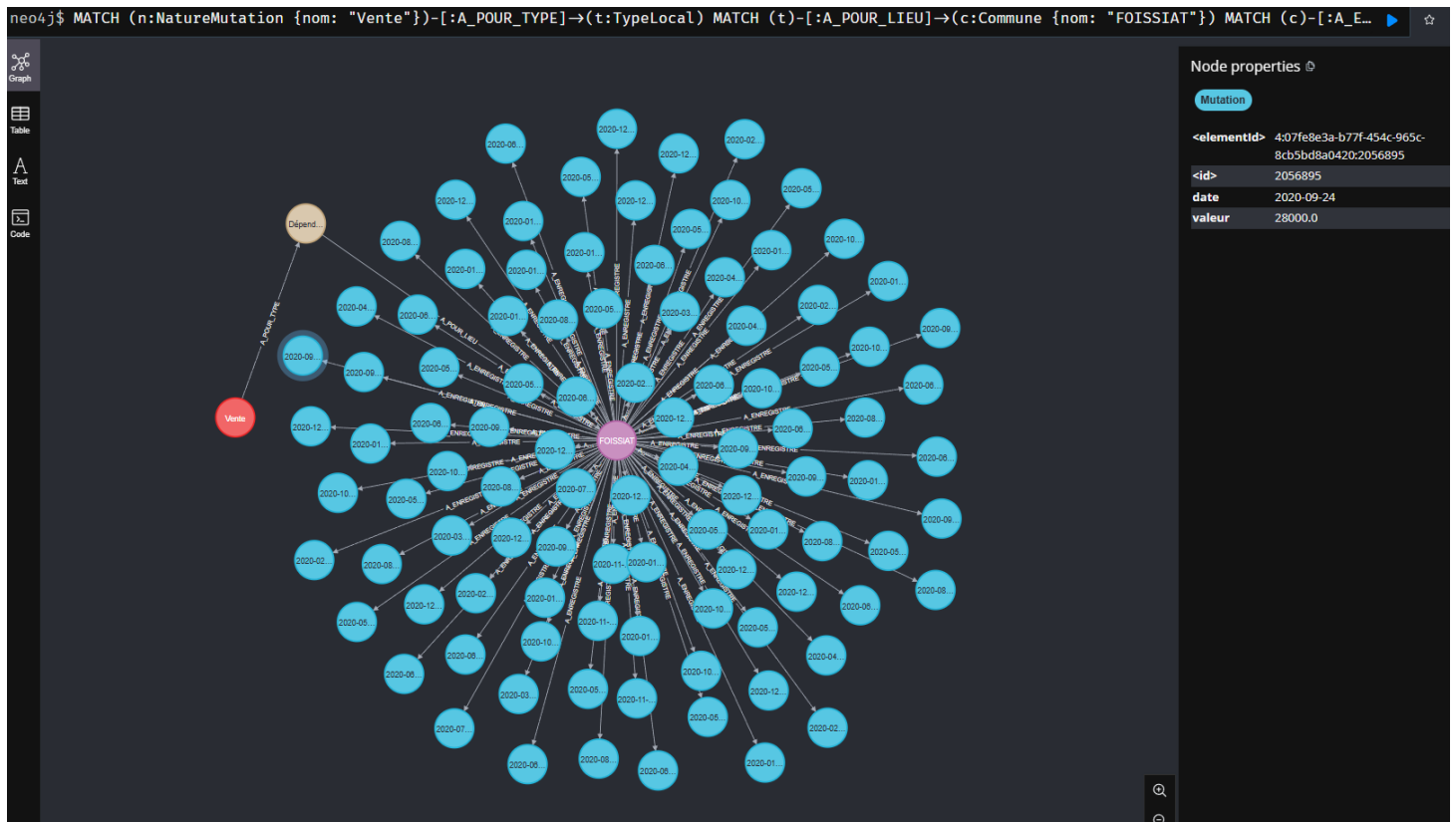
```
MATCH (n:NatureMutation {nom: "Vente"})-[:A_POUR_TYPE]->(t:TypeLocal)
```

```
MATCH (t)-[:A_POUR_LIEU]->(c:Commune {nom: "FOISSIAT"})
```

```
MATCH (c)-[:A_ENREGISTRE]->(m:Mutation)
```

```
WHERE t.nom <> "Inconnu"
```

```
RETURN n, t, c, m
```



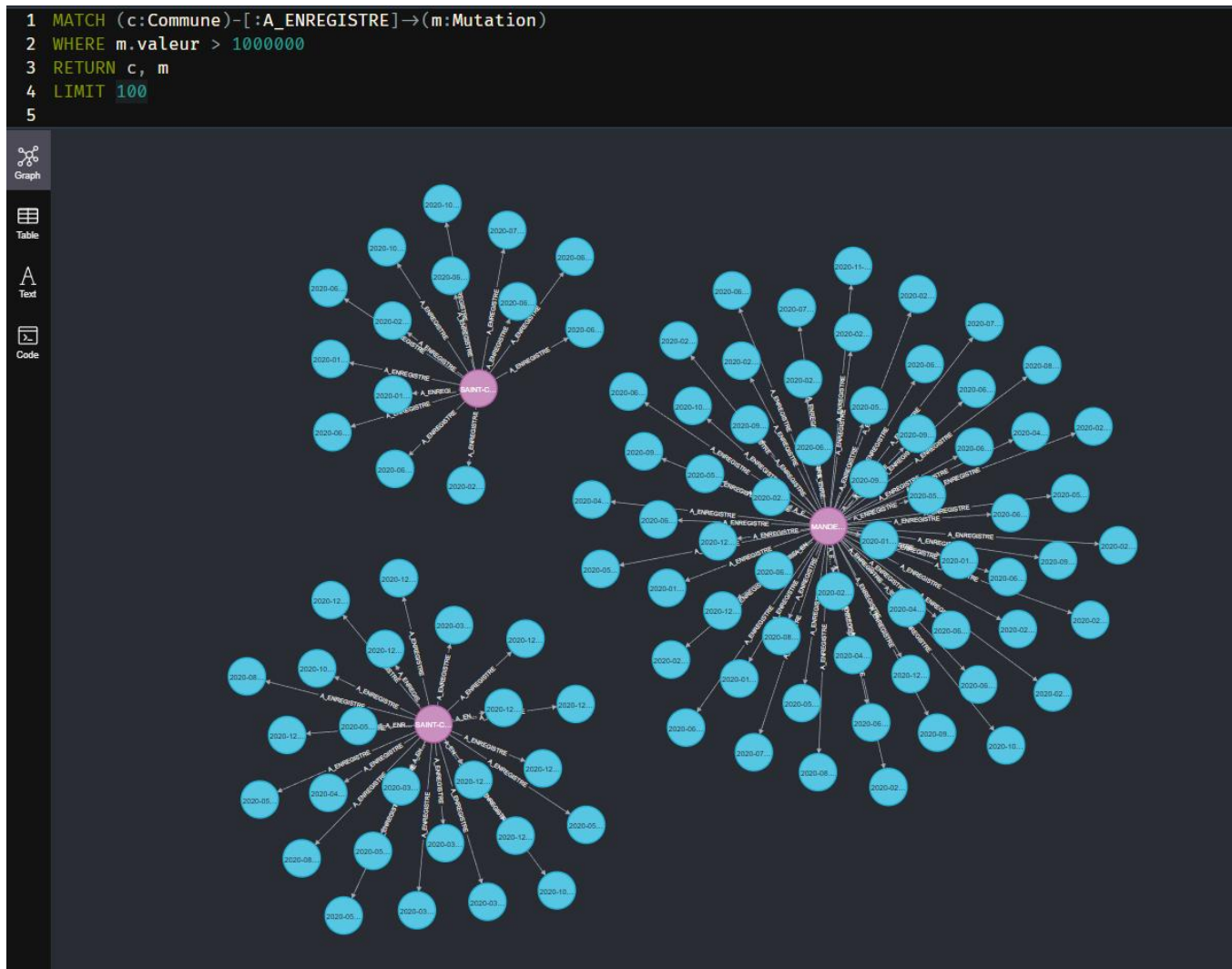
Nous pouvons aussi essayer de filtrer sur les transactions supérieures ou inférieures à un certain montant :

```
MATCH (c:Commune)-[:A_ENREGISTRE]->(m:Mutation)
```

```
WHERE m.valeur > 1000000
```

```
RETURN c, m
```

```
LIMIT 100
```



Pour finir, on peut afficher les différents types de biens ayant engrangé la plus haute valeur totale :

```

MATCH (t:TypeLocal)-[:A_POUR_TYPE]-(n:NatureMutation),
      (t)-[:A_POUR_LIEU]->(c:Commune)-[:A_ENREGISTRE]->(m:Mutation)

RETURN t.nom AS TypeLocal, sum(m.valeur) AS ValeurTotale

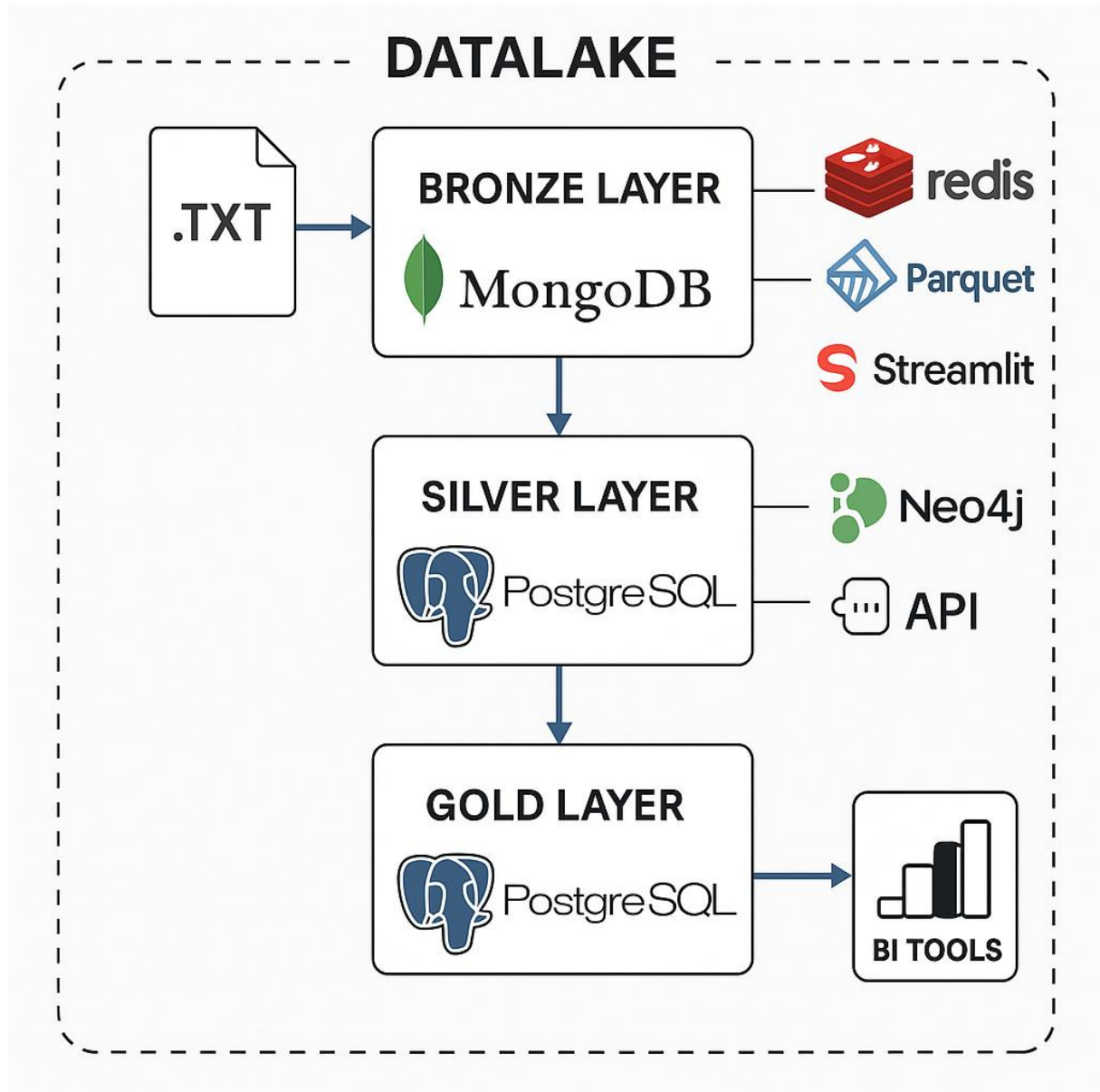
ORDER BY ValeurTotale DESC

```

	TypeLocal	ValeurTotale
1	"Inconnu"	4154567676569.751
2	"Maison"	4053696324103.9707
3	"Dépendance"	3965373977541.55
4	"Local industriel, commercial ou assimilé"	3830933669917.0615
5	"Appartement"	3777351974773.4453

Un datalake est mis en place :

Pour répondre au besoin de stockage et de traitement de nos données massives, nous avons donc mis en place un datalake. Ce dernier nous permet d'avoir différents types de données provenant de nos différentes sources (différents fichiers txt) et nous permet de les rendre accessible dans nos divers outils d'analyse.



C1

Notre datalake permet l'intégration de données de différents types. Nous avons :

- des données non structurées, issues de nos fichiers txt
- des données semi-structurées, issues du streaming des données. Nous stockons ces données dans notre base MongoDB.
- des données structurées, issues de notre base de données MongoDB et envoyées vers notre base PostgreSQL.

Ainsi, nous pouvons gérer tout type de données via notre structure.

C2

Notre solution est entièrement distribuée. Nous avons tous nos outils qui fonctionnent sur des conteneurs différents et dédiés (Spark, MongoDB, PostgreSQL, Redis, Neo4j, ...). Spark assure ainsi le traitement et la distribution des tâches garantissant la scalabilité de notre pipeline.

La redondance de notre architecture est assurée via notre couche `silver_layer`. Un prétraitement est effectué pour nettoyer les données qui sont envoyées de MongoDB vers PostgreSQL. Ainsi, l'architecture assure la redondance des données vers les différents systèmes. L'utilisation des volumes dockers garantit également la persistance des données au redémarrage de l'application (des scripts ont également été effectués pour redémarrer l'architecture partiellement ou totalement).

```
$ reset_complet.sh
$ reset_sans_postgre.sh
```

C3

Pour ce qui est de la surveillance de l'état du système, nous avons mis en place un monitoring des applications les plus critiques. De cette manière, nous pouvons avoir un suivi constant sur les données qui circulent dans notre infrastructure, et nous pouvons également, à l'avenir, faire une analyse dessus.

Exemples de fichiers que nous créons pour suivre les données :

```
▼ logs
  ≡ log_pipeline_spark_stream.txt
  ≡ statistiques_parquet.txt
  ≡ statistiques_redis.txt
```

Ces fichiers sont donc stockés en dur dans notre dossier `logs/`, et nous pouvons les consulter à tout instant. Nous avons donc un suivi en temps réel du fonctionnement du système et nous pouvons voir à tout instant le moindre point de blocage.

```
≡ log_pipeline_spark_stream.txt X
Partie_1 > logs > ≡ log_pipeline_spark_stream.txt
1116 [2025-05-14 14:49:56.325454] ✓ Batch 139 - 16761 lignes - 1.15s
1117 [2025-05-14 14:50:01.331355] ✓ Batch 140 - 17396 lignes - 1.24s
1118 [2025-05-14 14:50:06.840269] ✓ Batch 141 - 20000 lignes - 1.69s
1119 [2025-05-14 14:50:11.797876] ✓ Batch 142 - 20000 lignes - 1.68s
1120 [2025-05-14 14:50:15.540924] ✓ Batch 143 - 7318 lignes - 0.44s
1121 [2025-05-14 14:50:21.479287] ✓ Batch 144 - 21033 lignes - 1.35s
1122 [2025-05-14 14:50:25.770650] ✓ Batch 145 - 12958 lignes - 0.68s
```

Une API WEB Restfull est fonctionnelle :

Pour répondre au besoin de ce bloc, nous avons conçu une API Web RESTful permettant de gérer l'accès à des données foncières stockées dans une base PostgreSQL. L'objectif était double : rendre ces données consultables sous forme structurée, tout en garantissant leur sécurité via un système d'authentification et de limitation de requêtes.

Nous allons vous présenter le fonctionnement de l'API à la suite des explications techniques.

C1

Nous avons utilisé FastAPI pour concevoir cette dernière, car il permet une gestion simple des dépendances et une performance élevée. Nous avons associé cette API à notre base PostgreSQL pour le stockage des données, et utilisé SQLAlchemy comme ORM (Objet rationnel de mapping). Enfin, nous avons encapsulé toute la logique dans des conteneurs Docker pour garantir un déploiement reproductible, et assurer la sécurité fonctionnelle (conteneurisation) de notre projet global.

C2

Nos données retravaillées de la base PostgreSQL sont accessibles via une route GET */resultats*, avec la possibilité de filtrer selon plusieurs critères (code_postal, commune). Grâce à Pydantic, les réponses sont structurées, validées, et retournées au format JSON. Notre API est disponible sur cette adresse, une fois le conteneur lancé : *'http://localhost:8000/docs#'*.

C3

Nous avons limité la logique aux besoins réels : lecture filtrée des données de notre base, création d'utilisateurs et génération de tokens. La séparation claire des fichiers (routes, sécurité, modèles) assure une architecture maintenable. Nous avons choisi une complexité simple, tout en gardant une sécurité et une fiabilité des données traitées.

C4

Chaque utilisateur peut se créer un compte via POST */utilisateurs/*. Les mots de passe sont hachés avec bcrypt, ce qui assure la sécurité des ces derniers. L'identification se fait ensuite avec POST */login*, qui renvoie un token JWT. Ce token est nécessaire pour accéder aux données protégées. Une fois le token renseigné pour l'authentification, l'utilisateur peut alors faire des requêtes sur les données disponibles dans la route */résultats*.

C5

Les routes sensibles sont protégées via `@Depends(verify_token)`, qui vérifie le token JWT. Cela nous assure que seulement une personne avec un compte renseigné peut accéder aux données. Nous avons aussi mis en place une 'date de péremption' de chaque tokens. En effet, ces derniers ont une durée de vie de 30 minutes (modifiable). Il faudra ensuite que l'utilisateur en génère un nouveau. Pour finir, nous avons également mis en place une limitation de requêtes avec SlowAPI (5 requêtes/minute). De cette manière, nous nous assurons qu'aucune surcharge de l'API n'aura lieu.

C6

La rapidité d'accès a été assurée via des requêtes SQL optimisées et des limites de pagination (`limit(100)`). L'API répond en quelques millisecondes, et le Swagger UI permet de tester sans surcharger le serveur.

C7

Nous avons, tout au long du développement de l'API, voulu nous assurer de l'automatisation de cette dernière. Ainsi, nous avons veillé à garantir un processus d'intégration des données automatisé, lancement de l'application automatique, une conservation des données utilisations et sécurité des données sans intervention humaine.

Voici donc comment nous avons répondu à ces besoins :

- Conteneur docker automatisé : l'API est intégrée à notre environnement docker via un docker-compose. Ainsi, à chaque démarrage du projet, l'ensemble des services est lancé et interconnecté.
- Connexion automatique à notre base PostgreSQL : au lancement de notre conteneur, la connexion à la base de données se fait automatiquement.
- Ecriture automatique des logs : toutes les requêtes générant des logs sont automatiquement transférées dans notre fichier de logs. Cela nous facilite le suivi de notre API.
- Sécurisation et accès contrôlé : notre application inclut donc également une connexion/création d'utilisateur qui est entièrement automatisé par nos services. Une table utilisateur est créée dans notre base PostgreSQL pour répondre à ce besoin. Aucune intervention humaine n'est nécessaire lors de ce processus.

C8

Grâce à FastAPI, la documentation Swagger est générée automatiquement. Elle est disponible directement sur l'API, ce qui facilite la compréhension. Elle décrit les routes, les schémas attendus, et permet l'envoi de tokens dans l'interface. Un développeur externe peut utiliser l'API immédiatement.

Les réponses de l'API respectent également les codes HTTP standards (200, 401, 429). Les erreurs sont explicites : "token invalide", "trop de requêtes", etc. Cela facilite le débogage et l'utilisation côté client.

Nous avons aussi mis en place un fichier de logs, qui va stocker les messages générés par l'API. Ainsi, nous pouvons à tout moment récupérer ces données là pour vérifier que tout est bien fonctionnel. Cela nous permet de bien monitorer notre API.

C9

Nous utilisons git tout au long de notre projet. Git permet, au fur et à mesure de son utilisation, de gérer l'historique des versions de chaque fichier du projet. Ainsi, nous nous servons de Git comme contrôleur de version de notre projet. Ce choix nous apporte plusieurs bienfaits :

- Une bonne traçabilité des modifications de l'API.
- Un travail efficace à plusieurs, tout en assurant un suivi simple et clair.
- Restauration à d'anciennes versions de l'application.

Démonstration du fonctionnement de l'API :

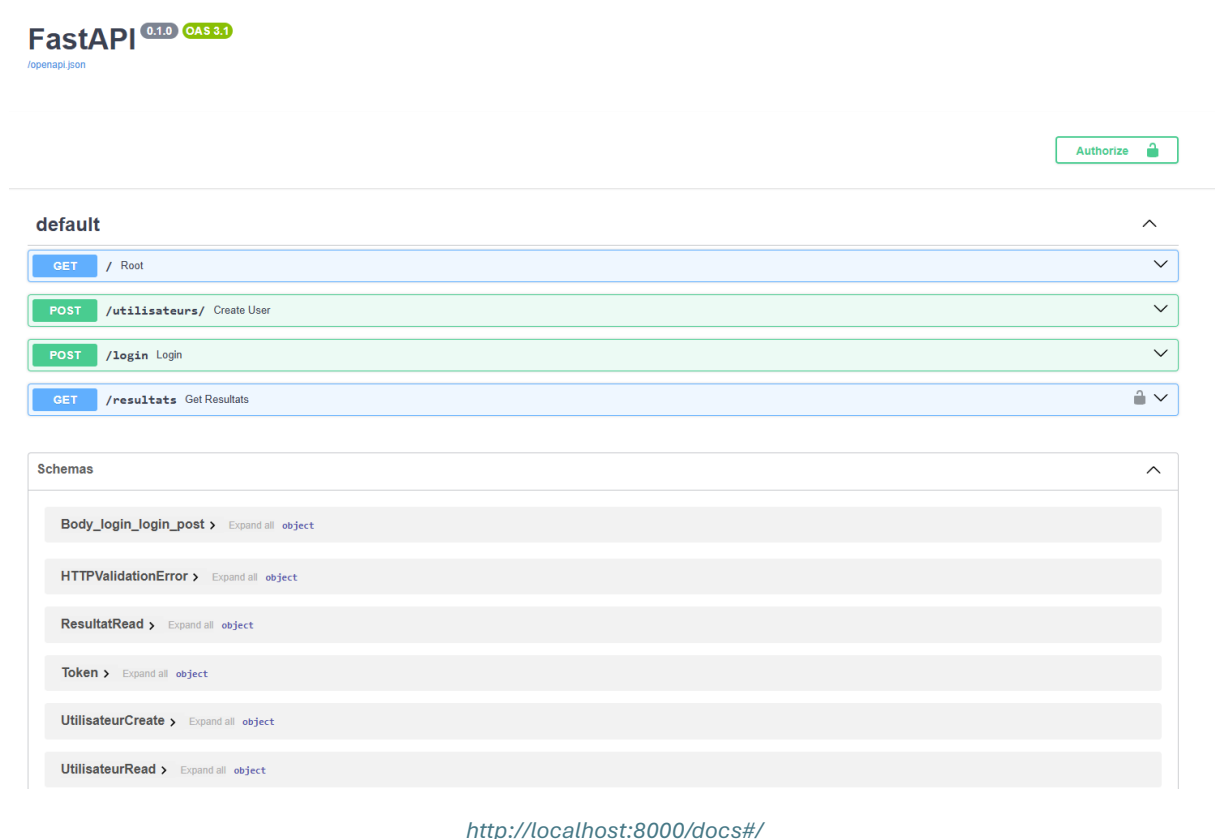
Tout d'abord, il faut s'assurer que l'API tourne bien. Nous pouvons vérifier tout simplement après le lancement de l'application, en allant dans un terminal et en lançant : *docker ps*

```
D:\Projet\projet_fin_etudes\projet_final\Partie 1> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
dcf91b2b11	partie_1-spark-runner	"/opt/bitnami/script..."	2 hours ago	Up 2 hours		spark-runner
5bf25bf8e7	partie_1-spark-worker	"/opt/bitnami/script..."	2 hours ago	Up 2 hours		partie_1-spark-worker-1
4e589ddd27	partie_1-spark	"/opt/bitnami/script..."	2 hours ago	Up 2 hours	0.0.0.0:8080->8080/tcp	partie_1-spark-1
92bc2f8950	partie_1-streamlit	"streamlit run app.p..."	2 hours ago	Up 2 hours	0.0.0.0:8501->8501/tcp	partie_1-streamlit-1
216484845c	partie_1-api-postgre	"/bin/sh -c 'uvicorn..."	2 hours ago	Up 2 hours	0.0.0.0:8000->8000/tcp	api_postgre
9eb21debda	bitnami/kafka:2.8.1	"/opt/bitnami/script..."	13 hours ago	Up 48 minutes	0.0.0.0:9092->9092/tcp	partie_1-kafka-1
9e98f828b9	postgres:14	"docker-entrypoint.s..."	13 hours ago	Up 2 hours	0.0.0.0:5432->5432/tcp	projet_final_postgres
1c7b951e38	redis:7	"docker-entrypoint.s..."	13 hours ago	Up 2 hours	0.0.0.0:6379->6379/tcp	partie_1-redis-1
8ff51f70a4	neo4j:5.18	"tini -g -- /startup..."	13 hours ago	Up 2 hours	0.0.0.0:7474->7474/tcp, 7473/tcp, 0.0.0.0:7687->7687/tcp	projet_final_neo4j
8beea07643	mongo:latest	"docker-entrypoint.s..."	13 hours ago	Up 2 hours	0.0.0.0:27017->27017/tcp	partie_1-mongodb
939c51cbb1	bitnami/zookeeper:latest	"/opt/bitnami/script..."	13 hours ago	Up 2 hours	2181/tcp, 2888/tcp, 3888/tcp, 8080/tcp	zookeeper

docker ps – Ici, le conteneur de l'API est surligné en jaune

Maintenant qu'on voit que notre API tourne bien, on peut se rendre sur son adresse, à savoir <http://localhost:8000/docs#/>. Une fois dessus, une interface comme ça se présente :



Nous pouvons voir les différentes routes dont nous aurons besoin :

- /utilisateurs : Nous pourrons nous créer un compte grâce à cette route.
- /login : Une fois notre compte créé, en nous connectant grâce à cette route, nous pourrons récupérer notre token, indispensable pour pouvoir faire des requêtes sur notre base de données.
- /resultats : Route principale de notre API, qui va nous retourner des résultats en fonction des éléments que nous allons lui donner.

Pour commencer, nous devons donc nous rendre sur la route */utilisateurs*. Nous allons nous créer un compte. En cliquant sur la route, un menu va dérouler. Il faut cliquer sur 'Try it out', en haut à droite de la route.

Nous devons ensuite renseigner les informations essentielles de notre compte, à savoir le nom, l'adresse mail et le mot de passe. Les informations sont envoyées en JSON sur l'API, et les données seront ensuite cryptées (pour les sensibles) via une méthode `bcrypt`. Cette dernière fait un hachage du mot de passe, qui le transforme ainsi en chaîne de caractère. Chaque chaîne de caractère est unique, même si deux utilisateurs ont le même mot de passe.

POST /utilisateurs/ Create User

Parameters Cancel Reset

No parameters

Request body required application/json

```
{
  "nom": "test",
  "email": "test@email.com",
  "mot_de_passe": "testmdp"
}
```

Execute Clear

Après cela, il faut exécuter le script json, et normalement, un message avec un 'response 200' devrait apparaître. Cela nous indiquera que notre requête a correctement fonctionné.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/utilisateurs/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "nom": "test",
    "email": "test@email.com",
    "mot_de_passe": "testmdp"
  }'
```

Nous pouvons aussi aller vérifier dans la base de données postgres SQL que notre utilisateur à bien été créé.

utilisateurs 1

SELECT id, nom, email, mot_de_passe FROM public.utilisateurs

	123 id	A-Z nom	A-Z email	A-Z mot_de_passe
1	1	Victor	victor@email.com	\$2b\$12\$JFHx9nErGJdUotLN6ShsTO04hMDDULR0Rv2UcuukpOBYPD5NnWHyK
2	2	test	test@email.com	\$2b\$12\$Z7WFrg.p4uTARIQrHAEJnupchGzJTYaZNBs0hXoRHEK2eq57PfK/i

Maintenant que l'utilisateur existe bien, il faut se connecter avec ses identifiants pour récupérer le token, qui nous permettra de faire des requêtes sur notre API.

Ainsi, nous pouvons aller dans la route `/login`, cliquer sur ‘Try it out’, et renseigner les valeurs suivantes :

grant_type : password

username : mettre votre email (test@email.com)

password : mettre votre mot_de_passe (testmdp)

scope : laisser/mettre vide

client_id : laisser/mettre vide

client_secret : laisser/mettre vide

Une fois que l’on a bien renseigné ces informations, nous pouvons exécuter en cliquant en dessous :

POST /login Login

Parameters

No parameters

Request body required

application/x-www-form-urlencoded

grant_type
string | (string | null)
pattern: ^password\$
password

Send empty value

username * required
string
test@email.com

password * required
string
testmdp

scope
string
scope

Send empty value

client_id
string | (string | null)
client_id

Send empty value

client_secret
string | (string | null)
client_secret

Send empty value

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=password&username=test340email.com&password=testmdp&scope=&client_id=&client_secret='
```

Request URL

http://localhost:8000/login

Server response

Code Details

200

Response body

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ0ZXN0QGVtYmVzLnV4cCI6MTc0MDA4NTA3Mj0. gID32B9Mpu0F9B4Sbs5JRQEDMoATbJCQXrgqY1FK1iM",
  "token_type": "bearer"
}
```

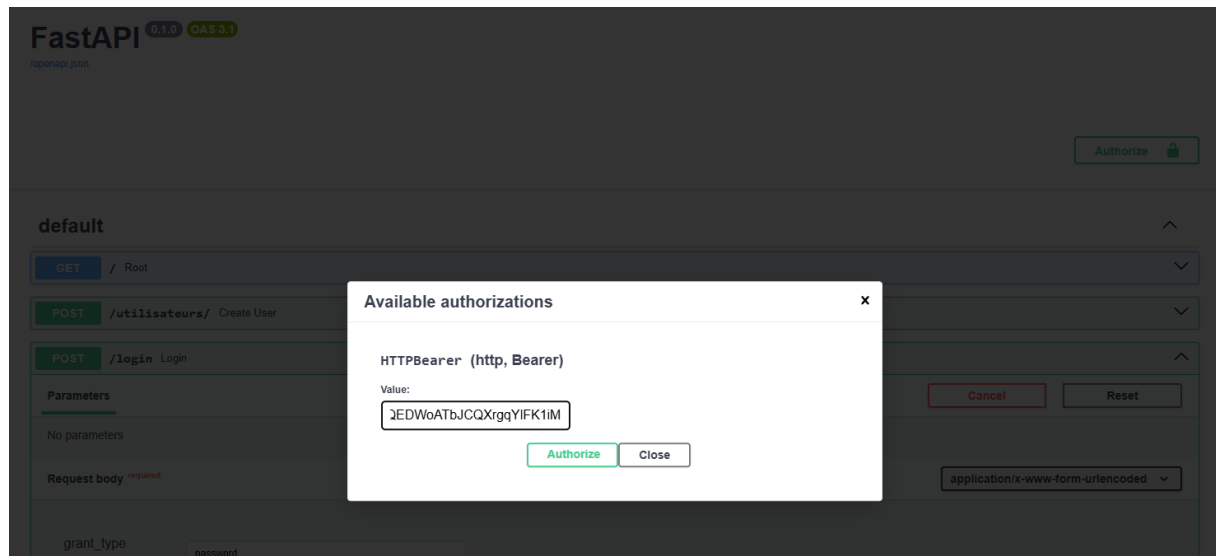
Download

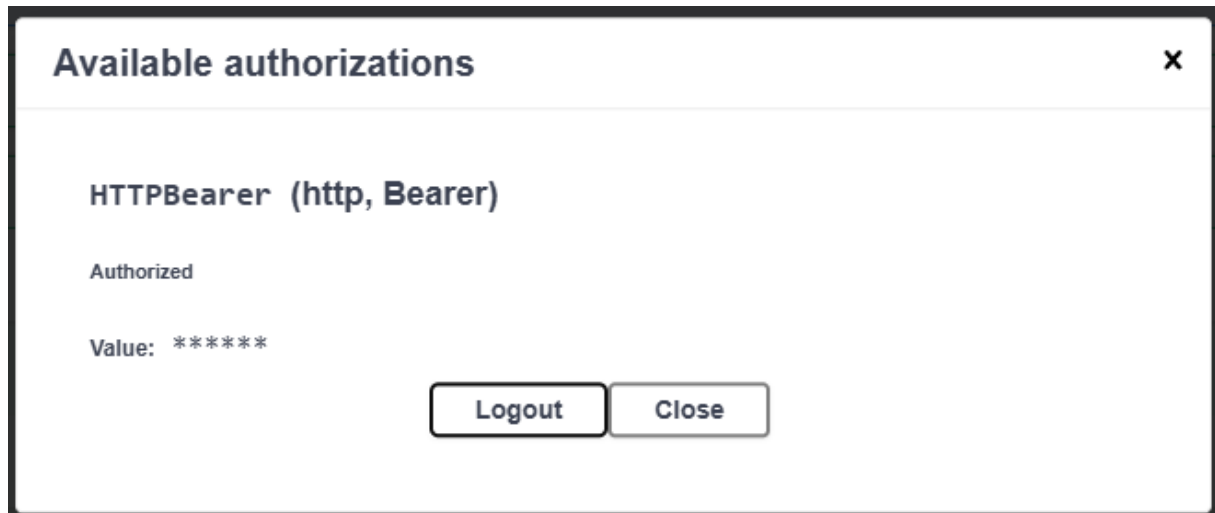
Response headers

```
content-length: 177
content-type: application/json
date: Sat, 24 May 2025 10:41:11 GMT
server: uvicorn
```

On peut voir sur la réponse que le code de retour est bien 200, ce qui indique que la requête a correctement fonctionné.

Il nous suffit maintenant de récupérer le token disponible dans la réponse, et de se connecter avec. Tout en haut de la page, il y a un petit onglet avec marqué 'Authorize', et en cliquant dessus il nous suffira d'entrer le token :





Après avoir bien renseigné le token, nous pouvons voir que nous sommes bien connectés (Authorized), et que nous pouvons ensuite faire des requêtes sur notre API. Il faut tout de même noter que chaque token, pour des raisons de sécurité, n'est disponible que 30 minutes. Il faudra donc se reconnecter de cette manière toutes les 30 minutes.

Pour faire une requête sur l'API, il faut simplement se rendre sur la route `/resultats`, indiquer le code postal d'une ville ainsi que son nom, et l'API va nous retourner toutes les informations disponibles sur cette dernière :

GET

/resultats

Get Resultats



Parameters

Cancel

Name	Description
code_postal	
string	73490
(query)	
commune	
string	LA RAVOIRE
(query)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8000/resultats?code_postal=73490&commune=LAX20RAVOIRE' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIj02XN0QGVtYmIsImVhbSI6ImV4cCI6MTc0ODAA4NTA3Mj0.gID3ZB9Mpu0F9B4SbsS3RQEDMoATb3CQXrgqY1FK1iM'
```

Request URL

```
http://localhost:8000/resultats?code_postal=73490&commune=LAX20RAVOIRE
```

Server response

Code	Details
200	Response body <pre>{ { "id_unique": "0adb5b2378e4e0b093ef25bf26faa7c548d7b1014218703a77c6d760716c8b8b", "code_postal": "73490", "commune": "LA RAVOIRE", "valeur_fonciere": 274800, "type_local": "Maison", "surface_reelle_bati": 150, "nature_mutation": "Vente", "fichier_source": "ValeursFoncieries-2020", "date_mutation": "2020-01-13", "nombre_pieces_principales": 0, "part_date": "2025-05-14" }, { "id_unique": "146234b206fa27589bc51f1d1a927c6038e31b619e90d31101886a43bd03cd3", "code_postal": "73490", "commune": "LA RAVOIRE", "valeur_fonciere": 331000, "type_local": "Appartement", "surface_reelle_bati": 81, "nature_mutation": "Vente en l'état futur d'achèvement", "fichier_source": "ValeursFoncieries-2020", "date_mutation": "2020-01-10", "nombre_pieces_principales": 4, "part_date": "2025-05-14" }, { "id_unique": "7b6c6e2572025057222ec8a2be3126fa8d2f15dab28f8520f5b5ea234ebcd8aa" } }</pre> Download

exemple de retour de l'API sur les données de 'La Ravoire'

BLOC3 - Implémenter et optimiser des solutions de stockage et de traitement de données sur le cloud

Objectifs du bloc :

Mettre en œuvre des solutions de stockage de données dans le cloud pour permettre aux entreprises d'explorer leurs données en implémentant des techniques et stratégies adaptées à leur utilisation et en choisissant les types de stockage et d'archivages appropriés
Concevoir et développer des pipelines et des solutions de traitement de données dans le cloud en chargeant les données en les transformant et les mettant à disposition des utilisateurs pour permettre leur exploitation
Mettre en place une politique de sécurité des données dans le cloud en développant une stratégie de chiffrement et de gestion des identités et des accès et protéger les données en étant et être conforme aux règlements en vigueur
Optimiser les solutions de stockage et de traitement des données dans le cloud en définissant des indicateurs de performance pour assurer la disponibilité des services et optimiser les coûts

Analyse de la Pollution, de la Population et des Conditions Météorologiques en France

Introduction

Ce rapport présente une analyse complète de la relation entre la pollution, la population et les conditions météorologiques dans différentes villes de France. L'objectif de ce projet est de comprendre les facteurs qui influencent la pollution dans ces villes en utilisant un modèle de données relationnel.

Objectif du Projet

L'objectif de ce projet est d'analyser la pollution en France en fonction des différentes variables telles que la population et les conditions météorologiques. En utilisant une base de données relationnelle comprenant des informations sur la pollution, la population et les conditions météorologiques, nous avons pu identifier les relations entre ces différentes variables et visualiser les résultats à l'aide de Power BI.

Avant de détailler les étapes, présentons l'architecture en médaillon (Bronze, Silver, Gold) :

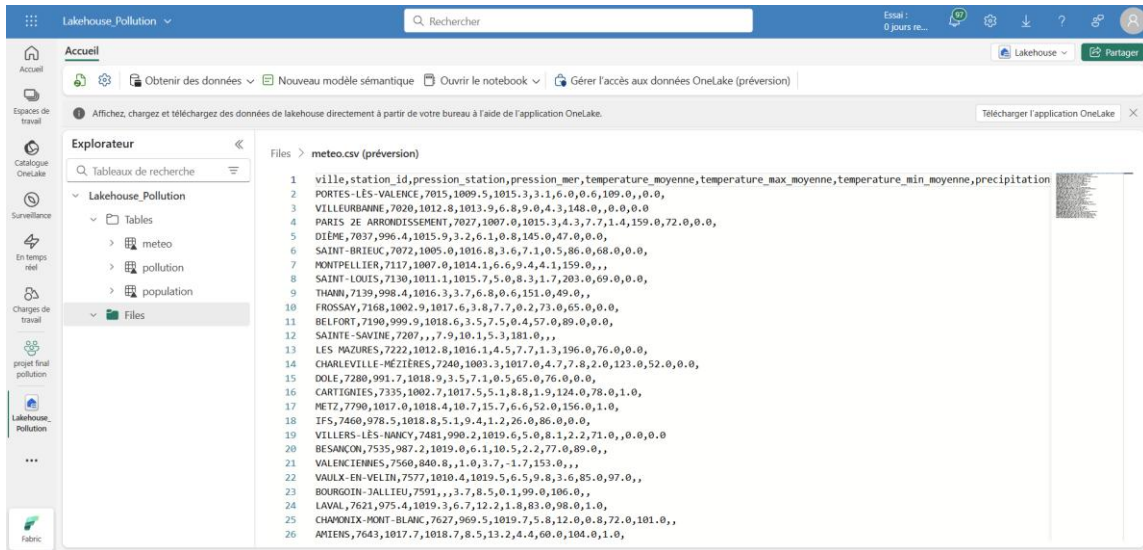
Couche Bronze (Données Brutes)

La couche Bronze contient les données originales, directement importées depuis les fichiers sources (pollution.csv, meteo.csv, population.csv).

Ces données sont stockées telles quelles, sans transformation.

Contenu : Données parfois incomplètes ou non nettoyées
Doublons possibles
Types de données non uniformes
Colonnes non normalisées.

Rôle : Conserver une trace fidèle de la source, et servir de base à toute transformation future.



Voici sur les fichiers (brutes) importées dans le Lakehouse depuis les fichiers CSV.

Couche Silver (Données Nettoyées et Structurées)

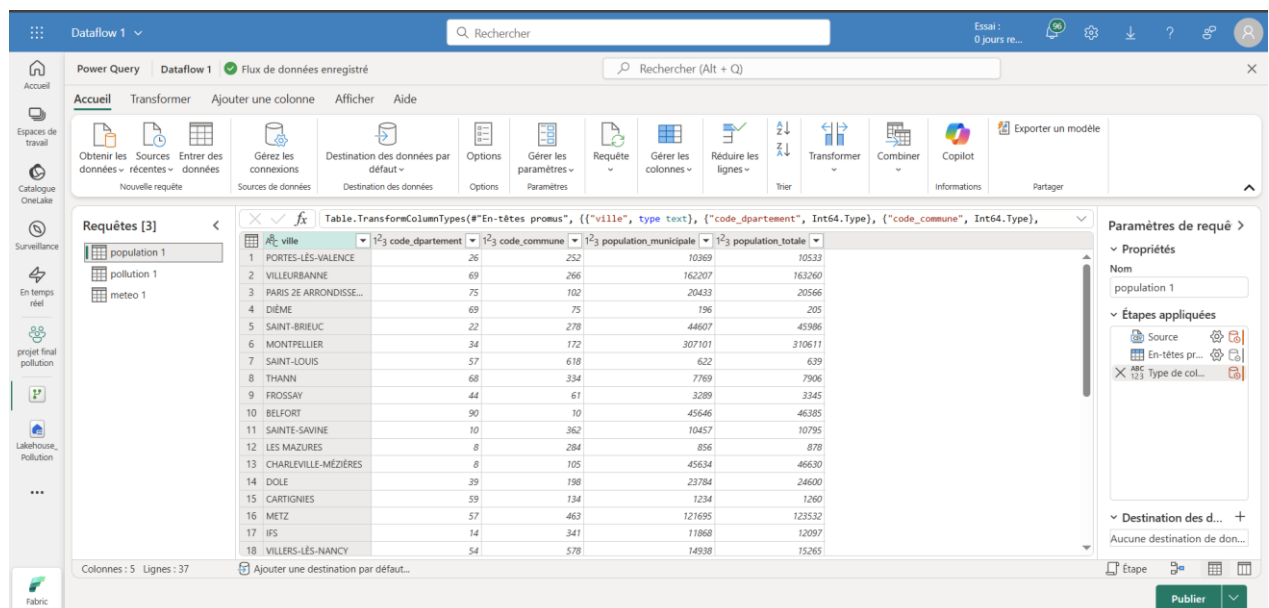
La couche Silver représente le résultat des traitements appliqués aux données brutes.

Les données sont ici filtrées, normalisées et jointes pour permettre une exploitation fluide.

Traitements appliqués : Suppression des doublons Uniformisation des noms de colonnes et des villes Correction des types de données (ex : température en float, population en int)

Jointures entre les tables sur le champ ville Sélection des colonnes utiles uniquement

Rôle : Proposer un socle de données propre, cohérent et prêt à être modélisé Supporter la construction de dimensions (SCD) et d'indicateurs métiers



Voici l'étape de nettoyage appliqué dans Power Query lors du passage de Bronze à Silver

Pour passer de la couche Bronze à la couche Silver, j'ai utilisé les Dataflows Gen2 dans Microsoft Fabric.

J'ai appliqué manuellement plusieurs étapes de transformation dans Power Query : suppression de colonnes inutiles, normalisation des noms de villes, suppression des doublons, et correction des types de données.

J'ai aussi vérifié manuellement que toutes les villes étaient présentes dans les 3 fichiers pour pouvoir les joindre correctement ensuite.

Chaque table nettoyée a été chargée en tant que table dans le Lakehouse (couche Silver), prête à être modélisée. J'ai également vérifié la cohérence des données (pas de doublons de villes, bonnes correspondances entre population et pollution, etc.).

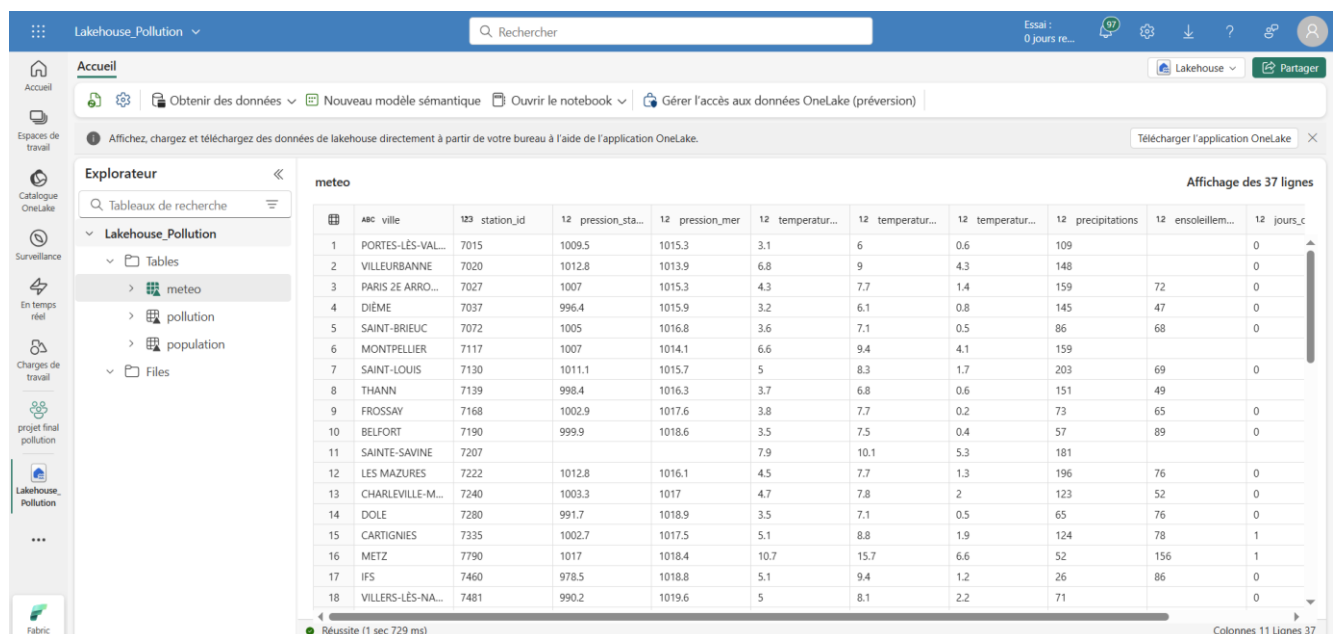
Couche Gold (Données Modélisées pour l'Analyse)

La couche Gold contient les données modélisées et optimisées pour la visualisation et l'analyse, en particulier dans Power BI.

C'est ici qu'interviennent les concepts de tables de faits et tables de dimensions, ainsi que la gestion des SCD.

Caractéristiques : Tables prêtes à l'analyse (format tabulaire propre) Clés primaires et relations bien définies (via ville) Mesures calculées (ex : pollution par habitant) Intégration avec Power BI via un modèle sémantique

Rôle : Offrir une couche performante, documentée et exploitable pour l'utilisateur final
Alimentation directe des rapports Power BI



	abc ville	123 station_id	12 pression_sta...	12 pression_mer	12 temperatur...	12 temperatur...	12 temperatur...	12 precipitations	12 ensoleille...	12 jours_c
1	PORTES-LÈS-VALL...	7015	1009.5	1015.3	3.1	6	0.6	109		0
2	VILLEURBANNE	7020	1012.8	1013.9	6.8	9	4.3	148		0
3	PARIS 2E ARRO...	7027	1007	1015.3	4.3	7.7	1.4	159	72	0
4	DIÈME	7037	996.4	1015.9	3.2	6.1	0.8	145	47	0
5	SAINT-BRIEUC	7072	1005	1016.8	3.6	7.1	0.5	86	68	0
6	MONTPELLIER	7117	1007	1014.1	6.6	9.4	4.1	159		0
7	SAINT-LOUIS	7130	1011.1	1015.7	5	8.3	1.7	203	69	0
8	THANN	7139	998.4	1016.3	3.7	6.8	0.6	151	49	0
9	FROSSAY	7168	1002.9	1017.6	3.8	7.7	0.2	73	65	0
10	BELFORT	7190	999.9	1018.6	3.5	7.5	0.4	57	89	0
11	SAINT-LOUIS	7207			7.9	10.1	5.3	181		0
12	LES MAZURES	7222	1012.8	1016.1	4.5	7.7	1.3	196	76	0
13	CHARLEVILLE-M...	7240	1003.3	1017	4.7	7.8	2	123	52	0
14	DOLE	7280	991.7	1018.9	3.5	7.1	0.5	65	76	0
15	CARTIGNIES	7335	1002.7	1017.5	5.1	8.8	1.9	124	78	1
16	METZ	7790	1017	1018.4	10.7	15.7	6.6	52	156	1
17	IFS	7460	978.5	1018.8	5.1	9.4	1.2	26	86	0
18	VILLERS-LÈS-NA...	7481	990.2	1019.6	5	8.1	2.2	71		0

Voici la Table Météo nettoyées et structurées prêtes à être modélisées.

Maintenant voyons voir en détail les étapes de préparation et le traitement de données.

Détail complet des étapes de préparation et de traitement des données

1. Lecture et Exploration des Fichiers Sources

- **Fichiers Sources :**
 - **Pollution.csv :** Contient les mesures de pollution (PM2.5, PM10, NO2, O3, CO) par ville et par date.
 - **Météo.csv :** Contient les conditions météorologiques par ville et par mois (température moyenne, maximale, minimale, précipitations, ensoleillement).
 - **Population.csv :** Contient les informations démographiques (population municipale et population totale) par ville.
- **Objectif :** Comprendre la structure de chaque fichier (colonnes, types de données, valeurs manquantes) afin de déterminer les étapes de nettoyage nécessaires.

2. Nettoyage des Données

L'objectif du nettoyage est de s'assurer que les données sont cohérentes, sans doublons ni erreurs, et prêtes pour une intégration fluide.

- **Harmonisation des noms de colonnes :**
 - Les noms de colonnes ont été standardisés pour être uniformes dans les trois fichiers.
 - Exemple : Ville, Polluant, Valeur, Unité, Date dans Pollution.csv, et Température Moyenne, Pluie, Ensoleillement dans Météo.csv.
- **Normalisation des noms de villes :**
 - Les noms de villes ont été convertis en majuscules pour éviter les erreurs de jointure (par exemple, "paris" est devenu "PARIS").
 - Les accents et les caractères spéciaux ont été supprimés pour une meilleure compatibilité.
- **Suppression des lignes et colonnes inutiles :**
 - Les lignes avec des valeurs nulles dans les colonnes essentielles (Ville, Valeur, Date) ont été supprimées.
 - Les colonnes sans utilité analytique (ID technique, commentaires) ont été retirées.

3. Jointure des Fichiers

- Les trois fichiers (Pollution, Météo, Population) ont été fusionnés sur la colonne Ville, qui est commune aux trois jeux de données.

- Pour garantir la cohérence, seules les villes présentes dans les trois fichiers ont été conservées.
- Cette étape a permis de s'assurer que chaque ville présente dans l'analyse dispose des trois dimensions :
 - Les niveaux de pollution
 - Les conditions météorologiques
 - Les données de population

4. Création des Fichiers Finaux Filtrés

- Après la jointure, trois fichiers finaux ont été générés :
 - **Pollution_Filtré.csv** : avec les valeurs de pollution par ville et par date.
 - **Météo_Filtré.csv** : avec les conditions météorologiques moyennes par ville.
 - **Population_Filtré.csv** : avec les valeurs de population par ville.
- Ces fichiers contiennent uniquement les villes ayant des données complètes pour les trois dimensions.

Implémentation dans Microsoft Fabric :

Microsoft Fabric a été utilisé pour centraliser les données, améliorer la performance et faciliter la gestion du projet.

1. Importation des Données dans Microsoft Fabric (Lakehouse)

- Les trois fichiers nettoyés (Pollution_Filtré.csv, Météo_Filtré.csv, Population_Filtré.csv) ont été importés dans une zone de stockage de Microsoft Fabric appelée Lakehouse Fabric.
- Cette zone de stockage permet de gérer les fichiers de manière sécurisée dans un environnement cloud.

2. Création des Tables de Données

- Trois tables distinctes ont été créées dans le Lakehouse Fabric :
 - **Table Pollution** : contenant les mesures de pollution avec les colonnes Ville, Polluant, Valeur, Unité, Date.
 - **Table Météo** : contenant les conditions météorologiques (Température Moyenne, Précipitations, Ensoleillement).

- **Table Population** : contenant les valeurs de population par ville (Population Municipale, Population Totale).

3. Mise en Place d'un Modèle Sémantique (Schéma en étoile)

- Un modèle sémantique a été créé pour organiser les données sous forme d'un schéma en étoile (Schéma en étoile) :
 - **Table de faits : Pollution**
 - Les mesures de pollution sont les données les plus fréquemment analysées (PM2.5, PM10, etc.).
 - **Tables de dimensions :**
 - Dimension Météo : pour fournir les informations climatiques associées à chaque ville.
 - Dimension Population : pour fournir les données démographiques par ville.

4. Création des Relations entre les Tables (voir le MCD)

- Les relations entre les tables ont été définies sur la colonne Ville :
 - Pollution → Météo (1:N) : Une ville peut avoir plusieurs enregistrements de météo (par mois).
 - Pollution → Population (1:1) : Une ville a une seule valeur de population.
- Validation des Types de Données et des Jointures :
 - Les types de données ont été vérifiés (texte pour les villes, numérique pour les valeurs de pollution).
 - Les jointures ont été testées pour s'assurer qu'elles ne produisent pas de doublons ou de valeurs nulles.

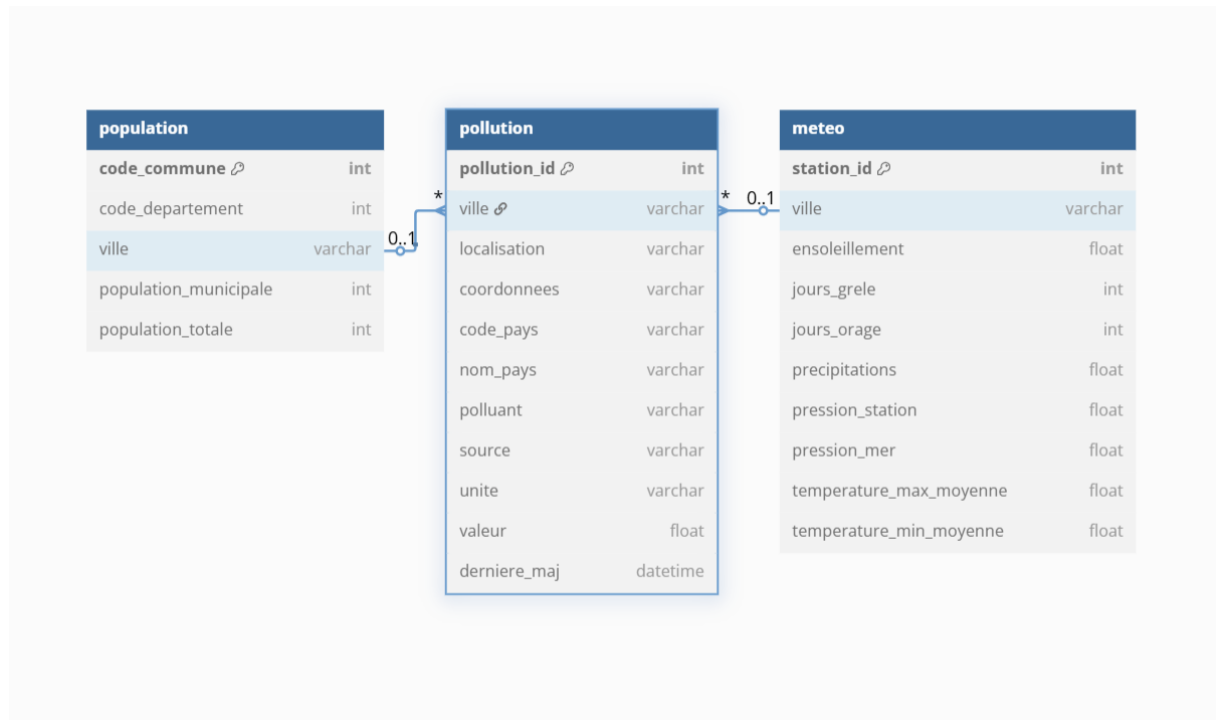
Les données utilisées dans ce projet proviennent de trois sources principales :

- La table "pollution" qui contient les informations sur les différents types de polluants mesurés dans les villes de France.
- La table "population" qui fournit des informations sur la population municipale et totale par ville.

- La table "météo" qui rassemble les données météorologiques telles que la température, l'ensoleillement, les précipitations, etc.

Modèle Conceptuel de Données (MCD)

Voici le modèle conceptuel de données utilisé dans ce projet :

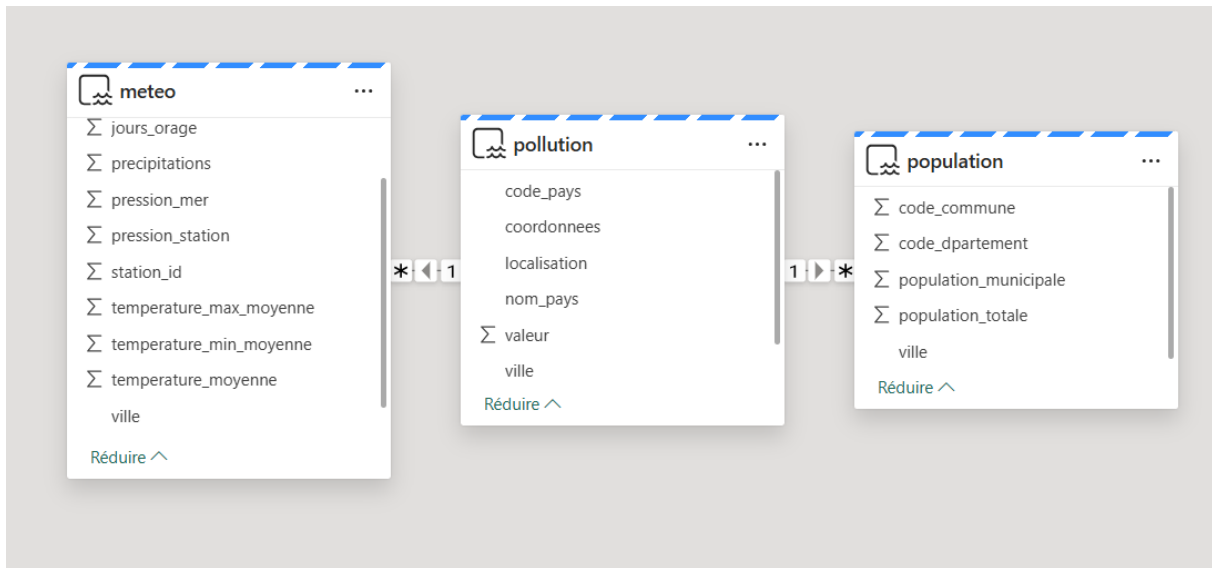


Cardinalités :

- Une ville peut avoir plusieurs relevés de pollution : (1,N)
- Une ville peut avoir plusieurs mesures météo (par mois) : (1,N)
- Une ville a une seule entrée de population dans le fichier fourni : (1,1)

Modèle Sémantique

Le modèle sémantique suivant a été conçu pour faciliter les analyses dans Power BI :



Application de SCD 1 sur la Table Population

SCD 1 est une méthode de gestion des changements dans les dimensions de données.

Les valeurs sont mises à jour sans conserver d'historique. Seule la valeur la plus récente est conservée.

La table Population contient les valeurs de population pour chaque ville. Lorsque la population d'une ville change (par exemple, mise à jour annuelle des données de l'INSEE), la nouvelle valeur remplace l'ancienne.

Pourquoi SCD 1 pour Population ?

La population est une donnée qui change rarement, et l'historique n'est pas essentiel pour l'analyse de la pollution.

Ce choix simplifie le modèle en gardant uniquement les valeurs les plus récentes.

Application de SCD 2 sur la Table Pollution

SCD 2 est une méthode qui conserve un historique complet des valeurs.

Chaque mise à jour des valeurs génère un nouvel enregistrement avec une date de début et de fin de validité.

Les valeurs de pollution sont enregistrées avec une date (par exemple, 1er janvier 2023).

Chaque nouveau relevé de pollution (par exemple, 2 janvier 2023) est enregistré comme une nouvelle ligne dans la table.

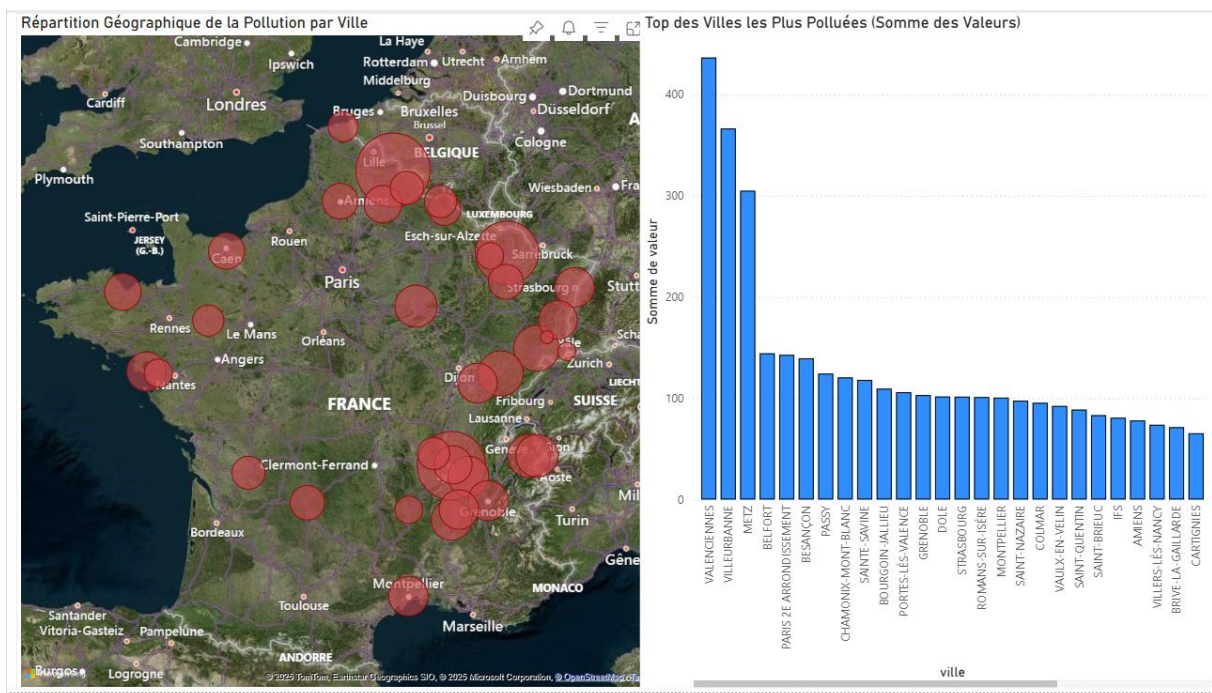
Les anciennes valeurs sont conservées pour permettre une analyse historique.

Pourquoi SCD 2 pour Pollution ?

La pollution est une donnée qui change fréquemment, et il est important de suivre l'évolution au fil du temps. Conserver l'historique permet de comparer les valeurs de pollution entre différentes périodes (saisons, années).

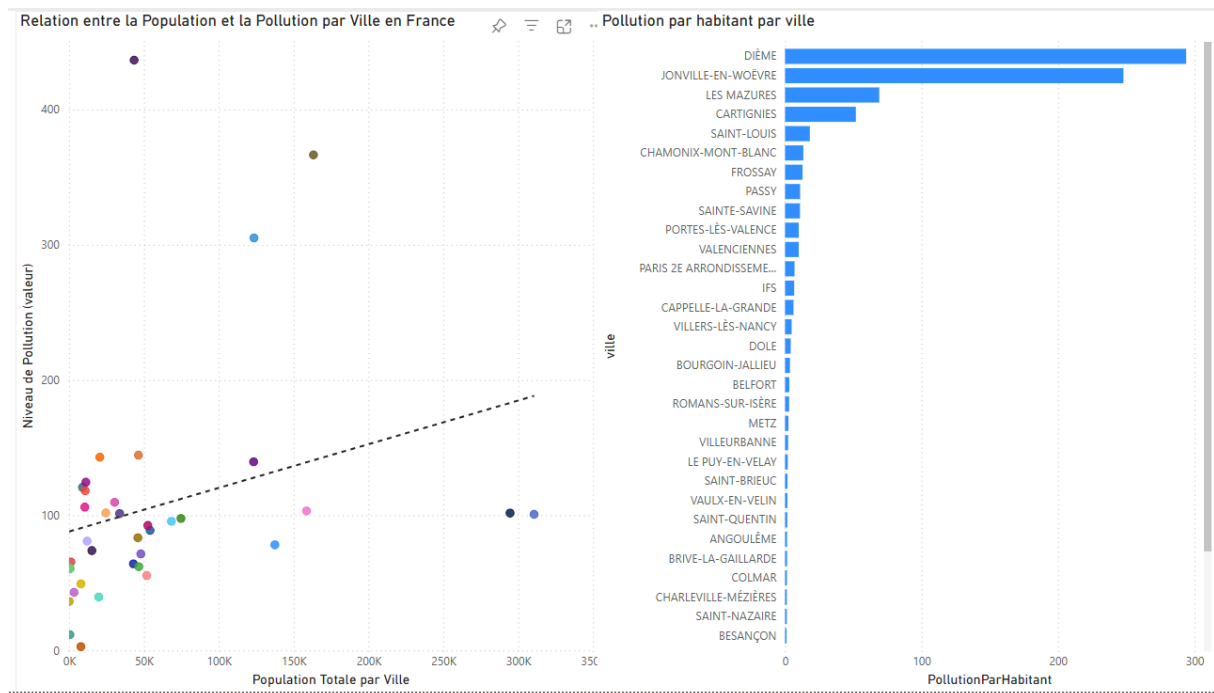
Analyse Du rapport : Power BI

Feuille 1 : Analyse de la Pollution par Ville



- Une carte géographique présente la répartition des niveaux de pollution par ville.
- Un histogramme affiche les villes les plus polluées.
Une carte géographique permet de visualiser rapidement les zones géographiques les plus exposées à la pollution. Elle met en évidence les villes à forte concentration de polluants, facilitant l'identification de foyers critiques.

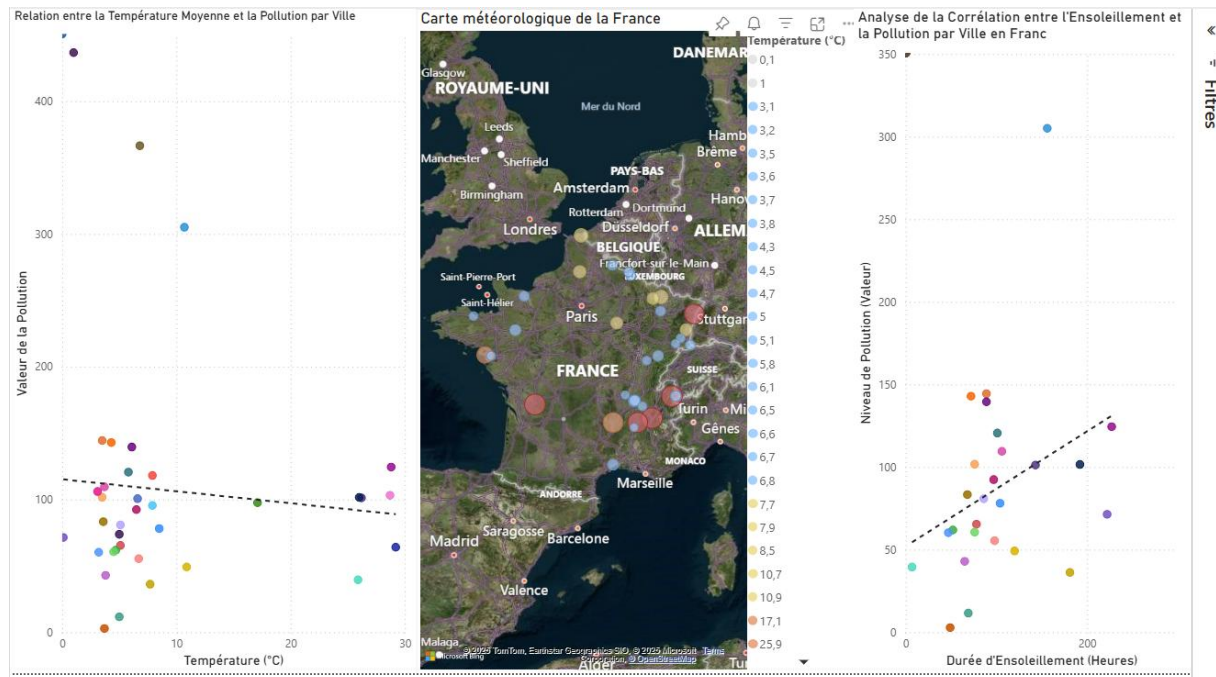
Feuille 2 : Corrélation entre Population Totale et Niveau de Pollution



- A gauche : Un nuage de points montre la relation entre la population et les niveaux de pollution.
- A droite : Un classement des villes par pollution par habitant est également affiché.

Le nuage de points accompagné de la droite de régression révèle une tendance générale selon laquelle les villes les plus peuplées présentent aussi des niveaux de pollution plus élevés. Cela suggère un lien potentiel entre densité urbaine et pollution atmosphérique.

Feuille 3 : Analyse des Conditions Météorologiques et de la Pollution



- Une analyse de la relation entre la température moyenne et la pollution.
En effet d'après la droite de régression qui est légèrement décroissante, cela nous affirme qu'il existe une faible corrélation entre la température et la pollution, plus la température plus la pollution est élevée.
- Une carte des conditions météorologiques et leur impact sur la pollution.
- Et une dernière analyse qui montre la corrélation entre l'Ensoleillement et la pollution par ville en France, la droite de régression est croissante donc plus il y a d'ensoleillement plus il y a de pollution.

Sécurité et conformité :

Dans le cadre de ce projet, les données ont été stockées et manipulées dans un environnement sécurisé fourni par Microsoft Fabric, dont l'accès est restreint via l'authentification Azure Active Directory.

Les transformations de données ont été réalisées dans Power Query à travers les Dataflows Gen2, et les analyses ont été partagées via Power BI Service, garantissant ainsi une traçabilité et une gestion des accès conformes aux standards de sécurité.

Le code source et les documents associés ont été centralisés dans un dépôt GitHub, assurant une gouvernance simple et efficace.

Conclusion

Ce projet a permis de mettre en œuvre une démarche complète de traitement et d'analyse de données dans un contexte décisionnel, en utilisant l'environnement Microsoft Fabric. À partir de trois jeux de données brutes (pollution, météo et population), nous avons appliqué une architecture en couches (Bronze, Silver, Gold) pour garantir la qualité, la traçabilité et la performance des données utilisées.

L'intégration et le nettoyage des données ont été réalisés avec rigueur : suppression des doublons, normalisation des valeurs, cohérence des types et jointure des sources. Cette étape de transformation, opérée via Power Query dans les Dataflows Gen2, a permis de passer de données brutes (Bronze) à des tables prêtes à la modélisation (Silver), puis à un modèle sémantique structuré (Gold), intégrable dans Power BI.

Grâce à ce modèle, trois rapports analytiques ont été construits :

1. Une analyse géographique de la pollution par ville en France.
2. Une étude de la corrélation entre la population et les niveaux de pollution.
3. Une exploration de l'impact des conditions météorologiques (température, ensoleillement) sur la pollution.

Chacune de ces visualisations a permis de dégager des tendances intéressantes, notamment l'influence de la densité de population sur les pics de pollution, ainsi qu'une corrélation partielle entre température et concentration en polluants.

Enfin, la mise en place de Slowly Changing Dimensions (SCD) a permis de traiter correctement les évolutions dans les données temporelles (SCD1 pour la population, SCD2 pour la météo), renforçant ainsi la robustesse du modèle.

Ce projet illustre l'importance d'une architecture claire et d'un pipeline de données bien pensé pour produire des analyses fiables, compréhensibles et utiles à la prise de décision.

Conformité aux attendus du Bloc 3 – Implémenter et Optimiser des Solutions de Stockage et de Traitement des Données sur le Cloud

Ce projet s'inscrit pleinement dans les exigences du BLOC 3 du référentiel, portant sur la mise en œuvre de solutions cloud de traitement et d'analyse de données. L'architecture mise en place repose sur une modélisation en couches (Bronze, Silver, Gold) dans Microsoft Fabric, exploitant notamment Lakehouse, Dataflows Gen2 et Power BI.

- Une solution de stockage fiable est implémentée dans le Lakehouse, avec conservation des données brutes, normalisation des types, et schéma relationnel entre les entités.
- La gestion des traitements est assurée via Power Query (Dataflows Gen2), avec un pipeline d'extraction, de nettoyage (normalisation des noms de villes, suppression de doublons), et de transformation avant la modélisation.
- Les aspects de sécurité sont couverts par le chiffrement natif d'Azure/Fabric, l'hébergement privé, et le respect des bonnes pratiques de gouvernance (pas de données personnelles sensibles, journalisation des étapes).
- L'optimisation des performances est assurée par la structuration en tables de faits et dimensions, la définition de mesures calculées, et une alimentation ciblée dans Power BI, ce qui garantit une expérience fluide et scalable.

Ainsi, les compétences du bloc 3 sont pleinement couvertes à travers ce projet, tant au niveau technique qu'organisationnel.

BLOC 4 - Implémenter des méthodes d'intelligence artificielle pour modéliser et prédire de nouveaux comportements et usages.

Objectifs du bloc :

Extraire des données en provenance de systèmes d'information multivariés pour les exploiter et les analyser en utilisant les outils professionnels courants
Préparer les données en les transformant et les nettoyant pour faire l'analyse et le reporting selon les besoins des différents métiers
Élaborer une communication infographique visuelle en construisant des tableaux de bord interactifs afin de communiquer les résultats d'analyses et d'assurer l'extraction de connaissances en temps réel.
Développer un modèle prédictif pour identifier de nouveaux comportements et usages en implémentant des algorithmes d'apprentissage automatique supervisés
Évaluer la performance d'un modèle de Machine Learning en analysant ses résultats et en comparant avec d'autres modèles afin d'implémenter la solution la plus convenable à un cas d'usage

1. Préparation des données

a. Extraction des données :

L'extraction des données constitue la première étape fondamentale de tout processus de modélisation prédictive. Dans notre projet, nous avons opéré une extraction à partir de notre base de données relationnelle PostgreSQL, stockant les données foncières agrégées au sein d'une table intermédiaire silver_resultats. Ce choix s'inscrit dans une logique de traitement de données massives via une architecture analytique en couches, permettant une meilleure gouvernance et traçabilité des transformations.

La connexion à la base a été établie à l'aide de la bibliothèque SQLAlchemy, qui permet d'abstraire la logique SQL dans un cadre Pythonique tout en conservant les performances d'exécution des requêtes. L'utilisation de pandas.read_sql() a ensuite permis de charger la table cible directement dans un DataFrame pour faciliter l'analyse exploratoire.

```
import pandas as pd
from sqlalchemy import create_engine

# Paramètres de connexion
username = "spark"
password = "spark123"
host = "localhost"
port = 5432
database = "projet_final"

# Chaîne de connexion SQLAlchemy
engine = create_engine(f"postgresql://{username}:{password}@{host}:{port}/{database}")

# Lecture de la table en DataFrame pandas
df = pd.read_sql("SELECT * FROM public.silver_resultats", engine)
```

Code de connexion à notre bdd

b. Nettoyage des données :

Contrairement à une phase de nettoyage isolée, notre projet adopte une démarche incrémentale, dans laquelle les opérations de préparation des données sont intégrées au fil des analyses exploratoires et des besoins des modèles. Ce choix reflète une pratique professionnelle réaliste : le nettoyage est rarement linéaire, et dépend des connaissances que l'on acquiert progressivement sur le jeu de données.

Ainsi, nous avons utilisé différentes méthodes et appliqué différents nettoyages :

- Suppression des valeurs nulles (dans la colonne 'valeur_fonciere'). Nous avons estimé, au vu du but de notre analyse, que ces dernières ne pourraient que perturber notre prédiction.
- Séparation de notre jeu de données en 3 datasets distincts :

Ce choix peut sembler étrange, mais il a bien été réfléchi. En effet, en parcourant nos données, nous avons constaté que ces dernières allaient de quelques centaines d'euros à plusieurs centaines de millions d'euros. Nous avons estimé qu'il était donc pas possible de traiter de la même manière toutes ces données.

Tout d'abord, nous avons analysé la répartition des données globales. Nous avons constaté, que sur les plus de 9 millions de transactions que nous avons seulement 60 000 valeurs qui dépassaient les 2 millions d'euros. Pour faciliter notre analyse, nous avons décidé d'écarter ces transactions. Ces dernières ayant un rayon allant de 2M à plus de 500M d'euros, il nous était compliqué de l'analyser en même temps que les autres transactions.

En partant de ce nouveau jeu de données, il nous fallait donc savoir comment délimiter des groupes pour continuer notre travail de répartition. Nous avons donc décidé de partir sur 3 datasets distincts :

- Segment standard : de 10 000 à 470 000€
- Segment haut de gamme : de 470 000 à 1.3 Millions d'€
- Segment ultra haut de gamme : d'1.3M d'€ à 2M d'€

En utilisant cette division, nous limitons le nombre de valeurs aberrantes de notre dataset, et cela nous permet une analyse plus précise pour chaque segment.

Nous avons aussi décidé de retirer les valeurs qui étaient inférieures à 10 000€, car nous avons considéré qu'il était anormal qu'une transaction immobilière soit inférieure à ce cout. De cette manière, nous limitons également les valeurs aberrantes, et les mauvais biais de notre analyse.

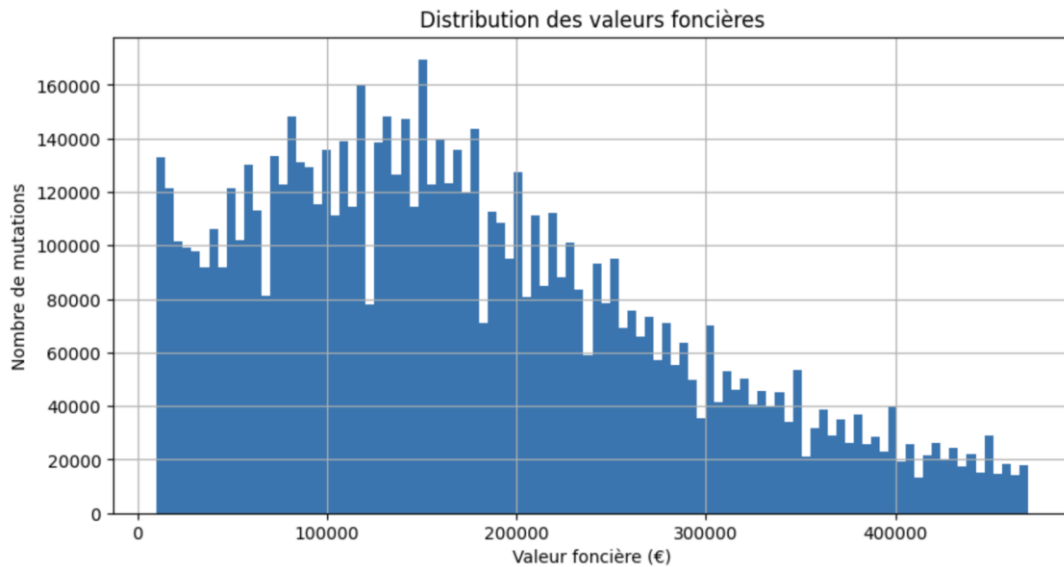
```
# Dataset 1 : transactions courantes
df_1 = df_clean[(df_clean['valeur_fonciere'] >= 10_000) & (df_clean['valeur_fonciere'] <= 470_000)]

# Dataset 2 : transactions semi élevées (ex: très grandes maisons)
df_2 = df_clean[(df_clean['valeur_fonciere'] >= 470_000) & (df_clean['valeur_fonciere'] <= 1_300_000)]

# Dataset 3 : transactions élevées (ex: biens d'exception)
df_3 = df_clean[(df_clean['valeur_fonciere'] >= 1_300_000) & (df_clean['valeur_fonciere'] <= 2_000_000)]
```

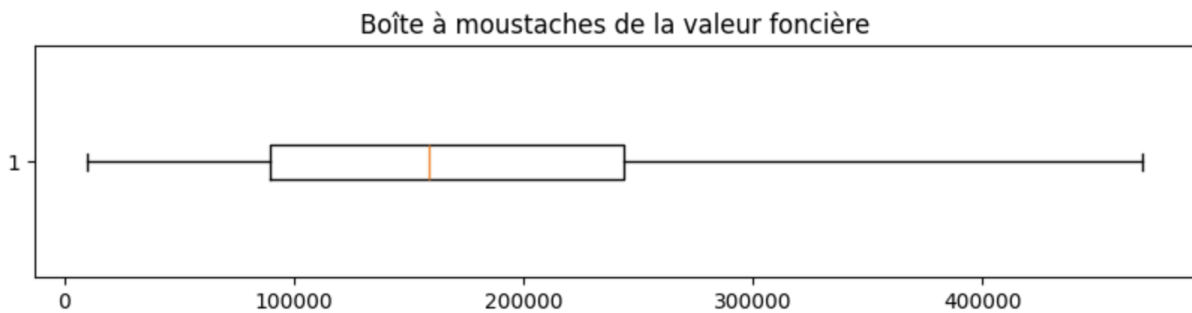
Division de notre dataset en 3 segments

Ainsi, si l'on prend l'exemple du premier segment, les biens communs, nous arrivons à ce graphique de dispersion :



Dispersion des transactions du segment commun

Et ce digramme pour vérifier s'il existe des valeurs aberrantes :

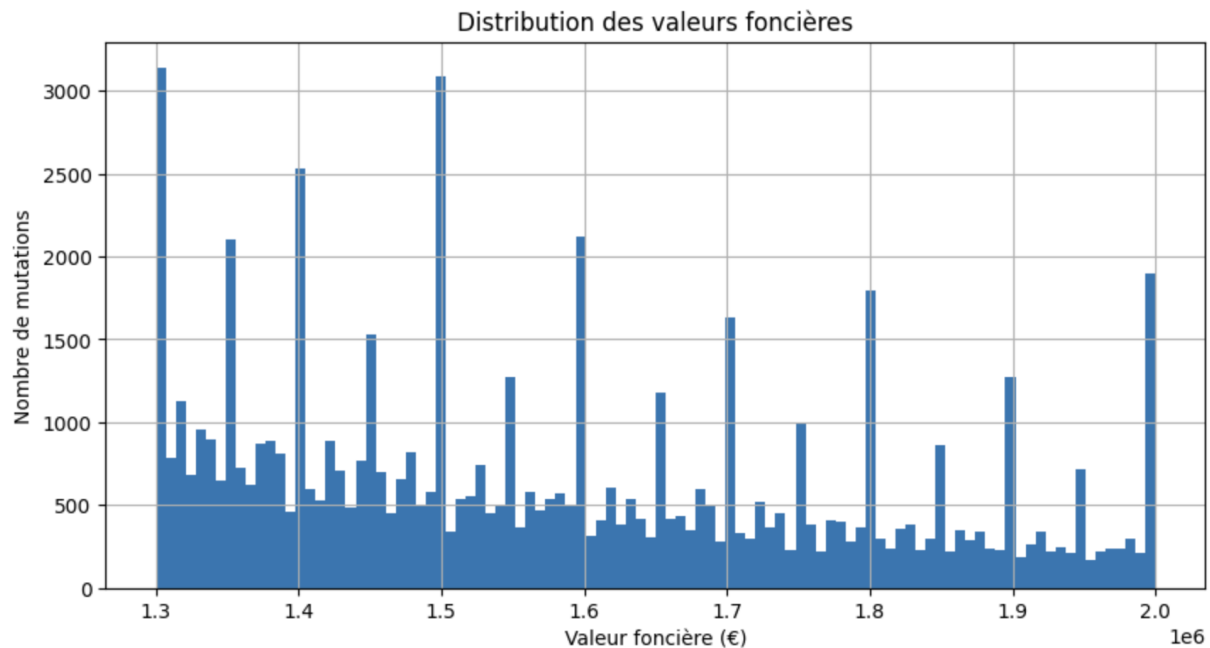


Analyse des valeurs aberrantes du segment commun

Nous pouvons voir que la dispersion de nos données est nickel.

Information humaine intéressante :

Nous avons également constaté une information étrange. En effet, pour le segment ultra, nous avons constaté qu'il y avait des piques de transactions. Nous pensions à une erreur au début, mais après avoir analysé de plus près, nous avons constaté que ces derniers avaient lieu tous les 100 000€. Ainsi, nous avons pu déterminer que lorsque les valeurs foncières sont élevées, les propriétaires ont beaucoup plus tendance à arrondir leurs biens pour les vendre.



Dispersion des transactions du segment ultra

Pour poursuivre notre travail de nettoyage, nous nous sommes attaqués aux surfaces, essentielles dans l'immobilier. Nous avons voulu nous assurer qu'aucune valeur aberrante et nulles n'étaient présentes dans l'ensemble de nos dataframes. Ainsi, nous avons donc retiré les surfaces nulles, et utilisé la méthode IQR pour éliminer les valeurs aberrantes.

Pour chaque dataframe, nous avons calculé chaque quartile, ici sur la distribution des surfaces. Normalement, vu que nous avons déjà travaillé sur les prix, et séparé chaque tranche de prix dans des dataframes différents, les excès au niveau des surfaces devraient être des erreurs dans le jeu de données. C'est pourquoi nous pensons que cette méthode est la meilleure et la plus efficace pour ce cas.

Ainsi, chaque dataframe aura ses valeurs de surfaces filtrées sur ses propres quartiles, ce qui va éliminer les surfaces anormalement petites ou grandes par rapport à la distribution générale de son dataframe.

```
Comptage de lignes nulles ou égales à 0 :
```

```
Nombre de lignes dans df_1 avec surface nulle ou égale à 0 : 3799424  
Nombre de lignes dans df_2 avec surface nulle ou égale à 0 : 318787  
Nombre de lignes dans df_3 avec surface nulle ou égale à 0 : 30114
```

Lignes contenant des surfaces nulles que nous avons dû nettoyer

```
def remove_surface_outliers_iqr(df, column='surface_reelle_bati'):
    # Calcul des quartiles sur la colonne 'surface_reelle_bati' de df
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    # Définition des bornes inférieure et supérieure pour considérer un outlier
    borne_inf = Q1 - 1.5 * IQR
    borne_sup = Q3 + 1.5 * IQR

    # On conserve ici uniquement les lignes dont la surface est dans [borne_inf, borne_sup]
    df_filtré = df[(df[column] >= borne_inf) & (df[column] <= borne_sup)].copy()
    return df_filtré

# 1) Nettoyage de df_1
df_1 = remove_surface_outliers_iqr(df_1, 'surface_reelle_bati')
print(f"df_1 : après = {df_1.shape[0]} lignes")

# 2) Nettoyage de df_2
df_2 = remove_surface_outliers_iqr(df_2, 'surface_reelle_bati')
print(f"df_2 : après = {df_2.shape[0]} lignes")

# 3) Nettoyage de df_3
df_3 = remove_surface_outliers_iqr(df_3, 'surface_reelle_bati')
print(f"df_3 : après = {df_3.shape[0]} lignes")
```

```
df_1 : après = 7696306 lignes
df_2 : après = 694820 lignes
df_3 : après = 60609 lignes
```

Suppression des outliers (valeurs aberrantes) sur les surfaces

```
Nettoyage des valeurs nulles ou égales à 0 pour chaque dataframes :
df_1 avant nettoyage : 7696306 lignes
df_1 après nettoyage : 3896882 lignes (lignes supprimées = 3799424)

df_2 avant nettoyage : 694820 lignes
df_2 après nettoyage : 376033 lignes (lignes supprimées = 318787)

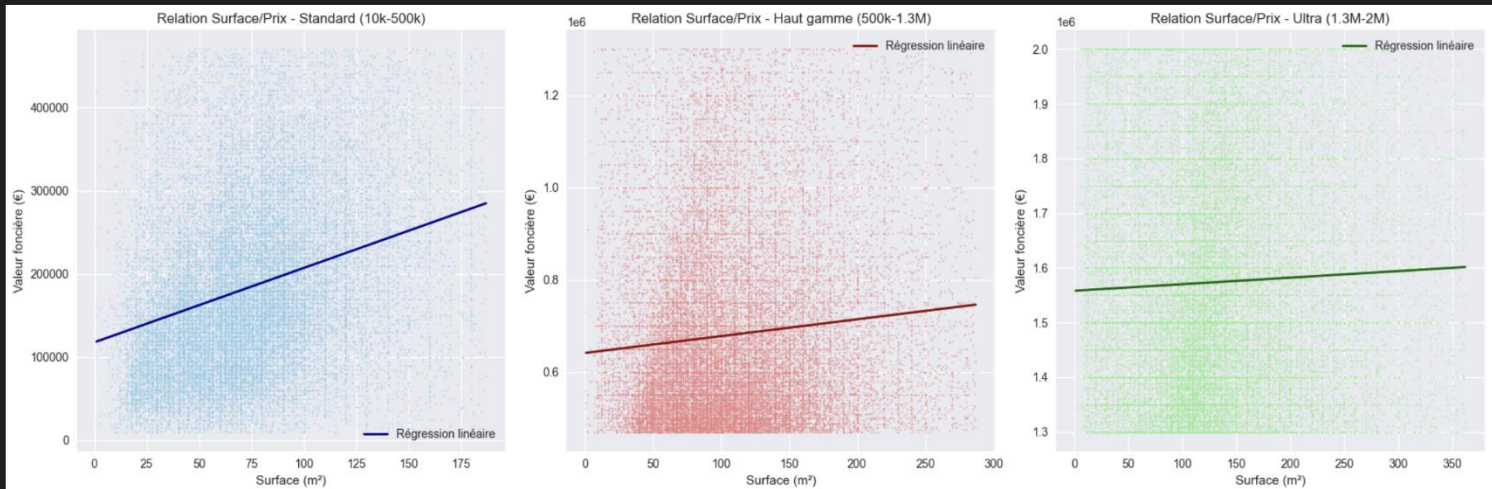
df_3 avant nettoyage : 60609 lignes
df_3 après nettoyage : 30495 lignes (lignes supprimées = 30114)
```

Nombre de lignes de chaque dataframe après notre nettoyage

Une fois que le nettoyage de nos dataframes étaient fini, nous avons voulu avoir une première analyse sur nos données. Nous avons donc voulu les afficher de manière graphique :

3. RELATION SURFACE/PRIX PAR SEGMENT

=====
Prix au m² moyen - Standard (10k-500k) : 2975 €/m²
Prix au m² moyen - Haut gamme (500k-1.3M) : 8559 €/m²
Prix au m² moyen - Ultra (1.3M-2M) : 19260 €/m²



Corrélation Surface–Prix au m² selon les gammes de valeur foncière

Nous avons donc pu constater sur ces différentes analyses que nos données sont bien propres et prêtes à l'utilisation.

2. Création du modèle prédictif

L'objectif principal de ce projet est la mise en œuvre d'un modèle d'apprentissage automatique capable de prédire la valeur foncière d'un bien immobilier à partir de ses caractéristiques (surface, localisation, nature de mutation, type local, etc.)

Nous allons pour ce travail de prédiction, utiliser les données nettoyées et segmentées que nous avons rendues disponibles.

a. Utilisations des algorithmes de prédiction

Pour réaliser notre prédiction, nous avons voulu tester différents algorithmes pour voir lequel était le plus adapté à notre jeu de données et à notre volume de données.

Ainsi, nous avons décidé de regarder plus en détails les modèles suivants :

- Régression linéaire (baseline simple, interprétable) : elle va nous servir de base pour pouvoir étudier les autres modèles. Elle est rapide à entraîner et ne nécessite pas de gros paramètres.
- Ridge regression : la Ridge Regression est une variante de la régression linéaire qui ajoute une pénalisation L2 sur les coefficients. Concrètement, au lieu de minimiser simplement la somme des carrés des erreurs.

- **Random Forest** : Le Random Forest est un algorithme d'ensemblage basé sur la technique du bagging (Bootstrap Aggregating). On construit plusieurs arbres de décision chacun entraîné sur un sous-échantillon aléatoire (avec remise) de l'ensemble d'entraînement. À chaque nœud de chaque arbre, la division (split) est choisie parmi un sous-ensemble aléatoire de features. La prédiction finale est la moyenne (pour la régression) des sorties de tous les arbres.
- **Gradient Boosting Regressor (GBR)** : au lieu de construire tous les arbres en parallèle, on les construit séquentiellement. On entraîne un premier petit arbre (faible complexité), on calcule les résidus (différences entre prédictions et vérité). On entraîne un deuxième arbre pour corriger les erreurs du premier (sur les résidus). On enchaîne ainsi, chaque nouvel arbre cherchant à prédire les résidus des arbres précédents.
- **XGBoost** (environnement optionnel selon les bibliothèques installées) : c'est une implémentation optimisée du Gradient Boosting.
- **LightGBM Regressor** : c'est une autre implémentation optimisée de l'algorithme de Gradient Boosting, développée par Microsoft.

Ces algorithmes sont des standards du machine learning, connus pour leur capacité à gérer des données hétérogènes, à modéliser des relations non linéaires et à limiter l'overfitting.

b. Préparation des featurings et modèles :

L'objectif de cette étape est de préparer les données de chaque segment immobilier pour qu'elles soient exploitables par un modèle de deep learning (ou de machine learning). Cela passe par la création de nouvelles variables ("features"), l'encodage et la structuration des datasets.

Comment nous allons nous y prendre :

- **Enrichissement temporel** : On extrait de la date de mutation des informations comme l'année, le mois, le trimestre, le jour de la semaine, ainsi que des variables cycliques (sinus/cosinus pour la saisonnalité). Ces variables permettront au modèle de mieux capter les effets temporels et saisonniers sur les prix.
- **Création de features de surface** : On ajoute des indicateurs comme le prix au m², la surface par pièce, des catégories de taille de bien (très petit, moyen, grand...), et des transformations (log, carré de la surface). Cela permet de mieux modéliser les relations entre surface, agencement du bien et valeur foncière.
- **Ajout de variables géographiques** : On agrège les statistiques par département (prix moyen, médian, nombre de transactions, etc.) et on les rattache à chaque ligne du dataset. Ces features captent le contexte local (niveau de prix du département, densité des transactions, etc.).
- **Création d'interactions** : On crée des variables "mixtes" combinant plusieurs features (année x surface, type de bien x catégorie de surface...) pour donner au modèle plus de puissance prédictive.

- Encodage des variables catégorielles : Toutes les variables textuelles (type_local, département, etc.) sont transformées en valeurs numériques (one-hot encoding ou label encoding), pour être utilisables par un réseau de neurones.
- Préparation finale : On nettoie les NaN, on retire les colonnes inutiles, on sépare les features (X) de la variable cible (y = prix de vente), et on prépare le dataset final pour l'entraînement.

En utilisant cette méthode sur chacun de nos datasets, nous avons obtenu les résultats suivants :

```
=====
PIPELINE FEATURE ENGINEERING - SEGMENT STANDARD
=====
Dataset initial : 3896882 lignes, 13 colonnes
1. CRÉATION DES FEATURES TEMPORELLES
=====
✓ Features temporelles créées : ['annee', 'mois', 'trimestre', 'mois_sin', 'mois_cos', 'jours_depuis_debut']

2. CRÉATION DES FEATURES DE SURFACE
=====
✓ Features de surface créées : ['prix_m2', 'categorie_surface', 'log_surface', 'surface_par_piece']

3. CRÉATION DES FEATURES GÉOGRAPHIQUES
=====
✓ Features géographiques créées pour 94 départements

4. CRÉATION DES FEATURES D'INTERACTION
=====
✓ Features d'interaction créées

5. ENCODAGE DES VARIABLES CATÉGORIELLES
=====
✓ Colonne 'id_unique' supprimée (identifiant unique non utile)
✓ One-hot encoding appliqué à 'fichier_source' (4 catégories)
✓ One-hot encoding appliqué à 'nature_mutation' (6 catégories)
✓ One-hot encoding appliqué à 'type_local' (2 catégories)
✓ One-hot encoding appliqué à 'part_date' (1 catégories)
✓ Label encoding appliqué à 'departement' (94 catégories)
✓ One-hot encoding appliqué à 'categorie_surface' (4 catégories)
✓ One-hot encoding appliqué à 'type_local_x_surface_cat' (8 catégories)
✓ Total features encodées : 33

6. PRÉPARATION FINALE POUR ML
=====
✓ Colonnes supprimées : ['date_mutation']
✓ Dataset final : 3701567 lignes, 61 features
✓ Variables cibles : 3701567 valeurs
```

Exemple de features pour le dataset 1 (transactions communes)

Nous allons maintenant regarder de plus près les résultats de notre analyse.

Homogénéité des traitements :

- On voit que les transformations sont appliquées de manière cohérente à chaque segment, ce qui facilitera la comparaison ou l'entraînement de modèles séparés (voire un modèle global multi-segment).

Richesse des features :

- Le nombre de variables finales (61-63) est équilibré.
- Les features temporelles, de surface et géographiques sont bien présentes.
- L'encodage mixte (one-hot et label) sur les catégories garantit une exploitation optimale, y compris pour des features à forte cardinalité comme les départements.

Gestion des valeurs manquantes :

- Le fait de détecter et de remplacer les NaN au dernier moment évite de propager des soucis dans le pipeline, ce qui est important pour la suite.

Exemples de features :

- Les variables en sortie (année, mois, jour_semaine, surface, interactions, etc.) sont exactement ce qu'attend un modèle de machine learning moderne sur ce type de données.

Statistiques de base :

Le prix moyen par segment est pertinent et réaliste :

- Segment standard $\approx 187\,000$ €
- Haut de gamme $\approx 678\,000$ €
- Ultra $\approx 1\,570\,000$ €

Cela nous montre que notre division du dataset principal était une bonne idée.

Maintenant que notre pipeline est prêt, nous pouvons commencer l'analyse. Pour cela, nous devons identifier quelles sont les features les plus pertinentes et vérifier la cohérence de nos données avant d'entraîner quoi que ce soit. Nous avons déjà pu en analyser certaines précédemment, mais nous voulons pousser un peu plus loin cette analyse.

Nous avons effectué pour chaque jeu de données (commun, haut de gamme et ultra) une préanalyse systématique, qui nous a assuré que nous travaillions bien sur des données nettoyées et prêtes à l'usage. Cette étape nous a permis de garantir la validité de nos résultats et d'éviter tout problème lié à des valeurs manquantes ou aberrantes. C'est pourquoi, à l'intérieur de la fonction `prepare_dataset_for_ml`, nous inspectons toujours :

- Les valeurs manquantes (NaN) et/ou infinies dans les colonnes features et dans la target.
- La distribution de la cible (valeur_fonciere) : on contrôle le minimum, le maximum, la médiane, l'écart-type et le skewness pour savoir si une transformation (par exemple logarithmique) est nécessaire.

Ensuite, nous appliquons la méthode de séparation des données en ensembles d'entraînement, de validation et de test selon un découpage 80/10/10 :

- 80 % des échantillons sont dédiés à l'apprentissage,
- 10 % à la validation (pour ajuster les hyperparamètres et surveiller le sur-apprentissage),
- 10 % au test final (pour évaluer la performance du modèle sur du « neuf »).

Cette division s'applique donc maintenant également sur les nouvelles features créées.

Sur chaque sous-ensemble d'entraînement, nous pouvons donc :

- Appliquer une normalisation des variables numériques à l'aide d'un RobustScaler (qui prend en compte la médiane et les quartiles plutôt que la moyenne et l'écart-type), ce qui nous protège des valeurs aberrantes et permet au réseau de neurones ou à tout autre algorithme d'apprendre plus efficacement.
- Calculer un score d'importance pour chaque variable numérique à l'aide de l'algorithme d'« information mutuelle » (`mutual_info_regression`), afin de dresser un premier classement des features les plus informatives vis-à-vis de la target. Cela nous offre trois avantages :
 - Vérifier que les features attendues (prix au m², indicateurs géographiques, agrégats départementaux, etc.) sont bien discriminantes.
 - Détecter d'éventuelles corrélations trop fortes ou des variables redondantes.
 - Prévoir, en amont, la possibilité de réduire la dimensionnalité ou de focaliser notre feature engineering sur les colonnes à forte importance.

Enfin, selon l'asymétrie de la distribution de la target (*skewness*), le code applique automatiquement une transformée logarithmique si le *skewness* dépasse un certain seuil (1,5). Cette transformation permet de « compressifier » les valeurs extrêmes lorsque les prix sont très hétérogènes (notamment dans `Dataset_2`, où l'on peut passer de quelques centaines de milliers d'euros à plus d'un million). La transformée logarithmique (via `np.log1p`) produit ensuite trois vecteurs cibles (`y_train_transformed`, `y_val_transformed`, `y_test_transformed`) utilisables directement pour l'entraînement.

Une fois toutes ces étapes réalisées pour chaque jeu de données, nous obtenons un résumé global des trois datasets :

- Le nombre total d'échantillons disponibles en train, validation et test (plus de 3 millions d'exemples d'entraînement au total),
- La liste des variables communes à tous les jeux, ainsi que celles qui sont spécifiques à chacun (pour pouvoir, si besoin, fusionner ou comparer les ensembles).

Nous proposons également une fonction `combine_datasets` qui permet d'agréger les trois jeux préparés en un seul grand dataset uniforme. On peut choisir de ne conserver que les features communes aux trois segments (pour éviter tout vide) ou de prendre l'union des colonnes (en imputant les colonnes manquantes par défaut à zéro), ce qui offre une grande flexibilité si l'on souhaite entraîner un modèle global couvrant l'ensemble des tranches de prix.

Il nous reste plus qu'ensuite à appliquer nos différents datasets sur les modèles vu précédemment. Nous allons donc procéder de la manière suivante :

```
TRAITEMENT DU DATASET : Dataset_2
=====
1. SÉPARATION FEATURES/TARGET
-----
✓ Features : 63 colonnes
✓ Target : 348660 valeurs
✓ Prix moyen : 677,998 €

2. VÉRIFICATION DES DONNÉES
-----
✓ Valeurs manquantes dans X : 0
✓ Valeurs manquantes dans y : 0

3. ANALYSE DE LA TARGET
-----
✓ Min : 470,000 €
✓ Max : 1,300,000 €
✓ Médiane : 615,000 €
✓ Écart-type : 195,326 €
✓ Skewness : 1.25

4. DIVISION DU DATASET
-----
✓ Train : 278,928 échantillons (80.0%)
✓ Validation : 34,866 échantillons (10.0%)
✓ Test : 34,866 échantillons (10.0%)

5. NORMALISATION DES FEATURES
-----
✓ 27 colonnes numériques normalisées
✓ Scaler utilisé : RobustScaler
```

Exemple sur le dataset 2

6. ANALYSE DE L'IMPORTANCE DES FEATURES

✓ Top 10 features importantes :

1. ratio_prix_dept	5.0261
2. prix_m2	2.2524
3. dept_prix_x_surface	0.0596
4. departement_encoded	0.0533
5. dept_surface_reelle_bati_mean	0.0519
6. dept_valeur_fonciere_mean	0.0519
7. dept_valeur_fonciere_std	0.0517
8. dept_valeur_fonciere_count	0.0511
9. dept_densite_transactions	0.0506
10. dept_valeur_fonciere_median	0.0472

7. TRANSFORMATION DE LA TARGET

✓ Pas de transformation de la target nécessaire

8. RÉSUMÉ FINAL - Dataset_2

- ✓ Features finales : 63
- ✓ Échantillons train : 278,928
- ✓ Échantillons validation : 34,866
- ✓ Échantillons test : 34,866
- ✓ Target transformée : Non

Suite de l'exemple sur le dataset 2

i. Chargement et vérification des données :

- On commence par recharger les données préparées (train, validation, test).
- On vérifie que toutes les features sont bien numériques, et on corrige au besoin les colonnes de type date ou objet pour les convertir en variables exploitables.

ii. Harmonisation des jeux de données :

- On s'assure que les colonnes des jeux d'entraînement, validation et test sont strictement identiques (en type et en ordre).
- Toutes les valeurs sont converties en numérique, et les éventuelles valeurs manquantes sont remplacées pour éviter tout souci lors de l'entraînement.

iii. Définition d'un protocole d'évaluation :

- Une fonction de calcul des métriques (RMSE, MAE, R^2 , MAPE) est définie pour comparer objectivement les modèles.
- Les résultats sont affichés de manière lisible et synthétique.

iv. Entraînement d'une sélection de modèles :

- Régression linéaire : sert de baseline pour évaluer les gains des autres méthodes.
- Ridge Regression : variante pénalisée, pour gérer la complexité et la colinéarité.
- Random Forest : modèle d'arbres ensemblistes, performant sur les données tabulaires et robustes aux interactions complexes.
- Gradient Boosting : méthode séquentielle corrigeant les erreurs de prédiction à chaque étape.
- XGBoost et LightGBM : versions optimisées du Gradient Boosting, très performantes sur de gros volumes de données.

v. Comparaison et sélection du meilleur modèle :

- Les performances des différents modèles sont comparées sur le set de validation.
- Le modèle avec la plus basse erreur (RMSE) est sélectionné comme meilleur candidat.

vi. Évaluation finale sur le set de test :

- Le meilleur modèle est évalué sur les données de test pour obtenir une estimation réelle de la performance.
- Analyse des résidus pour vérifier qu'il n'y a pas de biais ou d'erreurs majeures.

vii. Analyse de l'importance des variables :

- On identifie les features qui contribuent le plus à la prédiction, ce qui aide à interpréter les résultats et à communiquer sur les déterminants du prix.

viii. Structuration et sauvegarde des résultats :

- Tous les résultats, modèles et métriques sont stockés dans un dictionnaire Python, prêt à être réutilisé pour la suite (analyse, visualisation, reporting...).

Nous allons donc ensuite appliquer cette méthode sur le premier dataset, pour analyser les résultats que nous allons avoir.

1. RÉGRESSION LINÉAIRE (BASELINE)

 Linear Regression
RMSE: 17,937 €
MAE: 11,758 €
R ² : 0.9698
MAPE: 11.46%
 Temps d'entraînement: 0:00:21.874971
2. RIDGE REGRESSION

 Ridge Regression
RMSE: 17,931 €
MAE: 11,758 €
R ² : 0.9698
MAPE: 11.46%
 Temps d'entraînement: 0:00:03.351376
3. RANDOM FOREST

 Random Forest
RMSE: 258 €
MAE: 128 €
R ² : 1.0000
MAPE: 0.08%
 Temps d'entraînement: 0:35:26.966291
4. GRADIENT BOOSTING

 Gradient Boosting
RMSE: 1,350 €
MAE: 955 €
R ² : 0.9998
MAPE: 0.62%
 Temps d'entraînement: 0:59:30.147056
5. XGBOOST

 XGBoost
RMSE: 1,931 €
MAE: 1,091 €
R ² : 0.9997
MAPE: 0.75%
 Temps d'entraînement: 0:00:20.728984
6. LIGHTGBM

 LightGBM
RMSE: 2,509 €
MAE: 1,555 €
R ² : 0.9994
MAPE: 1.05%
 Temps d'entraînement: 0:00:17.984057

Entrainement de nos modèles sur le premier dataset

Préparation et cohérence des données :

- Propreté parfaite : Toutes les features sont converties en numériques, sans valeurs manquantes ou incohérentes.
- Échantillons nombreux et équilibrés : Plus de 2,9 millions de lignes pour l'entraînement, 370 000 pour validation et test : un jeu de données exceptionnellement riche pour la robustesse statistique.

Comparaison des performances modèles

- Régression linéaire / Ridge :
 - Baseline correcte, mais RMSE d'environ 18 000 € et MAPE de 11,5 %.
 - R^2 autour de 0,97 (déjà très élevé pour ce type de problème).
- Modèles avancés (random forest, boosting, etc.) :
 - Random Forest : surclasse tous les autres avec un RMSE de 258 € sur validation, $R^2 \approx 1,00$, MAPE < 0,1 %. C'est une amélioration de 98,6 % par rapport à la baseline linéaire : quasi-absence d'erreurs globales.
 - Gradient Boosting, XGBoost, LightGBM : résultats proches, RMSE variant de 1 350 à 2 500 €.

Ces modèles confirment la faisabilité d'une modélisation très fine sur ce dataset.

```
1. RÉGRESSION LINÉAIRE (BASELINE) - Dataset 2
-----
Linear Regression 2
RMSE: 10,100 €
MAE: 6,887 €
R²: 0.9973
MAPE: 1.01%
Temps d'entraînement: 0:00:02.170172

2. RANDOM FOREST - Dataset 2
-----
Random Forest 2
RMSE: 622 €
MAE: 199 €
R²: 1.0000
MAPE: 0.03%
Temps d'entraînement: 0:01:23.618909

3. XGBOOST - Dataset 2
-----
XGBoost 2
RMSE: 2,503 €
MAE: 1,371 €
R²: 0.9998
MAPE: 0.19%
Temps d'entraînement: 0:00:01.744277

4. LIGHTGBM - Dataset 2
-----
LightGBM 2
RMSE: 2,834 €
MAE: 1,756 €
R²: 0.9998
MAPE: 0.25%
Temps d'entraînement: 0:00:01.585625
```

Entraînement sur le dataset 2 en gardant seulement les modèles souhaités

Évaluation finale sur le test set

- Random Forest reste le meilleur :
 - RMSE de 271 €, MAE de 128 €, R^2 de 1,00, MAPE de 0,08 % sur le jeu de test.
 - Analyse des résidus : résidu moyen quasi nul, faible écart-type (271 €), médiane à 0 €.
 - Importance des variables : les agrégats départementaux (prix moyen, ratio, médiane, etc.) dominent largement, ce qui confirme leur pouvoir prédictif sur la valeur foncière.

Ces résultats sont très bons, voir trop bons. Nous pouvons donc nous inquiéter d'overfitting ou autre problème au niveau de l'analyse que gère notre modèle :

Surapprentissage probable (overfitting) :

- Les scores R^2 à 1,00 et des erreurs aussi faibles, même sur un set de test très large, suggèrent soit une modélisation ultra-efficace, soit un possible manque de bruit ou de complexité dans les données.
- Attention : si les données de test ne sont pas totalement indépendantes (fuites, corrélation spatiale/temporelle), cela peut surestimer les performances réelles en production.

Pour finir sur cette partie-là, nous avons sauvegardé les résultats dans une structure claire (final_results), facilitant la poursuite des analyses ou l'export vers d'autres outils.

```
=====
COMPARAISON DES MODÈLES
=====

🏆 CLASSEMENT PAR RMSE (plus bas = meilleur):
-----
1. Random Forest      RMSE:      258 € | R²: 1.0000
2. Gradient Boosting  RMSE:    1,350 € | R²: 0.9998
3. XGBoost            RMSE:    1,931 € | R²: 0.9997
4. LightGBM           RMSE:    2,509 € | R²: 0.9994
5. Ridge Regression   RMSE:   17,931 € | R²: 0.9698
6. Linear Regression  RMSE:   17,937 € | R²: 0.9698

🏆 MEILLEUR MODÈLE: Random Forest
  RMSE: 258 €
  Amélioration vs baseline: 98.6%

=====
ÉVALUATION FINALE SUR LE SET DE TEST
=====

🎯 PERFORMANCE FINALE SUR LE SET DE TEST:
🏆 Random Forest (Test)
  RMSE: 271 €
  MAE:  128 €
  R²:   1.0000
  MAPE: 0.08%

📋 ANALYSE DES RÉSIDUS
-----
✓ Résidus moyens: 1 €
✓ Écart-type des résidus: 271 €
✓ Médiane des résidus: 0 €

🔍 IMPORTANCE DES FEATURES
-----
✓ Top 10 features les plus importantes:
  1. ratio_prix_dept      0.8227
  2. dept_valeur_fonciere_mean 0.1448
  3. dept_valeur_fonciere_median 0.0303
  4. dept_surface_reelle_bati_median 0.0009
  5. dept_surface_reelle_bati_mean 0.0009
  6. dept_valeur_fonciere_std 0.0003
  7. code_postal          0.0001
  8. dept_valeur_fonciere_count 0.0000
  9. departement_encoded  0.0000
  10. dept_densite_transactions 0.0000
```

Résultat de l'analyse des features et comparaison des modèles

Nous avons ensuite appliqué exactement la même méthode pour les deux autres datasets, qui contenaient des valeurs foncières beaucoup plus grandes. Nous avons trouvé des résultats identiques (le RMSE était plus grand du fait des valeurs plus importantes). Nous avons donc voulu analyser nos résultats pour vérifier qu'il n'y avait pas de leakage ou d'overfitting.

Lorsque l'on regarde de plus près, on se rend compte que les features les plus utilisées par notre modèle sont "ratio_prix_dep" à plus de 80% et "dept_valeur_fonciere_mean", à 15%. Cela indique que le modèle se sert essentiellement de la feature ratio_prix_dep et trouve de bons résultats grâce à ces variables.

En pratique, cela veut dire que, tant qu'il connaît la moyenne (ou le ratio) des prix du département, il est capable de prédire le prix individuel à quelques centaines d'euros près. Le reste des features (surface, date, etc.) pèse beaucoup moins dans la décision finale.

Cette homogénéité des résultats à l'intérieur d'un même département explique l'extrême efficacité du Random Forest, mais elle soulève aussi la question du "leakage" : si ces statistiques départementales ont été calculées sur l'ensemble des données (train + validation + test), alors le modèle a indirectement accès à l'information qu'il cherche à prédire. Dans ce cas, la performance n'est pas représentative de ce que l'on obtiendrait en production.

Pour corriger cela, il faudrait recalculer les moyennes départementales en n'utilisant que le sous-ensemble d'entraînement, puis appliquer ces "moyennes du train" au moment des prédictions sur validation et test. Nous pouvons donc tester notre modèle en retirant cette colonne sur-utilisée pour voir si ce dernier reste toujours aussi performant.

```
Colonnes suspectes (potentiel leakage) : ['dept_valeur_fonciere_mean', 'dept_valeur_fonciere_median', 'dept_va.  
Shape original X2_train : (278928, 68)  
Shape X2_train sans suspects : (278928, 59)  
  
--- Résultats sans les colonnes suspectes ---  
RMSE (val) : 3,166 €  
MAE (val) : 877 €  
R² (val) : 0.9997
```

Analyse des résultats, vérification du leakage (sur le dataset 2, le segment haut de gamme):

Constat initial :

- Les résultats initiaux étaient trop parfaits ($R^2 = 1$, MAPE proche de 0 %), principalement portés par des variables "agrégées" comme ratio_prix_dept ou les moyennes départementales.
- Cela trahit une forte homogénéité des prix à l'intérieur d'un département et/ou un problème de fuite d'information (le modèle "voit" la réponse à travers ces variables).

Etude de nos nouveaux résultats :

- RMSE : 3 166 €
- MAE : 877 €

- R^2 : 0,9997
- L'erreur remonte (mais reste très faible), ce qui montre que le modèle, sans notre colonne surutilisée fait encore de très bonnes prédictions. Sachant que nous avons fait cette analyse sur le dataset2, soit des données allant d'une plage de 470 000 à 1.3M d'euros, cette marge reste très très fine.

Conclusion :

Le modèle reste extrêmement efficace, même sans les features créées. La suppression des variables "leakées" ramène la performance à un niveau crédible.

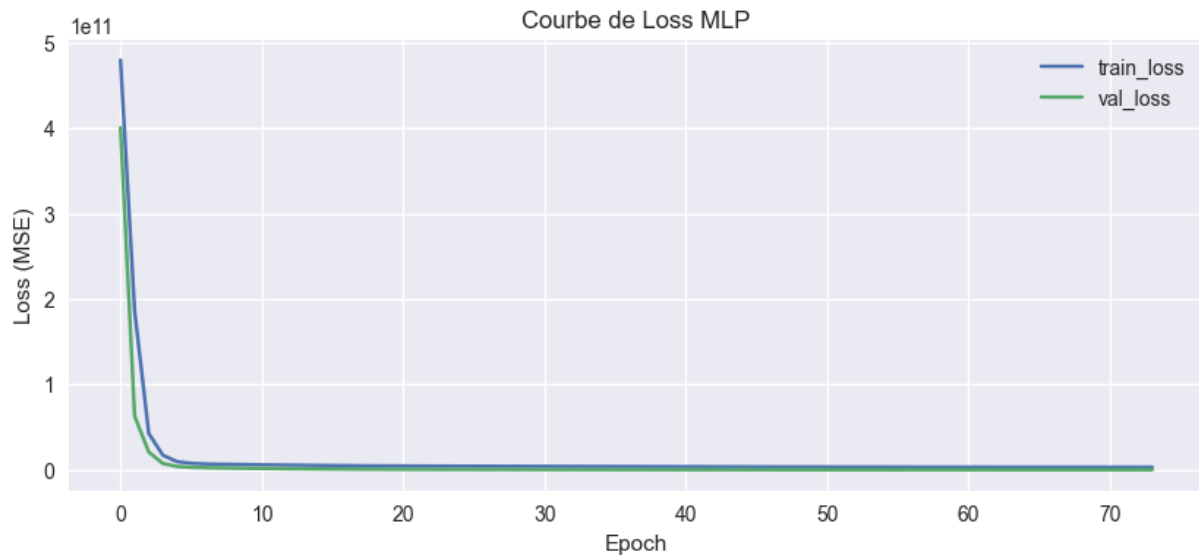
Nous pouvons donc passer à la suite de notre projet, l'utilisation de notre modèle choisit pour la prédiction sur des valeurs de 2025.

Nous allons maintenant tester d'autres méthodes de deep learning pour réaliser notre prédiction : le MLP.

Nous avons mis en place un script qui crée un pipeline de régression avec un réseau de neurones dense (MLP) sur des données tabulaires :

- **Prétraitement** : Normalisation des variables numériques avec StandardScaler pour garantir une échelle comparable entre les features.
- **Modélisation** : Construction d'un MLP avec plusieurs couches cachées et dropout pour limiter l'overfitting. Optimiseur Adam et fonction de perte MSE adaptés à la régression.
- **Entraînement** : Utilisation du EarlyStopping pour éviter le surapprentissage, suivi des performances sur un jeu de validation.
- **Évaluation** : Calcul des métriques RMSE, MAE et R^2 sur le jeu de test pour évaluer la qualité des prédictions.
- **Visualisation** : Affichage de la courbe de loss (entraînement/validation) pour diagnostiquer l'apprentissage.

Après avoir développé des modèles nous servant de base, nous pourrions comparer les résultats obtenus.



Courbe de pertes MSE en fonction des epochs réalisés

Analyse des résultats du MLP :

Convergence rapide : La courbe de loss montre une forte diminution en quelques epochs, puis une stabilisation des erreurs train/validation, preuve d'un bon apprentissage sans surapprentissage.

- Performances :
 - RMSE (test) : ~7 500 €
 - MAE (test) : ~5 500 €
 - R^2 (test) : 0.9985
- Ces scores indiquent que le MLP est performant, et est totalement en accord aux modèles classiques (Random Forest, LightGBM...).
- Comparaison : Le MLP offre une alternative efficace et stable pour la régression tabulaire, mais les modèles d'ensemble conservent un léger avantage sur ce dataset, comme souvent en tabulaire.

Le MLP "from scratch" est un modèle robuste pour la tâche : il apprend bien la structure des données et généralise correctement, avec un écart d'erreur raisonnable par rapport à la baseline optimale.

Pour finir sur l'analyse des différents modèles, nous avons voulu utiliser TabNet (Pytorch). L'utilisation de Pytorch TabNet, un modèle de deep learning spécialisé pour les données tabulaires, à la régression des valeurs foncières va nous permettre de

i. Prétraitement :

- Encodage automatique de toutes les variables catégorielles avec LabelEncoder.

- Transformation automatique des colonnes de type date/heure en variables numériques exploitables par le modèle.
- Conversion des jeux de données en format numpy, typés en float32.

ii. Modélisation :

- Entraînement d'un modèle TabNet (PyTorch) avec configuration adaptée pour la régression : plusieurs étapes de traitement, attention mechanism, early stopping.
- Validation du modèle sur un jeu de validation pour surveiller les performances et éviter le surapprentissage.

iii. Évaluation :

- Prédiction sur le jeu de test avec calcul des métriques RMSE, MAE et R^2 .
- Visualisation de l'importance des variables via les scores fournis par TabNet.

Ce modèle permet d'exploiter efficacement les données tabulaires, en profitant du transfert learning et de l'attention, et sert de point de comparaison face au MLP et aux modèles classiques.

```
===== Évaluation TabNet sur le set de test =====  
RMSE : 5,742 €  
MAE : 4,191 €  
R2 : 0.9991
```

Résultats de l'entraînement du modèle avec TabNet

Nous devons, pour notre prédiction, décider du modèle que nous allons utiliser. Pour rappel, 3 modèles sont sortis du lot :

Le Random Forest :

- Le Random Forest est un modèle d'ensemble éprouvé pour la régression sur données tabulaires. Il offre une grande robustesse aux données bruitées et gère très bien les relations non-linéaires entre variables. Utilisé comme baseline, il permet de fixer un seuil de performance réaliste, à comparer avec les approches de deep learning. Son principal avantage est la simplicité de mise en œuvre et l'interprétabilité relative (importance des variables).

Le MLP (Multi-Layer Perceptron) :

- Le MLP représente la forme la plus simple des réseaux de neurones pour la régression. Il a l'avantage de pouvoir approximer des fonctions complexes, notamment lorsqu'il existe de multiples relations non-linéaires dans les données. Cependant, il ne tient pas compte explicitement de la structure des données tabulaires (absence de prise en compte de la sparsité ou de l'interaction naturelle des colonnes comme le fait un arbre de décision), ce qui en limite parfois la performance face à des modèles spécifiquement conçus pour le tabulaire.

Et pour finir, TabNet :

- TabNet est un modèle de deep learning spécialement conçu pour les données tabulaires. Il combine la capacité d'apprentissage profond avec un mécanisme d'attention qui sélectionne dynamiquement les variables les plus pertinentes à chaque étape de l'entraînement. Ce mécanisme, inspiré des arbres de décision, permet à TabNet d'égaliser voire de surpasser les performances des meilleurs modèles d'arbres sur des données structurées, tout en offrant une meilleure capacité de généralisation et d'interprétabilité (importance des features). Il s'adapte bien à de grands volumes de données tabulaires hétérogènes.

Avec ces modèles, nous avons obtenu les résultats suivants :

- RandomForest : RMSE de 618 €, un MAE de 194 € et un R^2 de 1.0000.
- MLP : RMSE de 7 469 €, un MAE de 5 484 € et un R^2 de 0.9985.
- TabNet : RMSE de 5 742 €, un MAE de 4 191 € et un R^2 de 0.9991.

Nous allons donc, pour notre prédiction, partir sur le Random Forest. Nous allons également nous focaliser sur le dataframe des données haut de gamme, car ces dernières sont en grand nombre, et il y a de grands écarts dans les valeurs foncières. Nous estimons donc que c'est le dataset qui va le plus nous permettre de mettre à l'épreuve notre prédiction.

c. Création de notre modèle de prédiction :

Nous allons ainsi suivre plusieurs étapes pour arriver à notre objectif :

i. Génération du jeu de données 2025 :

- Création d'un nouveau DataFrame à partir des données de 2024, en modifiant la colonne de date pour la projeter sur 2025.
- Ajout d'agrégats départementaux (calculés uniquement sur le train) pour enrichir les features.

ii. Prétraitement cohérent :

- Application du même pipeline de preprocessing que pour les jeux d'entraînement/test afin de garantir la cohérence des variables et éviter les fuites de données ("leakage").
- Suppression de toutes les colonnes qui pourraient contenir des informations indirectes sur la variable à prédire.

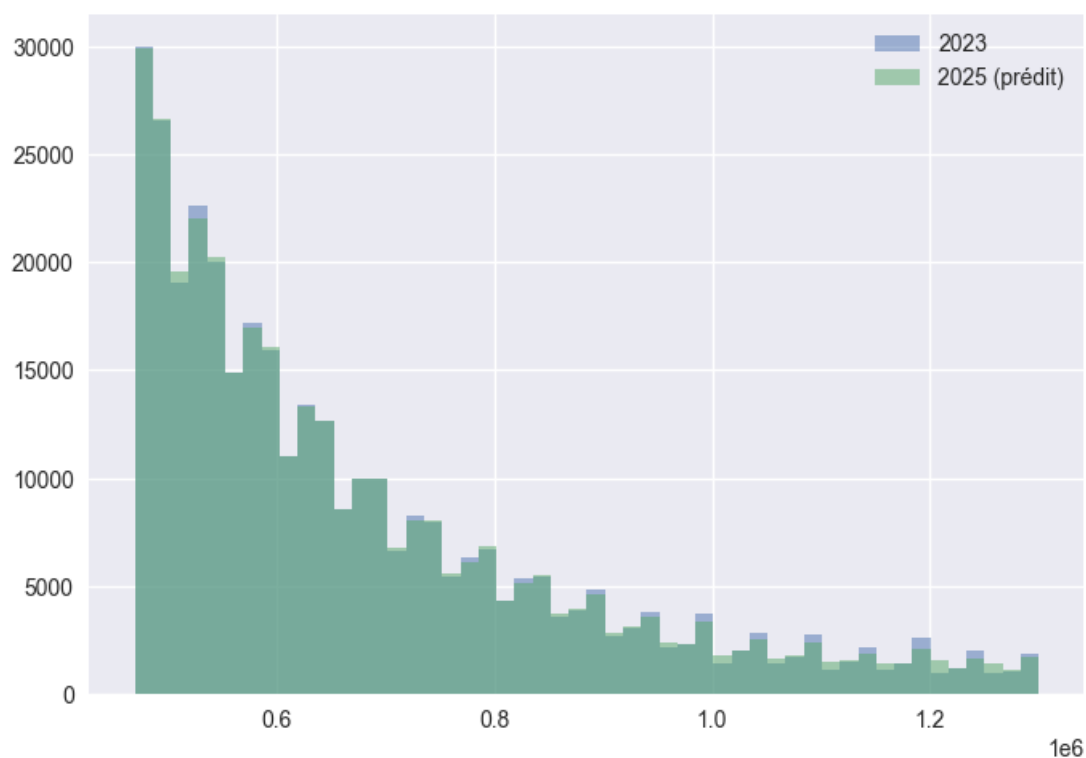
iii. Alignement des features :

- Sélection et réindexation des variables explicatives pour correspondre exactement à celles utilisées à l'entraînement.

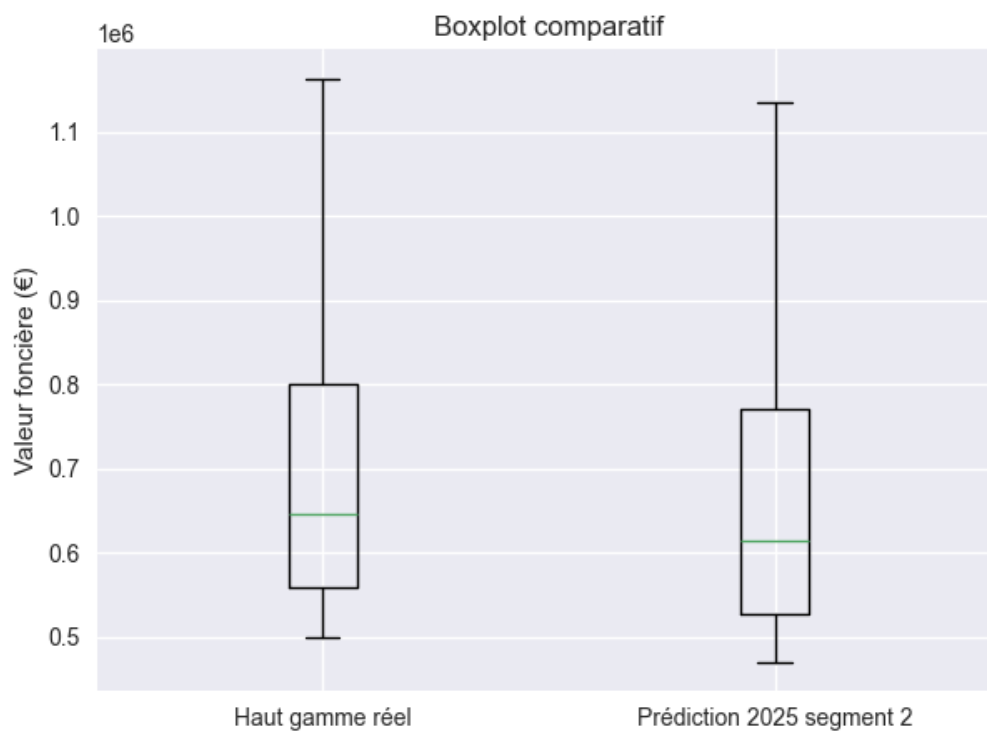
iv. Modélisation et prédiction :

- Entraînement d'un modèle Random Forest sur toutes les données propres disponibles.
- Prédiction des valeurs foncières pour l'année 2025 et ajout des résultats dans le DataFrame.

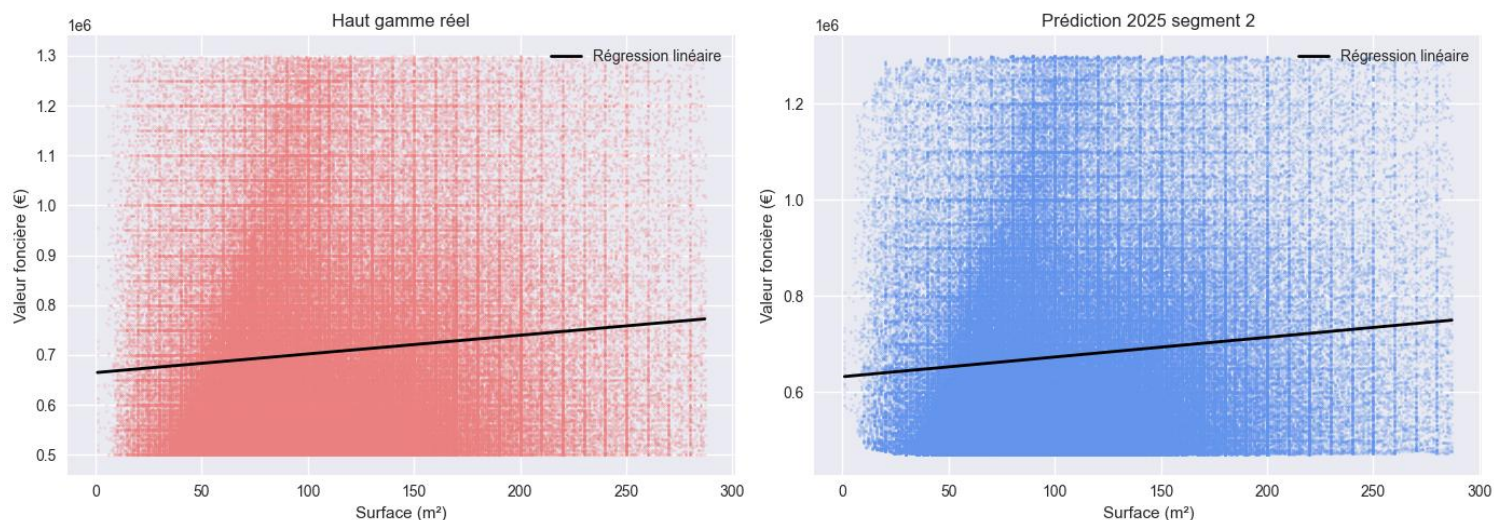
Nous avons réalisé notre prédiction sur les données du segment 2, à savoir les biens haut de gamme. Voici les résultats que nous avons obtenu :



Histogramme de dispersion des valeurs foncières



Boxplot comparatif entre les prédictions de 2025 et les données que nous avons



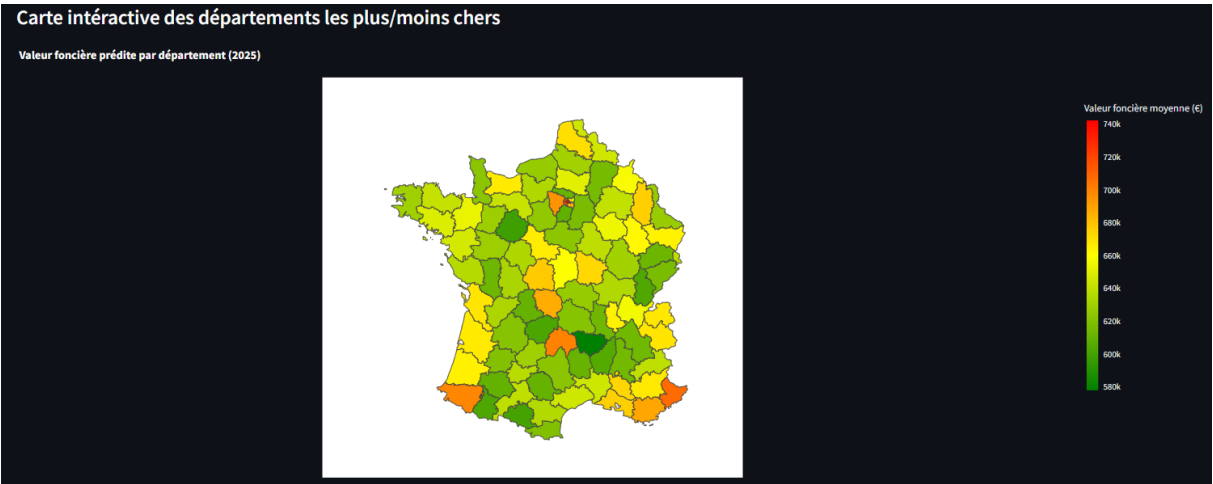
Comparatif des courbes de régression linéaire entre 2025 et l'ensemble de nos données

Pour finir, nous avons exporté nos données dans un fichier CSV, pour que nous puissions ainsi accéder à ces dernières au besoin.

Nous avons, grâce à ce fichier CSV, eu la possibilité de réaliser une courte présentation des données, dans un streamlit dédié à ces dernières :



Explication et présentation des données (Streamlit)



Carte interactive des moyennes des valeurs foncières par départements

Top 5 des plus chers			Top 5 des moins chers		
	departement	valeur_fonciere_predite		departement	valeur_fonciere_predite
72	75	742,142.5596	43	43	577,687.1555
89	92	716,405.3754	69	72	598,055.8715
0	00	715,139.7236	9	09	599,902.8147
6	06	707,672.9448	19	19	602,236.0920
15	15	700,042.0855	64	65	602,874.6962

Tableau comparatif des moyennes de prix par départements

BLOC5 - Concevoir une stratégie de management et de gouvernance de données pour transformer les données en informations créatrices de valeur

Objectifs du bloc :

Définir une gouvernance de données en mettant en place les politiques et les standards afin d'établir les rôles, les responsabilités et la propriété des données
Mettre en place les bonnes pratiques de gestion de données en appliquant les réglementations en vigueur pour protéger les données et respecter la vie privée dans une logique de transparence.
S'assurer que la qualité de la donnée permet d'atteindre les objectifs business en faisant un audit de qualité et en proposant des plans de prévention et de remédiation en cas de non-qualité
Identifier l'approche stratégique pour faire de la donnée un actif central ancré dans la culture de l'entreprise et intégrer l'approche dans un plan de communication diffusé aux concernés
Élaborer une stratégie de sécurité afin de protéger les données en utilisant les technologies adéquates permettant de parer aux menaces et en mettant en place une politique d'accès aux données

Projet de Data Gouvernance – Neocad

Préparation du projet -

A – Réalisation du planning

Étape	Responsable principal	Autres intervenants	Période indicative
1. Recueil & scan des documents terrain	Conducteur de travaux	Opérateurs de chantier	T0 – T0+1 semaine
2. Extraction automatique (OCR)	Responsable Data	Assistant Qualité	T0+1 à T0+2 semaines
3. Correction & validation manuelle	Assistant à maîtrise d'ouvrage	Responsable Qualité	T0+2 à T0+3 semaines
4. Structuration et stockage des données	Responsable Data / IT	DPO, Responsable Sécurité	Continu (dès la saisie validée)
5. Intégration au BIM / GED / Tableaux de bord	Responsable BIM	Responsable Data	Continu, par lot de fiches validées
6. Suivi, archivage, export	Responsable Qualité	IT, DPO	À chaque clôture de chantier
7. Revue qualité & indicateurs	Comité de gouvernance	Tous responsables métiers	Mensuel / Trimestriel

Planning prévisionnel

B – Constitution d'une équipe

Rôles principaux dans la gouvernance des données du projet :

- Responsable Data (Data Owner) : supervise la cohérence, la sécurité et la conformité des données.
- DPO (Data Protection Officer) : garantit la conformité RGPD, le respect de la vie privée, la gestion des accès.
- Responsable Qualité : valide la complétude et la conformité métier des fiches, suit les non-conformités.
- Conducteur de travaux / AMO : saisie et vérification terrain, validation des remontées de chantier.
- Responsable BIM (Modélisation des informations du bâtiment) / IT : assure l'intégration au jumeau numérique et le maintien de la base technique.
- Comité de gouvernance : réunit les personnes ci-dessus + MOA/MOE si besoin, décide des évolutions, veille à la qualité et à la sécurité globale.

Fonctions du comité :

- Définir et valider les politiques de données.
- Arbitrer en cas de conflits (accès, suppression, correction).
- Surveiller les indicateurs de qualité.
- Prévoir les audits internes/externe.

C – Classification des données

Pour chaque donnée/fichier/document géré dans l’outil, appliquer une classification selon la sensibilité :

Niveau de classification	Description	Exemples
Public	Peut être partagé sans restriction	Notices techniques génériques, communiqués
Interne	Pour usage des équipes projet	Comptes-rendus internes, planning de chantier
Confidentiel	Accès limité, données sensibles	Fiches de contrôle nominatives, non-conformités, données RH, accès BIM complet
Très confidentiel / restreint	Accès restreint à certaines personnes autorisées seulement	Litiges, audits, données personnelles, stratégies commerciales, signatures

Chaque fiche ou fichier importé dans l'outil devra porter un tag/classification, et les droits d'accès doivent être strictement appliqués selon cette étiquette.

D - Exemples d’indicateurs de qualité des données

- Taux de fiches complètes : % de fiches contenant tous les champs obligatoires renseignés.
- Taux de non-conformités traitées dans le délai : % de non-conformités clôturées dans le délai imparti.
- Délai moyen de validation des fiches : temps moyen entre création et validation finale.
- Nombre d’erreurs détectées lors de la saisie automatique (OCR) : suivi pour améliorer l’outil.
- Nombre de versions d’une fiche (versioning) : suivi de la stabilité des données (trop de versions = processus perfectible).
- Taux d’accès non autorisés détectés (tentatives ou incidents de sécurité).
- Taux d’intégration BIM réussie : % de fiches correctement synchronisées avec la maquette numérique.

I. Définition de la Structure de Données

1. Modélisation des Données

a. Structure de données optimisée

Pour digitaliser les fiches de contrôle et de non-conformité, la structure doit être flexible, scalable, et adaptée aux différents types de documents (textes, images, signatures, métadonnées).

Je propose une structure orientée « fiche », chaque fiche ayant :

- Des champs structurés : numéro, date, type, intervenants, statut, observations, anomalies, actions correctives, validation, historique des modifications.
- Des pièces jointes : photos, scans, signatures.
- Des liens vers d'autres documents ou vers des éléments du BIM.

Exemple de schéma générique d'une fiche :

Fiche :

- id_fiche (UUID)
- type_fiche (Contrôle / Non-conformité)
- chantier (projet, zone)
- date_creation
- statut (brouillon, validée, clôturée, archivée)
- auteur_creation
- intervenants (liste, rôles)
- description
- anomalies (liste)
- actions_correctives (liste)
- documents_associes (photos, scans)
- signature (nom, image, date)
- historique_modifications (date, auteur, modification)

Ajouts spécifiques pour le BIM :

- id_BIM_element (pour lier la fiche à un objet ou une zone du jumeau numérique)

b. Justification des choix de modélisation

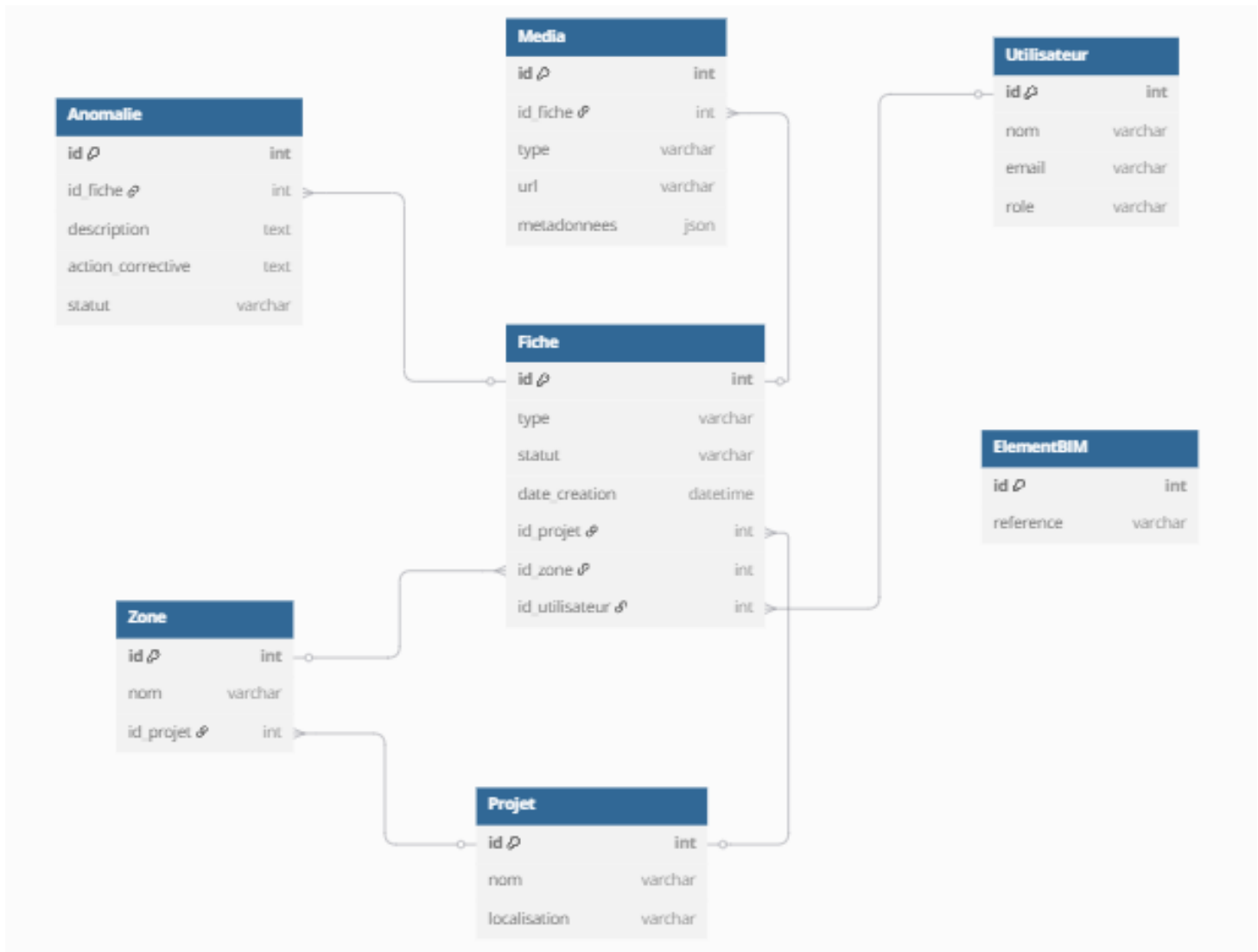
- Relationnelle (SQL) : adaptée pour la structuration forte, les contrôles d'intégrité, la traçabilité et les jointures (ex : fiche <-> chantier <-> intervenants).

- NoSQL (documentaire, ex: MongoDB) : plus souple pour stocker des fiches aux champs évolutifs, des pièces jointes, et gérer l'historique ou les modifications non linéaires.
- Hybride :
 - Base relationnelle pour les éléments maîtres (projets, utilisateurs, droits, référentiels).
 - Base NoSQL pour les fiches « vivantes » (contrôle, non-conformité, historiques, médias liés).
- Pourquoi ? : Les documents PDF, images, scans ne sont pas naturellement structurés. Le NoSQL facilite l'indexation, la recherche plein texte et l'intégration rapide de nouveaux champs. Le relationnel assure la cohérence et la gestion des droits.

c. Modèle conceptuel et logique

Modèle conceptuel (simplifié) :

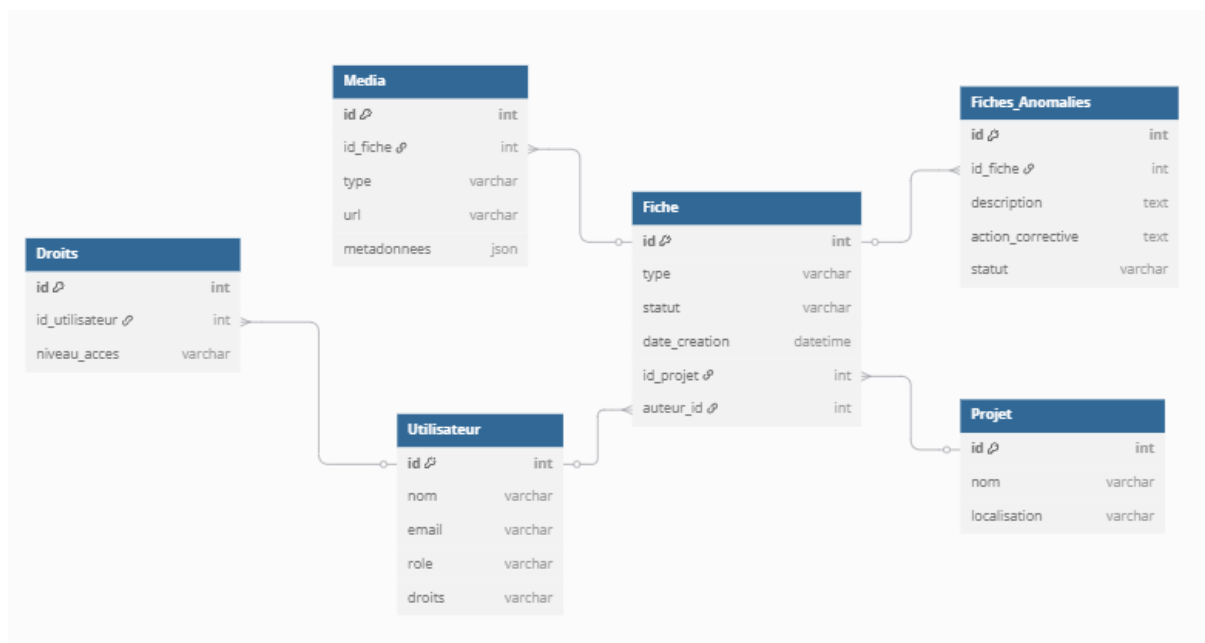
- Fiche — appartient à → Projet/Chantier
- Fiche — concerne → Zone
- Fiche — initiée/modifiée par → Utilisateur
- Fiche — contient → Anomalie(s)
- Fiche — liée à → Document(s) média (photo, PDF, signature)
- Fiche — liée à → Élément BIM (optionnel)
- Historique — pour audit et traçabilité



MLD Simplifié en utilisant dbdiagram.io

Modèle logique (exemple hybride SQL + NoSQL) :

- Table Utilisateurs (id, nom, rôle, droits, email, etc.)
- Table Projets (id, nom, localisation, etc.)
- Table Fiches (id, type, id_projet, statut, auteur, date, etc.)
- Table Fiches_Anomalies (id_fiche, description, action_corrective, statut, etc.)
- Table Droits (id_utilisateur, niveau_accès, etc.)
- Table Médias (id, id_fiche, type, url, métadonnées, etc.)
- Collection NoSQL FicheDetails (tous les champs « flexibles », logs, images, commentaires, versioning)



MLD hybride réalisé sur dbdiagram.io

d. Dictionnaire des données

Fiche de contrôle :

Nom du champ	Type	Description	Valeurs attendues / Règles	Entité liée
id_fiche	UUID	Identifiant unique de la fiche	Généré automatiquement	-
type_fiche	Texte	Type de fiche	Contrôle, Non-conformité, Observation	Référentiel types de fiches
chantier	Texte / Référence	Nom ou code du chantier concerné	Doit correspondre à une entrée valide dans la table Projets	Projets
zone	Texte	Zone du chantier ou localisation précise	Format libre, standardisé par gabarit	-
date_creation	Date	Date de création de la fiche	Format JJ/MM/AAAA, non vide	-
statut	Texte	Statut de traitement de la fiche	Brouillon, Validée, Clôturée, Archivée	Référentiel statuts

auteur_creation	Référence	Utilisateur ayant créé la fiche	Clé étrangère vers Utilisateurs	Utilisateurs
intervenants	Liste (références)	Liste des personnes impliquées	Liste de Utilisateurs avec rôle précisé (ex. AMO, Conducteur, etc.)	Utilisateurs
description	Texte long	Description libre de l'observation ou de l'anomalie	Champ texte obligatoire	-
anomalies	Liste (objets)	Liste des anomalies détectées	Objet : description, type, gravité, statut	Référentiel anomalies
actions_correctives	Liste (objets)	Liste des actions prévues ou effectuées pour corriger	Objet : action, responsable, date, état d'avancement	-
documents_associes	Liste (fichiers)	Liens vers les pièces jointes	Formats acceptés : pdf, jpg, png, taille max 10 Mo	Médias
signature	Objet	Signature électronique (ou scan)	nom_signataire, image, date_signature	-
historique_modifications	Liste (objets)	Historique des changements sur la fiche	date, auteur, champ modifié, ancienne valeur, nouvelle valeur	-
id_BIM_element	Texte / Référence	Lien vers l'élément du jumeau numérique BIM	Correspond à un identifiant IFC ou équivalent	Eléments_BIM

Données utilisateurs :

Nom du champ	Type	Description
id_utilisateur	UUID	Identifiant unique
nom	Texte	Nom complet de l'utilisateur
email	Texte	Adresse email professionnelle
rôle	Texte	Rôle principal dans le projet
droits (niveau d'accès)	Texte	Niveau d'accès aux données

Projets / Chantiers :

Nom du champ	Type	Description	Valeurs attendues / Règles
id_projet	UUID	Identifiant du chantier	Clé de référence
nom_projet	Texte	Intitulé du projet	Obligatoire
localisation	Texte	Adresse ou zone géographique	Normalisé (ville, code postal)
statut	Texte	Avancement du chantier	En cours, Clôturé, En pause, Archivé

Médias :

Nom du champ	Type	Description	Valeurs attendues / Règles
id_media	UUID	Identifiant unique du fichier	Généré automatiquement
id_fiche	Référence	Fiche associée au fichier	Clé étrangère
type	Texte	Type de fichier	photo, signature, document, etc.
url	Texte	Lien vers le fichier (stocké dans le cloud ou localement)	Chemin ou URI valide
métadonnées	JSON	Données sur le fichier (poids, dimensions, etc.)	Facultatif

2. Provenance et Standardisation des Données

- Sources identifiées :
 - Formulaires papier (scannés ou photos)
 - PDF générés automatiquement ou scannés
 - Images (prises sur le chantier)
 - Saisie manuelle (corrections ou ajouts)
- Standardisation :
 - Imposer des modèles de fiches numériques avec des champs obligatoires et un workflow de validation.
 - Stocker les données extraites sous forme normalisée (dates, formats des anomalies, catégories de non-conformités, etc.).

- Liaison avec référentiels (DTU, réglementations, objets BIM).
- Assurer la compatibilité avec les standards du BIM (IFC, etc.), pour la synchronisation.

3. Gouvernance des Données et Conformité

Droits et rôles :

- Création : conducteur de travaux, AMO, chef de chantier.
- Modification : auteur ou supérieur hiérarchique, AMO, conducteurs, responsables qualité.
- Validation : responsable qualité, AMO, MOE selon workflow.
- Suppression : restreinte (responsable data, DPO, ou sur justification et validation supérieure).
- Consultation : parties prenantes du chantier, accès restreint par profil, accès en lecture pour audit ou MOA.

Archivage et mise à jour :

- Archivage automatique des fiches clôturées, versionning des documents (toute modification est tracée, aucun effacement pur).
- Délais de conservation selon normes (minimum 10 ans pour des documents légaux dans le BTP).
- Export possible pour audit, transfert ou intégration dans le DOE (dossier des ouvrages exécutés) numérique.

Conformité RGPD & confidentialité :

- Stockage sécurisé (cloud ou sur site, data centers UE pour la France).
- Accès logué et traçable, gestion des habilitations.
- Droit à l'oubli et anonymisation si données à caractère personnel.
- Confidentialité renforcée pour les informations sensibles (par ex, accès par chantier ou par zone, double authentification pour les responsables).

Données maîtres :

Dans le cadre du projet, les données maîtres ont été identifiées afin d'assurer la cohérence globale du système d'information. Il s'agit principalement :

- des projets et chantiers (référentiels de localisation, d'identification et de phasage),
- des utilisateurs (conducteurs, responsables, intervenants, associés à des rôles et habilitations),
- des référentiels métier (types de fiches, anomalies, statuts, niveaux de sensibilité, etc.),

- des éléments BIM (objets du jumeau numérique servant de point d’ancrage des fiches). Ces données servent de socle à l’ensemble des processus de collecte, de traitement et d’analyse, et sont rigoureusement versionnées et auditées.

II. Collecte et Organisation des Données

Automatisation de la lecture :

- Pipeline de traitement automatique :
 1. OCR (Reconnaissance optique de caractères) pour extraire le texte des PDF ou images.
 2. Détection de zones clés (numéro de fiche, date, intervenants, zones d’observation, etc.) via modèles de reconnaissance et de classification.
 3. Extraction des images (photos prises sur le chantier, signatures) en tant que fichiers liés à la fiche.
 4. Structuration des données dans la base NoSQL/documentaire.

Identification des champs clés :

- Numéro de fiche (unique)
- Date de création
- Type (contrôle, non-conformité...)
- Projet, zone, lot concerné
- Auteur et intervenants
- Description / Observations
- Anomalies / défauts constatés
- Actions correctives proposées
- Statut et validation (workflow)
- Pièces jointes (photos, signatures)
- Historique (toutes modifications, validations)

Gestion des erreurs et saisie manuelle :

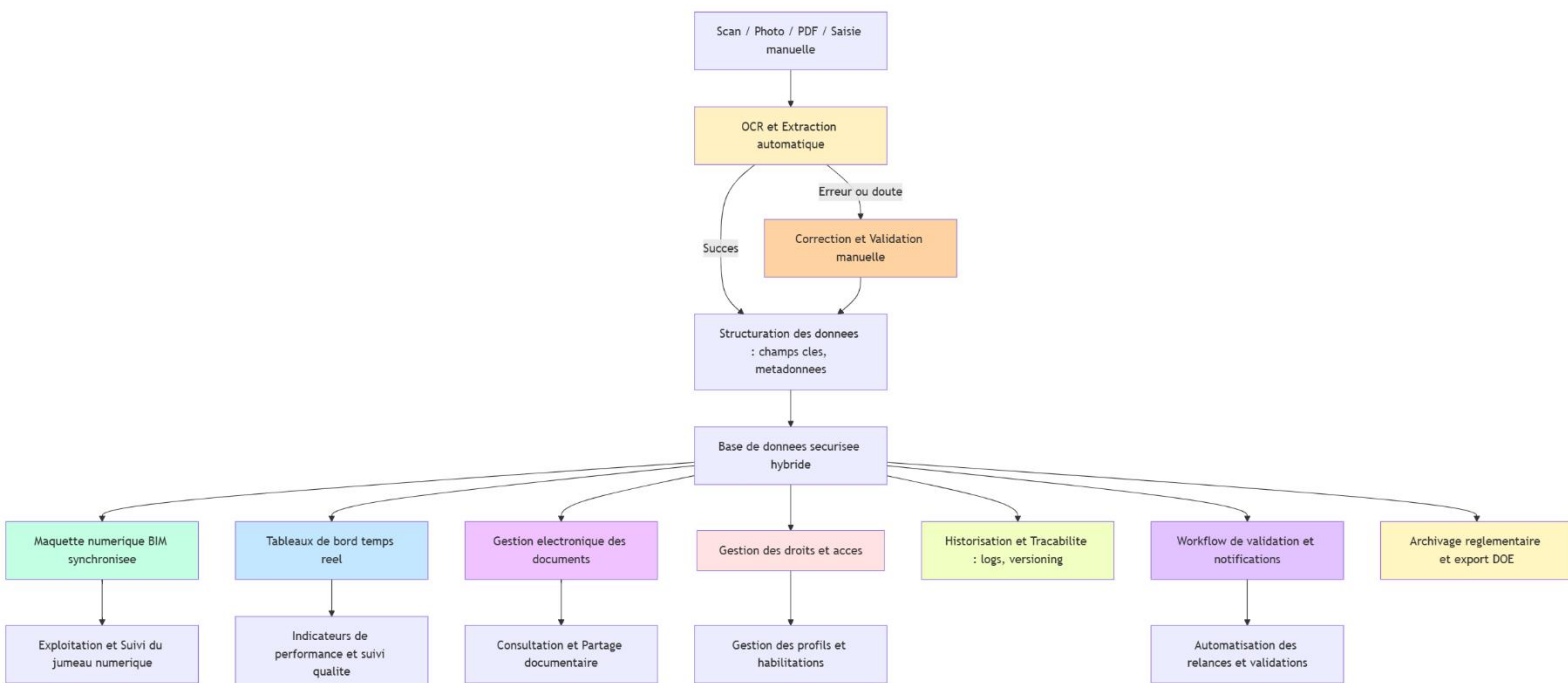
- Détection automatique d’anomalies de lecture (ex : texte illisible, champs manquants, incohérences).
- Interface de correction manuelle :

- Permet à un utilisateur habilité de compléter, corriger ou valider les données extraites.
- Chaque modification est historisée (traçabilité totale).
- Notifications pour validation ou correction en attente.
- Workflow intégré : rien ne part en base "validée" sans revue humaine si un doute est détecté.

Schéma général d'une solution technique

1. Collecte automatique (OCR + IA)
2. Stockage temporaire des fiches en attente de validation
3. Saisie/correction manuelle si erreur
4. Intégration dans la base de données (relationnelle + NoSQL)
5. Contrôle d'accès et droits selon les profils
6. Archivage, traçabilité, export, interopérabilité BIM

Résumé graphique du flux



Maquette réalisée sur Mermaid

III. Stratégie de gestion des données et sécurité

1. Plan de communication et de sensibilisation des collaborateurs :

a. Plan de communication et culture de la donnée :

Une stratégie de communication interne a été mise en place afin d’ancrer la culture de la donnée dans les pratiques quotidiennes des équipes. Ce plan repose sur plusieurs actions :

- Sensibilisation initiale : présentation des enjeux de qualité, de sécurité et de conformité des données lors des réunions de lancement de projet.
- Formations ciblées : sessions pratiques à destination des conducteurs de travaux, assistants qualité et équipes IT sur la saisie correcte, la classification et la protection des données.
- Supports de référence : mise à disposition de guides numériques, fiches réflexes et tutoriels expliquant les workflows de validation, les niveaux d’accès et les bonnes pratiques de gestion.
- Communication continue : newsletters internes, indicateurs partagés sur les tableaux de bord qualité, rappels sur les droits et responsabilités dans la gouvernance des données.

Ce plan vise à faire comprendre que la donnée n’est pas un simple "produit de saisie", mais un actif stratégique collectif, garant de la traçabilité, de la sécurité et de la performance des chantiers.

b. Modèle de maturité des données :

Un modèle de maturité a été étudié puis utilisé pour mesurer l’avancement du projet en matière de gestion des données. Il s’inspire du cadre DAMA-DMBOK et des bonnes pratiques issues des modèles CMMI.

Ce modèle repose sur cinq niveaux progressifs :

Niveau	Description
1 – Initial	Les données sont gérées de manière ad hoc, sans règles partagées.
2 – Répétable	Des procédures commencent à être définies localement.
3 – Défini	Des standards communs sont appliqués à l’échelle du projet.
4 – Contrôlé	La qualité des données est mesurée et suivie régulièrement.
5 – Optimisé	Les données sont pilotées stratégiquement et intégrées dans les décisions.

Lors de la mise en œuvre de Neocad, le projet s’est structuré pour atteindre un niveau 3, avec des objectifs d’évolution vers les niveaux 4 et 5 via des outils de data quality, des audits réguliers, et une intégration plus poussée dans la gestion de projet BIM.

2. Sécurité des données

a. Maintenance et sécurisation :

Nous avons également mis en place un plan de réponse aux incidents, en cas de cyberattaque, de tentative d'accès non autorisé ou de perte de données.

Ce plan prévoit plusieurs étapes clés :

- la détection des comportements anormaux grâce à des logs d'accès et des alertes automatiques,
- une analyse rapide de l'incident par le Responsable Sécurité et le DPO,
- une réaction immédiate, avec blocage des accès concernés, notification aux utilisateurs, et le cas échéant, déclaration à la CNIL dans les 72h si des données personnelles sont compromises.
Une fois l'incident clos, un retour d'expérience est réalisé pour améliorer en continu le dispositif.

En parallèle, une veille technique est assurée par les équipes IT et conformité. Cette veille repose notamment sur les recommandations de l'ANSSI (Agence nationale de la sécurité des systèmes d'information) et s'inspire des référentiels tels que l'ISO 27001 (norme internationale de sécurité des SI). Elle permet de suivre les mises à jour critiques, de détecter les nouvelles menaces, et d'adapter nos procédures en fonction des évolutions technologiques et réglementaires.

Cela garantit que la stratégie de sécurité des données reste vivante, proactive et conforme aux meilleures pratiques du secteur.

Conclusion :

Cette solution propose une gouvernance de la donnée robuste (sécurité, traçabilité, conformité RGPD), une structuration hybride adaptée au terrain et au BIM, une gestion automatisée mais contrôlée par l'humain en cas de doute, et la capacité d'intégration à des processus métiers du BTP de bout en bout.

Bibliographie :

Documents internes et exemples métiers :

1. neocad - Septembre 2023

Présentation interne du projet Neocad, enjeux de la digitalisation du suivi opérationnel en construction, analyse du marché BIM, proposition de solution SaaS et architecture fonctionnelle.

- Fichier : *neocad - Septembre 2023_20230918_103739_0000.pdf*

2. Fiche de Contrôle – Chantier CDG1

Exemple de fiche de contrôle utilisée pour le suivi qualité et la traçabilité sur le chantier de l'Aéroport Roissy Charles-de-Gaulle.

- Fichier : *106-Révolue_A_FC_GO_Zone 16_ 1 Fiche de Contrôle.pdf*

3. Plan technique – St Cyr - ZONE C - PH R+2

Plan technique détaillé d'un chantier (zone C, phase R+2), présentant la méthodologie, la conformité des matériaux, et les intervenants.

- Fichier : *2020 - St Cyr - ZONE C - PH R+2- ind 0.pdf*

4. Coupes techniques – St Cyr - Zone C

Croquis/coupes techniques détaillées pour la réalisation du bâti, servant de support documentaire.

- Fichier : *2021 - St Cyr - Coupes - Zone C - PH R+2- ind 00.pdf*

5. Exemple de fiche de non-conformité

Modèle de fiche permettant de signaler, suivre et archiver les non-conformités sur chantier.

- Fichier : *Exemple fiche de non-conformité.pdf*

Outils de modélisation et visualisation de données :

6. dbdiagram.io

Générateur en ligne de MCD/MLD, utilisé pour modéliser les schémas de données relationnelles et hybrides.

- <https://dbdiagram.io/>

```
Table Projet {
  id int [pk, increment]
  nom varchar
  localisation varchar
}

Table Zone {
  id int [pk, increment]
  nom varchar
  id_projet int [ref: > Projet.id]
}

Table Utilisateur {
  id int [pk, increment]
  nom varchar
  email varchar
  role varchar
}

Table Fiche {
  id int [pk, increment]
  type varchar
  statut varchar
  date_creation datetime
  id_projet int [ref: > Projet.id]
  id_zone int [ref: > Zone.id]
  id_utilisateur int [ref: > Utilisateur.id]
}

Table Anomalie {
  id int [pk, increment]
  id_fiche int [ref: > Fiche.id]
  description text
  action_corrective text
  statut varchar
}

Table Media {
  id int [pk, increment]
  id_fiche int [ref: > Fiche.id]
  type varchar
  url varchar
  metadonnees json
}

Table ElementBIM {
  id int [pk, increment]
  reference varchar
}
```

Code utilisé pour le MLD simplifié


```
Table Utilisateur {
  id int [pk, increment]
  nom varchar
  email varchar
  role varchar
  droits varchar
}

Table Projet {
  id int [pk, increment]
  nom varchar
  localisation varchar
}

Table Fiche {
  id int [pk, increment]
  type varchar
  statut varchar
  date_creation datetime
  id_projet int [ref: > Projet.id]
  auteur_id int [ref: > Utilisateur.id]
}

Table Fiches_Anomalies {
  id int [pk, increment]
  id_fiche int [ref: > Fiche.id]
  description text
  action_corrective text
  statut varchar
}

Table Droits {
  id int [pk, increment]
  id_utilisateur int [ref: > Utilisateur.id]
  niveau_acces varchar
}

Table Media {
  id int [pk, increment]
  id_fiche int [ref: > Fiche.id]
  type varchar
  url varchar
  metadonnees json
}
```

Code utilisé pour le MLD hybride

7. Mermaid Live Editor

Générateur en ligne de diagrammes de flux (workflow) à partir de code texte.

- <https://mermaid.live/>

```
graph TD
    A[Scan / Photo / PDF / Saisie manuelle] --> B[OCR et Extraction automatique]
    B -->|Succes| C[Structuration des donnees : champs cles, metadonnees]
    B -->|Erreur ou doute| D[Correction et Validation manuelle]
    D --> C

    C --> E[Base de donnees securisee hybride]
    E --> F[Maquette numerique BIM synchronisee]
    E --> G[Tableaux de bord temps reel]
    E --> H[Gestion electronique des documents]
    E --> I[Gestion des droits et acces]
    E --> J[Historisation et Tracabilite : logs, versioning]
    E --> K[Workflow de validation et notifications]
    E --> L[Archivage reglementaire et export DOE]

    F --> M[Exploitation et Suivi du jumeau numerique]
    G --> N[Indicateurs de performance et suivi qualite]
    H --> O[Consultation et Partage documentaire]
    I --> P[Gestion des profils et habilitations]
    K --> Q[Automatisation des relances et validations]

    style B fill:#fff1c2
    style D fill:#ffd2a2
    style F fill:#c2ffe3
    style G fill:#c2e7ff
    style H fill:#f0c2ff
    style I fill:#ffe2e2
    style J fill:#f2ffc2
    style K fill:#e2c2ff
    style L fill:#fff6c2
```

Code utilisé pour produire la maquette Mermaid